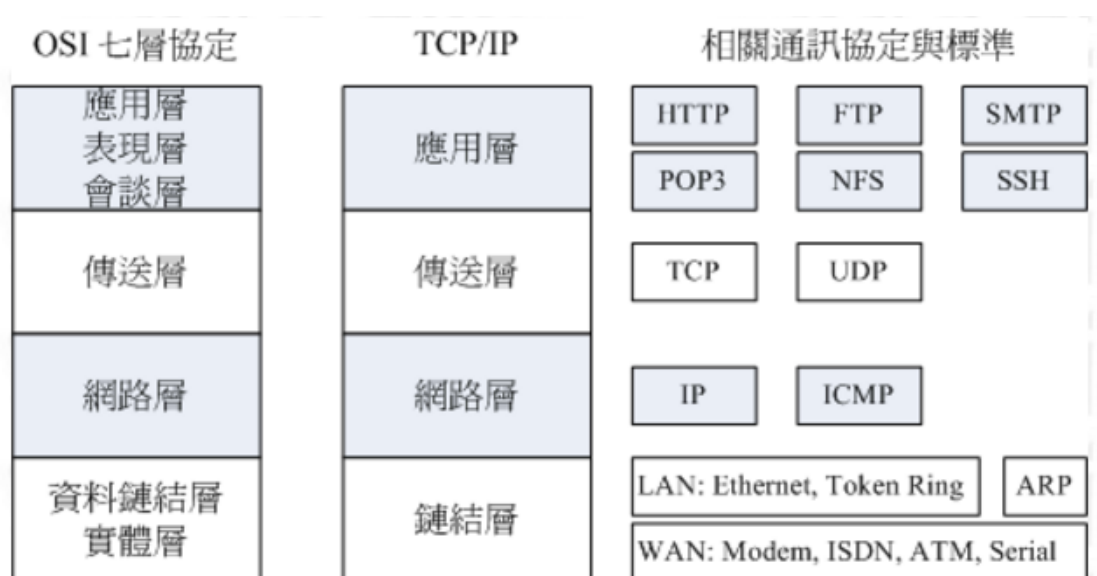


第五章 MQTT	2
5-1 TCP/IP vs WebSocket vs MQTT	2
TCP/IP vs WebSocket vs MQTT.....	8
5-2 MQTT 架設 in Windows	13
新增 Mosquitto 的使用者帳號和密碼.....	16
在 mosquitto.conf 中加入使用者帳密檔的路徑.....	17
手動執行 Server	25
防火牆設定.....	25
測試程式 MQTTX.....	32
5-3 MQTT 架設 in Linux.....	36
第六章 ESP8266	38
6-1 環境安裝	39
在 Arduino IDE 設定 NodeMCU:	40
6-2 Sample Code	43
6-3 Getting Started With the ESP8266 and DHT22 Sensor	44
6-4 Sending DHT22 Sensor Data from ESP8266 Using MQTT	50
6-5 建立 Django 顯示溫濕度系統.....	59
6-6 建立 Android 顯示溫溼度	64
顯示一個主題的溫溼度.....	64
顯示一個主題的溫溼度+歷史訊息.....	69
顯示三個主題的溫溼度.....	75
顯示三個主題的溫溼度+歷史訊息.....	75
6-7 ESP8266 and DHT22 Sensor Using HTTP POST Request.....	76
6-8 ESP8266 with 4-Channel Relay and Sensor Integration using MQTT	78
6-9 建立 Django 控制 4 pin relay	91
6-10 建立 Android 控制 4 pin relay	100
6-11 建立 Android 控制 4 pin relay	109

第五章 MQTT

5-1 TCP/IP vs WebSocket vs MQTT

按照 OSI 網路分層模型，IP 是網路層協定(Protocol)，TCP 是傳輸層協定，而 HTTP 和 MQTT 是應用層的協定。在這三者之間，TCP 是 HTTP 和 MQTT 底層的協定。大家對 HTTP 很熟悉，這裡簡要介紹下 MQTT。MQTT (Message Queuing Telemetry Transport，消息隊列遙測傳輸) 是 IBM 開發的一個即時通訊協議，有可能成為物聯網的重要組成部分。該協定支持所有平台，幾乎可以把所有聯網物品和外部連接起來，被用來當做傳感器的通信協議。

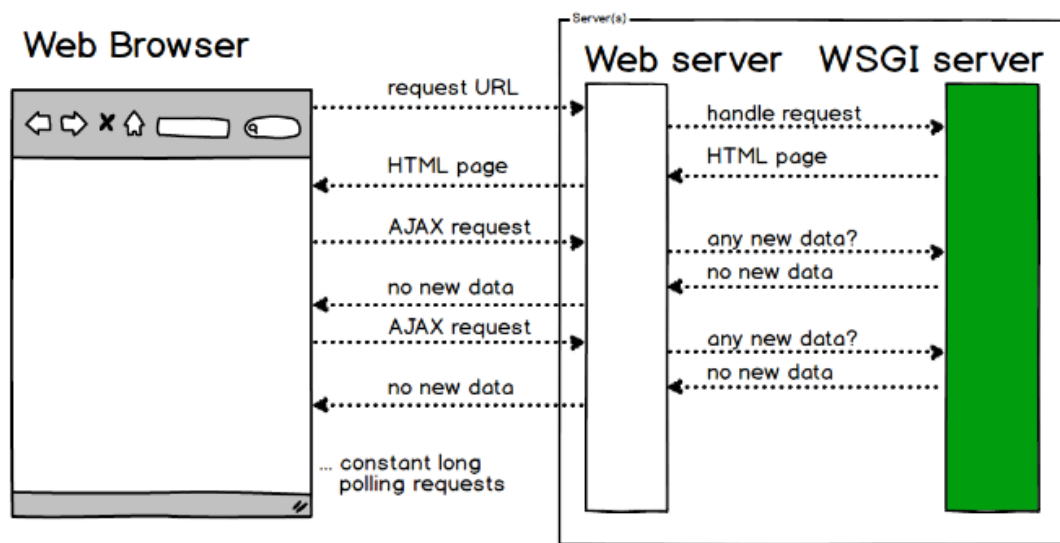


1、HTTP 的不足

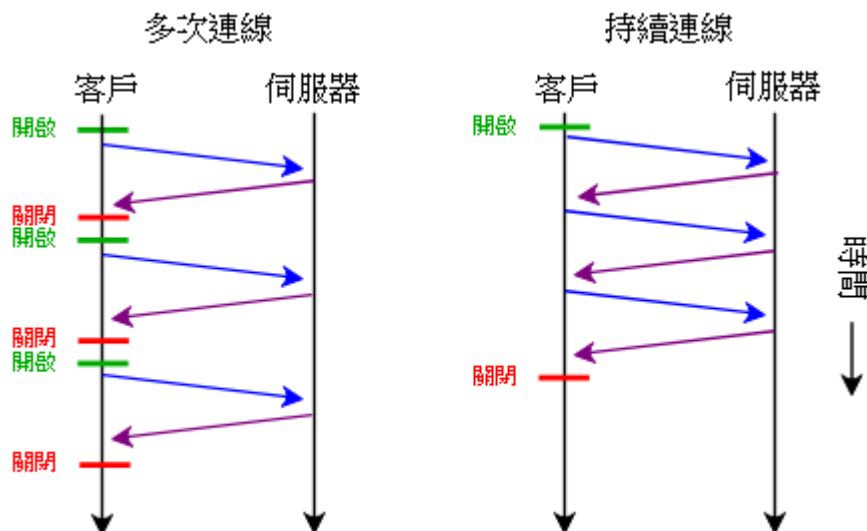
HTTP 協定經過多年的使用，發現了一些不足，主要是性能方面的，包括：HTTP 的連接問題，HTTP 客戶端和服務器之間的交互是採用請求/應答模式，在客戶端請求時，會建立一個 HTTP 連接，然後發送請求消息，服務端給出應答消息，然後連接就關閉了（後來的 HTTP1.1 支持持久連接），因為 TCP 連接的建立過程是有開銷的，如果使用了 SSL/TLS 開銷就更大。

在瀏覽器裡，一個網頁包含許多資源，包括 HTML，CSS，JavaScript 中，圖片等等，這樣在加載一個網頁時要同時打開連接到同一服務器的多個連接。

Long polling via AJAX



持續連線：

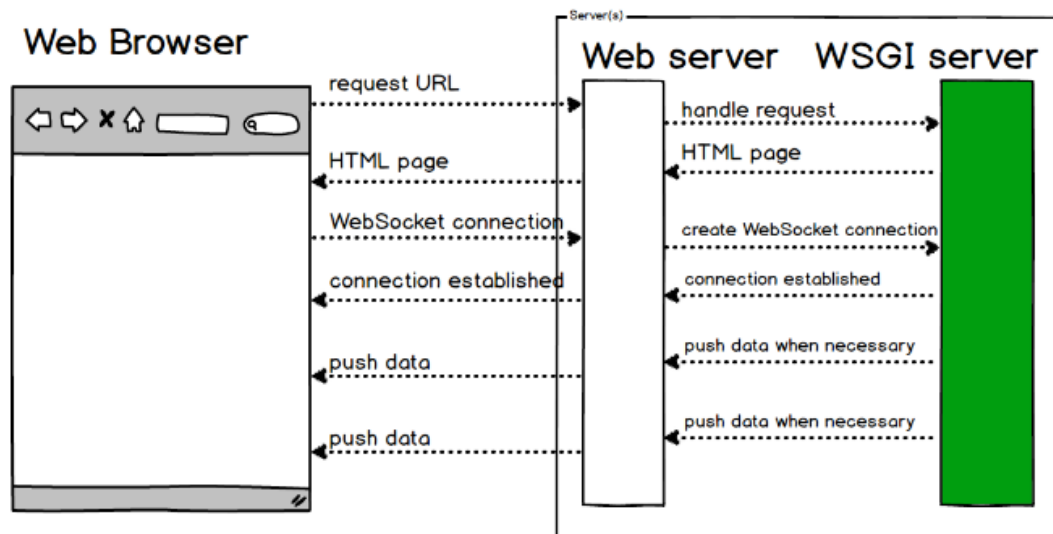


2、WebSocket

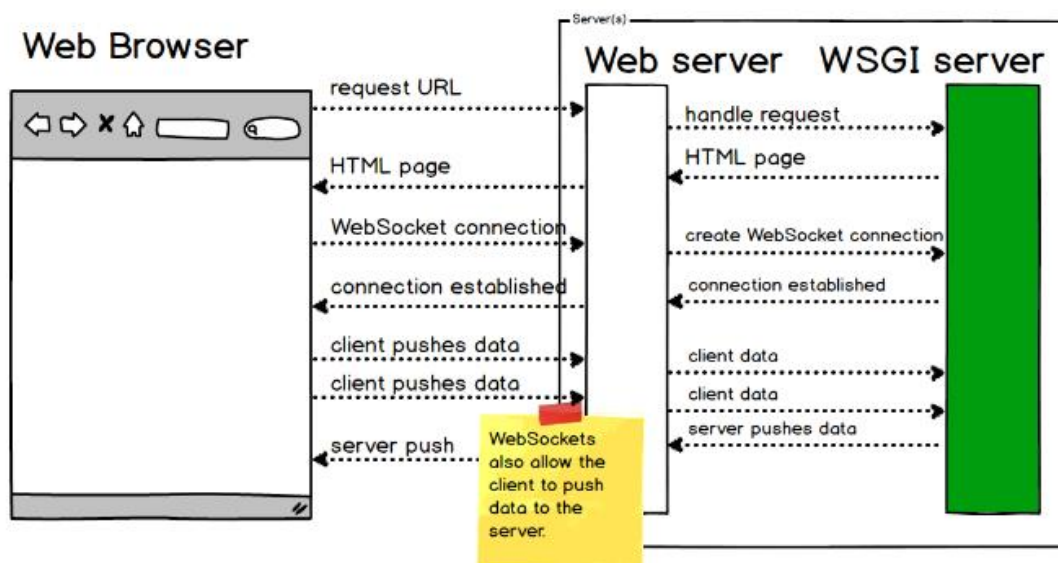
WebSocket 則提供使用一個 TCP 連接進行雙向通訊的機制，包括網絡協定和 API，以取代網頁和服務器採用 HTTP 輪詢進行雙向通訊的機制。

本質上來說，網頁套接字是不限於 HTTP 協定的，但是由於現存大量的 HTTP 基礎設施，代理，過濾，身份認證等等，WebSocket 的借用 HTTP 和 HTTPS 的端口。由於使用 HTTP 的端口，因此 TCP 連接建立後的握手消息是基於 HTTP 的，由服務器判斷這是一個 HTTP 協議，還是 WebSocket 協議。WebSocket 連接除了建立和關閉時的握手，數據傳輸和 HTTP 認何關係了。

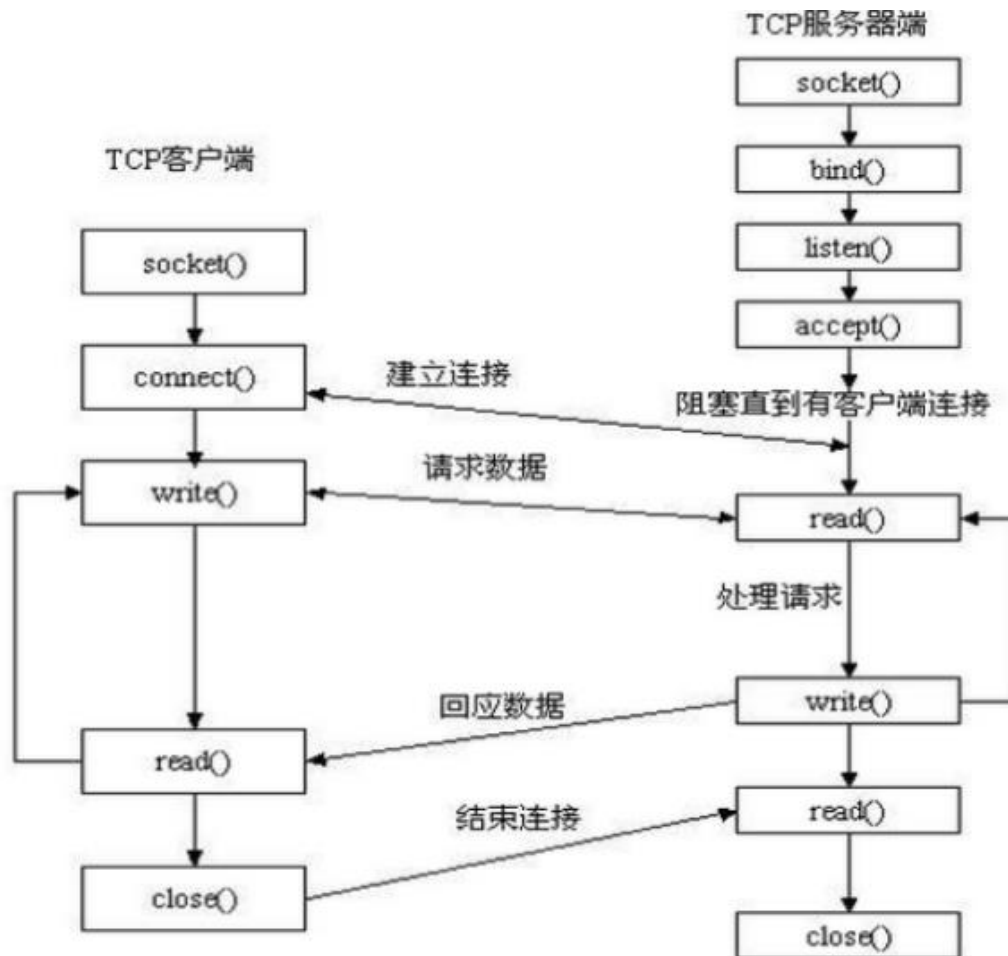
WebSockets



WebSockets



<http://www.ruanyifeng.com/blog/2017/05/websocket.html>



3、MQTT(Message Queuing Telemetry Transport)

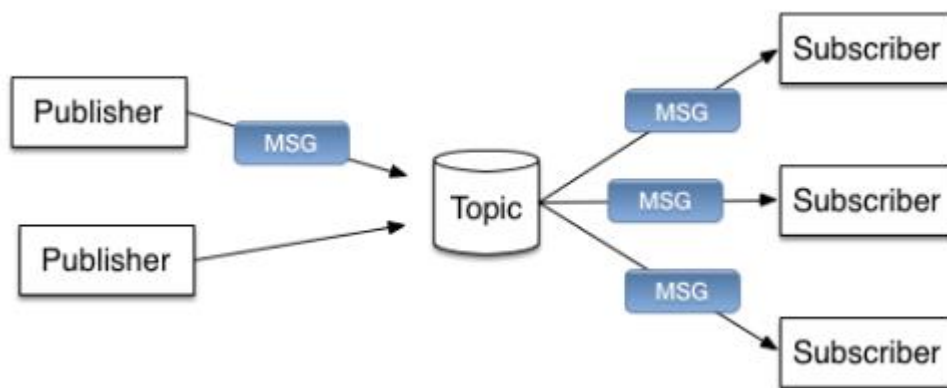
簡單來說，它是為了物聯網而設計的 **protocol**，並且它是透過 **publish/subscribe** 的方式來做訊息傳送。由於是為了物聯網而設計的協定，因此它所需要的網路頻寬是很低的，而所需要的硬體資源也是低的。

Publish/Subscribe：

在看 MQTT 之前，最好要先知道 Publish/Subscribe 的訊息傳送機制為何，這樣之後在看其協定時，才會更快上手。Publish/Subscribe 有三種主要的組成元件，分別為 Publisher、Subscriber 以及 Topic。

Publisher 為訊息的來源，它會將訊息發送給 Topic，而 Subscriber 向 Topic 註冊，表示他們想要接收此 Topic 的訊息；因此當有某個 Publisher 對 Topic 發送訊息時，只要有對此 Topic 註冊的 Subscriber，都會收到此則訊息。

它們的關係如下圖：



MQTT 特性：

- 1、Publish/Subscribe 的訊息傳送模式，來提供一對多的訊息分配。
- 2、使用 TCP/IP 來提供基本的網路連結。
- 3、三種訊息傳送服務的 qualities：
 - "At most once"，最多一次，訊息遺失或是重複發送的狀況可能會發生；這種 quality 適合應用在環境感測，不在意資料是否會遺失，因為下一次的資料取樣很快就會被 published 出來。
 - "At least once"，至少一次，這種 quality 保證訊息會送達，只是可能會發生重複發送訊息的狀況。
 - "Exactly once"，確定一次，確認訊息只會送到一次。這種 quality 適合用在計費系統，系統只要有重複收到資料、或是資料遺失狀況發生，就會造成系統錯誤。
- 4、由於他的 header 固定長度為 2byte，因此可以減少封包傳送時的額外負載，並減少所需的網路頻寬。
- 5、當異常斷線發生時，會使用最後遺囑(Last Will and Testament)的機制，通知各個感興趣的 client。

MQTT 現況：

MQTT 現階段並不是一個標準化的 Protocol，還在持續改進中，目前為 MQTTV3.1。不過 IBM 已於 2013 年已經將它交給 OASIS 進行標準化了，並且一直以來 IBM 對此協定採開放、免授權費的方式讓它能夠被散佈，因此相信不久的將來會成為一個主流的 Protocol。

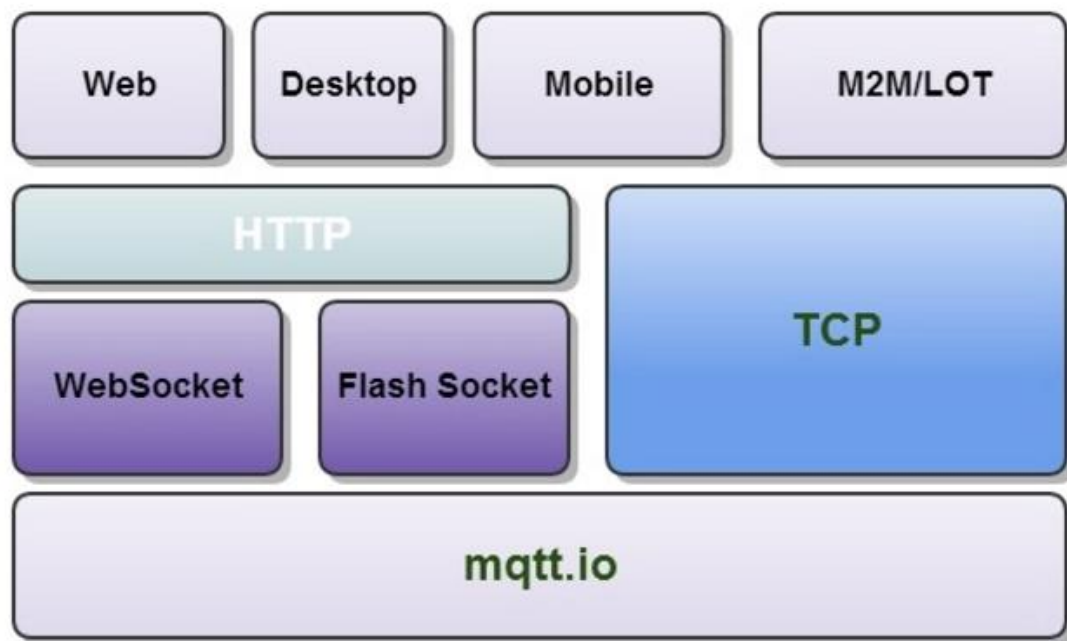
而目前支援 MQTT 的 Client API，有 Eclipse Phono Project 有對 MQTT client 支援，其支援 C、Java、Javascript、C++ 等等的語言，可說是支援度很高的 Project。而目已經在應用 MQTT 的，最知名的應該就是 Facebook Message App 了吧。

Example:

<https://thingspeak.com/>

4、比較

MQTT 和 TCP、WebSocket 的關係可以用下圖表示：



在 Internet 上最通用的協定就是網頁的 HTTP Protocol 了，所以你只要想到需要有 client server 的架構，你就會想到 HTTP，搭配 RESTful 的 API 設計（GET、POST、DELETE、PUT），就能實現無所不能的查詢、新增、刪除、修改。

這樣就夠解決所有的網路問題嗎？當然不能，但是幾乎。

最常見的難題就是聊天室，需要即時通訊的場合，HTTP 是一個簡單但是笨重的協定，一問一答與全純文字的傳輸，讓他的每一個 request 都需要承擔無法丟棄的包袱（在 HTTP/2 試圖解決的問題之一）。所以為了解決 server push 的問題，在傳統架構下可以用 polling 或 long polling 來盡量克服，在 Web 上則有 Websocket 等協定出現解決此問題。

而 MQTT(message queuing telemetry transport) 則是在物聯網界的 WebSocket + Message queuing 解決方案。

讓你能盡可能即時又省資源的情況下，確保訊息被 Push 至移動裝置。他的設計是發佈/訂閱模式(Publish subscribe pattern)，讓你能對特定裝置(broker)訂閱主題，並能盡可能的以節省資源的方式確保有發送到訂閱的 client 上。

文獻：

<http://www.cnblogs.com/shanyou/p/4085802.html>

<http://blog.maxkit.com.tw/2014/01/mqtt.html>

<https://www.fullstackpython.com/websockets.html>

➤ TCP/IP vs WebSocket vs MQTT

TCP/IP vs WebSocket vs MQTT
對於iot系統架構角度的優缺

從 IoT 系統架構角度來看，TCP/IP、WebSocket、MQTT其實是在「不同層級」的技術，不能完全當作同一層的替代品，但可以用來比較它們在 IoT 應用中的角色與優缺點。

我用「架構設計」的觀點幫你整理，讓你可以直接拿來決定系統怎麼選 🐣

◆ 一、技術定位（先釐清層級）

技術	層級	本質
TCP/IP	傳輸層	最底層通訊協議
WebSocket	應用層（基於 TCP）	全雙工即時通訊
MQTT	應用層（基於 TCP）	輕量級 Pub/Sub 協議

🐣 重點：

- TCP/IP 是「基礎」
- WebSocket / MQTT 是「應用協議」

◆ 二、IoT 架構角度比較

1 TCP/IP (直接用 Socket)

👍 優點

- 完全控制 (客製化協議)
- 效能最佳 (沒有額外協議開銷)
- 適合嵌入式/低階設備

👎 缺點

- 要自己設計 protocol (很麻煩)
- 沒有內建：
 - 訂閱機制
 - 訊息管理
 - QoS
- 難維護、難擴展

📌 適合場景

- MCU (Arduino / STM32)
- 超低資源設備
- 專用封閉系統

2 WebSocket

👍 優點

- 全雙工 (雙向即時)
- 比 HTTP 輕量 (建立後不需反覆請求)
- 瀏覽器原生支援 (前端很方便)
- Django / Node.js 很容易整合

👎 缺點

- 沒有 Pub/Sub (要自己做)
- 沒有 QoS
- 不適合大量設備 (Scaling較難)
- 每個 client 都要維持連線

📌 適合場景

- Web Dashboard (即時畫面)
- Chat / 即時資料顯示
- IoT 前端顯示層 (你現在 Django 很適合)

3 MQTT (IoT主流)

👍 優點 (IoT最強)

- Pub/Sub 架構 (解耦)
- 非常省頻寬 (header 小)
- QoS 支援 (0 / 1 / 2)
- 支援：
 - retain message
 - last will
- 非常適合大量裝置

👎 缺點

- 需要 broker (Mosquitto / EMQX)
- 不適合直接給瀏覽器 (需 WebSocket bridge)
- 架構多一層

📌 適合場景

- 感測器 → 伺服器
- IoT 大量設備
- 工業 / 智慧家庭



◆ 三、IoT 架構設計比較 (重點)

📊 架構差異

◆ TCP (點對點)

Sensor → Server



👉 強耦合、不易擴展

◆ WebSocket (Client-Server)

Sensor → Server → Web UI



👉 適合即時畫面，不適合設備網路

◆ MQTT (推薦 IoT)

Sensor → Broker → Subscriber
 ├ Django
 ├ Android
 └ Dashboard



👉 解耦 + 可擴展 + 多設備

◆ 四、關鍵能力比較

功能	TCP	WebSocket	MQTT
即時性	☆☆☆	☆☆☆☆	☆☆☆☆
雙向通訊	☆	☆☆☆☆	☆☆☆☆
Pub/Sub	×	×	✓
QoS	×	×	✓
低頻寬	☆☆	☆☆	☆☆☆☆
大量設備	×	×	✓
開發難度	✗高	☆中	☆☆中
IoT適合度	☆☆	☆☆☆	☆☆☆☆☆

◆ 五、實務建議 (你現在的架構)

你之前有：

- Raspberry Pi + DHT22
- MQTT server
- Django
- Android App

👉 最佳架構其實是：

```
[Sensor] → MQTT → [Broker] → Django (Subscriber)
                        ↓
                      WebSocket
                        ↓
                      Web UI
```



◆ 六、結論 (重點一句話)

👉 如果是 IoT 系統：

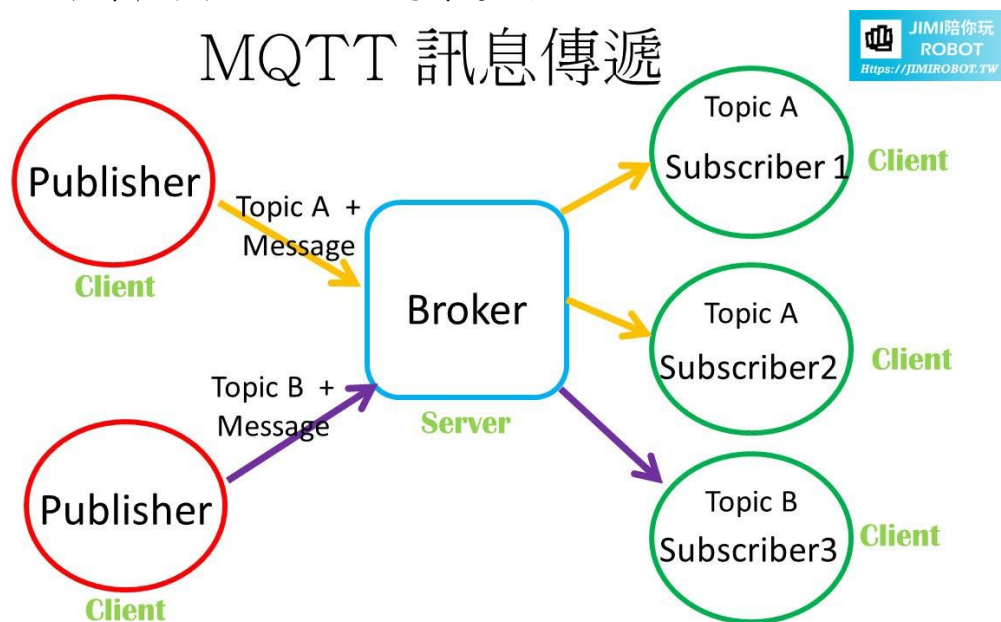
- 設備通訊 → 用 MQTT (核心)
- 前端即時顯示 → 用 WebSocket
- 底層控制 → TCP (特殊情況)



生成式 AI 範例: TCP_IP, WebSocket, MQTT 比較說明.mhtml

5-2 MQTT 架設 in Windows

MQTT 的全名是 Message Queuing Telemetry Transport，目前最新的版本為 MQTT v5.0（但 MQTT v3.1 是一般使用），當初最早是由 IBM 所提出，後來在 2014 年變成 OASIS（<https://www.oasis-open.org/>）的標準，是一種適用於 Server 與 Client 訊息傳遞的通訊協定，此協定建構在 TCP/IP Stack 上，利用『訂閱 Subscribe』與『發佈 Publish』的機制來進行訊息傳遞。因為其訊息結構簡單且輕量化，所以非常適合用於硬體效能較低的控制器，或作為物聯網的輕量資料收集應用，下面來簡單解釋 MQTT 的訊息傳遞過程：

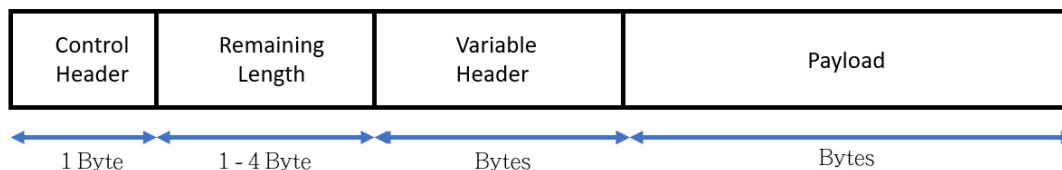


在 MQTT 的架構下，會有 3 種角色：**Broker 代理人**、**Publisher 訊息發佈者**、**Subscriber 訊息訂閱者**。Broker 負責接收來自 Publisher 的訊息並『配發』訊息給 Subscriber，Publisher 是訊息源頭，也就是發佈訊息的人，Subscriber 也就是接收訊息的人。

假設你今天有個資料擷取裝置比如溫濕度感測器，想要透過 MQTT 的通訊，傳遞溫度與濕度訊息傳到不同的裝置（多個 PC 或手機）使用，Publisher 就可以是溫濕度感測器，定時的送出溫度濕度資訊，Subscriber 就可以是不同 PC 和手機，Broker 的工作就是接收來自感測器的訊息，轉送給 PC 與手機，至於 Broker 怎麼決定訊息要轉送或配發給誰，Topic 主題就是關鍵的點，你有訂閱該 Topic，Broker 才會配發給你。

MQTT 訊息格式：

MQTT 訊息格式



在 MQTT 訊息中，『Control Header』跟『Remaining Length』這兩個部分都是必須存在，後面的『Variable Header』跟『Payload』則是依據傳送需求決定，如果以 Publish Message 來說，Variable Header 就會放入 topic 主題，Payload 區塊就是欲傳遞的訊息內容。

MQTT 還有幾個重要特色：

1. 在訊息傳遞的過程中，Publisher 不需要知道 Subscriber 的 IP，只需要知道 Broker 的位址就可以進行訊息傳遞。
2. MQTT 的 Topic 主題帶有階層式設計，用 / 來分開，比如說工廠會議室的溫度與濕度，就可以編寫成 factory/meeting/temp 與 factory/meeting/humid，英文大小寫是有差的。
3. 由於 MQTT 是設計允許在網路環境較差地方運行，所以針對網路的品質 (QoS)，使用者可以自己決定操作等級：
 - QoS Level0: 發送者丟出一次訊息給 Broker，並不保證 Subscriber 會收到。
 - QoS Level1: Publisher 丟出訊息後，Broker 必須回傳 PUBACK 訊息確定有收到，如沒回，Publisher 會再重傳一次。(缺點為 Subscriber 有可能會收到相同的訊息)
 - QoS Level2: 在 Publisher 確認 Broker 有收到訊息後，Broker 才可進行訊息傳遞，且 Subscriber 在收到訊息後，也必須回傳 PUBCOMP 訊息給 Publisher，此方法可以避免收到重複的訊息，但缺點就是較佔頻寬

MQTT 控制訊息最小只有 2 位元組的資料滿最多可以承載 256 Mb 的資料。有 14 種預定義的訊息類型用於：連接客戶端與代理、斷開連接、發布資料、確認資料接收、監督客戶端與代理的連接。

mosquitto 是什麼？mosquitto 是一款實現了訊息推送協定 MQTT v3.1 的開源訊息代理軟體，提供輕量級的，支援可發布/可訂閱的訊息推送模式，使裝置對裝置之間變得簡單，例如現在應用廣泛的低功耗感測器，手機、嵌入式電腦、微型控制器等行動裝置。一個典型的應用案例就是 Andy Stanford-Clark (Mosquitto 議創始人之一) 在家中實現的遠端監控和自動化。

Mosquitto

<https://mosquitto.org/download/>

Source

- [mosquitto-2.0.18.tar.gz](#) (GPG signature)
- [Git source code repository](#) (github.com)

Older downloads are available at <https://mosquitto.org/files/>

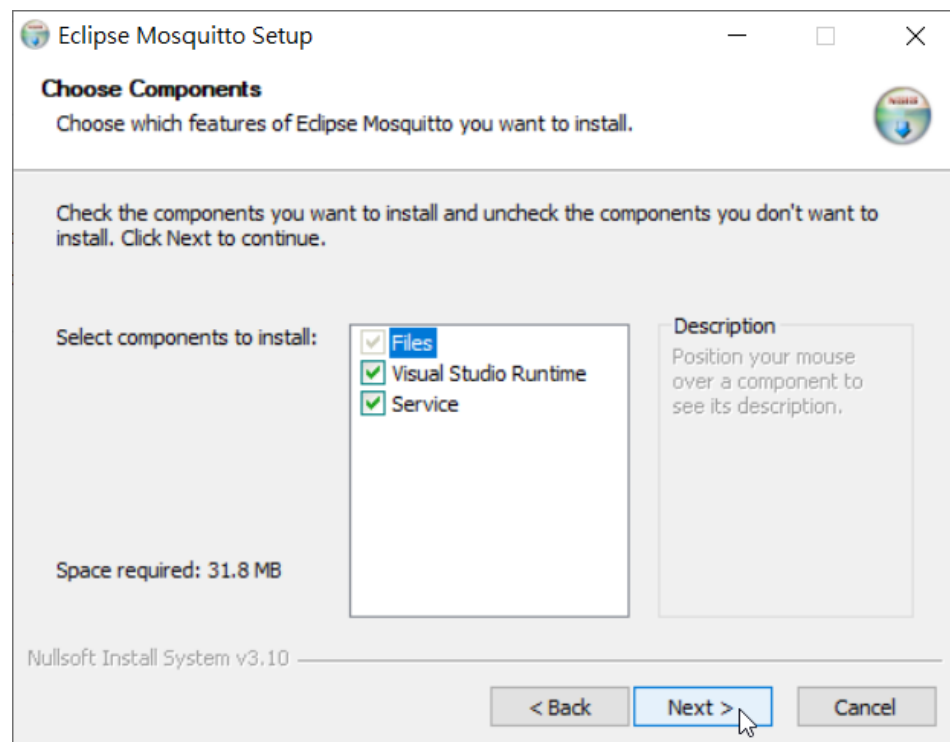
Binary Installation

The binary packages listed below are supported by the Mosquitto project. In many cases Mosquitto is also available directly from

Windows

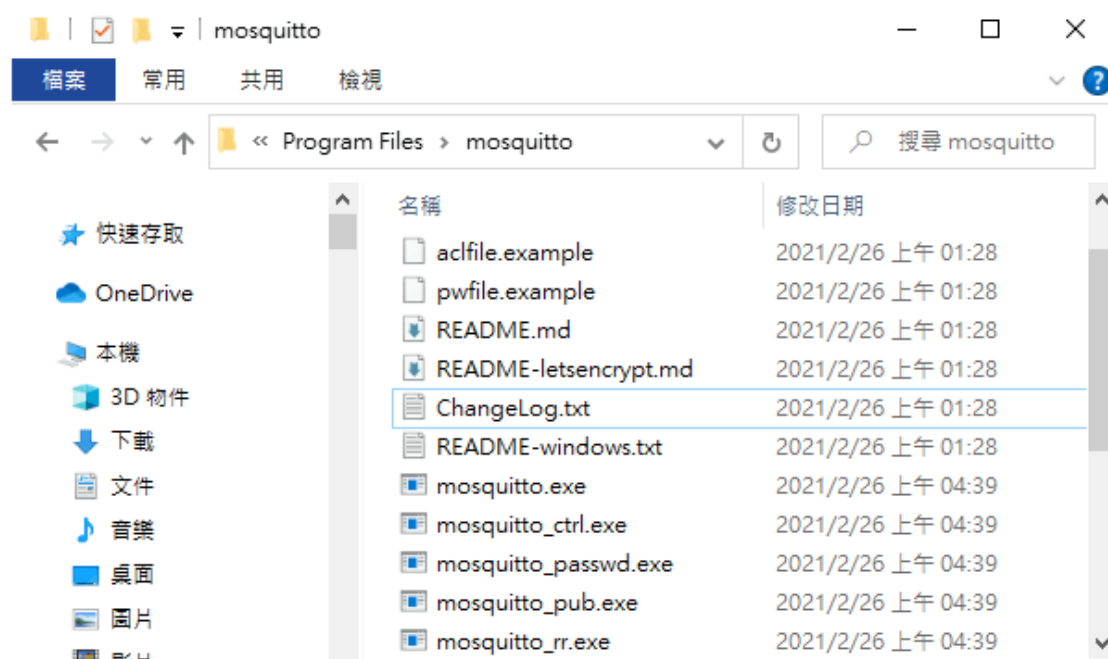
- [mosquitto-2.0.18a-install-windows-x64.exe](#) (64-bit build, Windows Vista and up, built with Visual Studio Community 2019)
- [mosquitto-2.0.18a-install-windows-x32.exe](#) (32-bit build, Windows Vista and up, built with Visual Studio Community 2019)

安裝 Mosquitto 的過程，勾選”Service”(服務)選項，讓它安裝成 Windows 服務，只要 Windows 開機，即可一併啟動 Mosquitto 伺服器 (broker)。



如果你只是要測試、開發 MQTT 應用程式，可以用手動方式啟動 MQTT 伺服器。

Mosquitto 預設安裝在 C:\Program File\mosquitto 路徑，其中的.exe 可執行檔僅能在命令行模式運作，也就是在命令提示字元或 PowerShell 裡面執行。

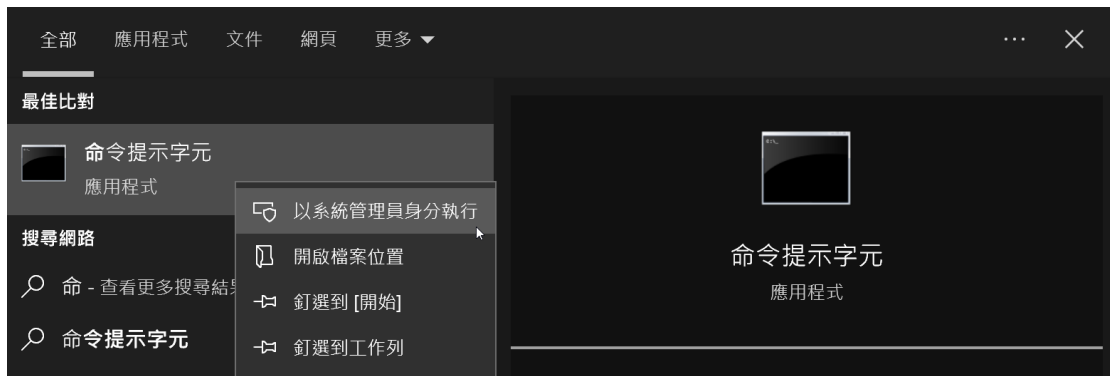


這些是 mosquitto 資料夾裡的部份檔案：

- **mosquitto**：執行 Mosquitto 伺服器
- **mosquitto_pub**：發布（publish）訊息給伺服器
- **mosquitto_sub**：訂閱（subscribe）伺服器上的主題
- **mosquitto_passwd**：產生密碼檔案
- **mosquitto.conf**：Mosquitto 伺服器的設置檔
- **pwfile.example**：使用者名稱和密碼範例檔

➤ 新增 Mosquitto 的使用者帳號和密碼

自行產生使用者帳號、密碼的命令行工具是 mosquitto_passwd，請先用系統管理員身份執行命令提示字元（因為 Windows 系統可能不允許非系統管理員寫入檔案到 C:\Program Files\mosquitto 路徑）：



儲存帳密的檔案可任意取名，將它命名成“froggy.pwd”。在其中儲存帳號“cubie”、密碼 1234 的指令如下：

```
# mosquito_passwd -c "froggy.pwd" cubie
```

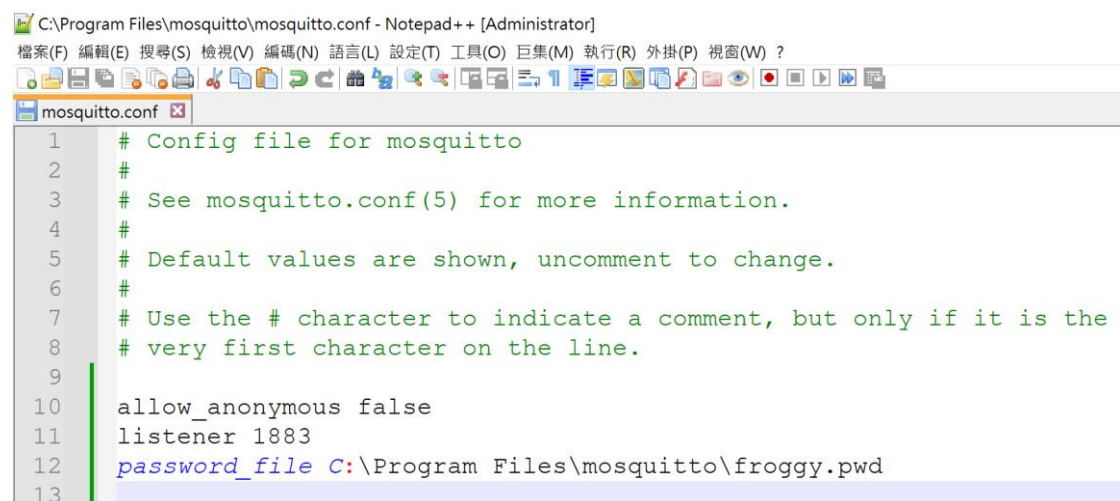
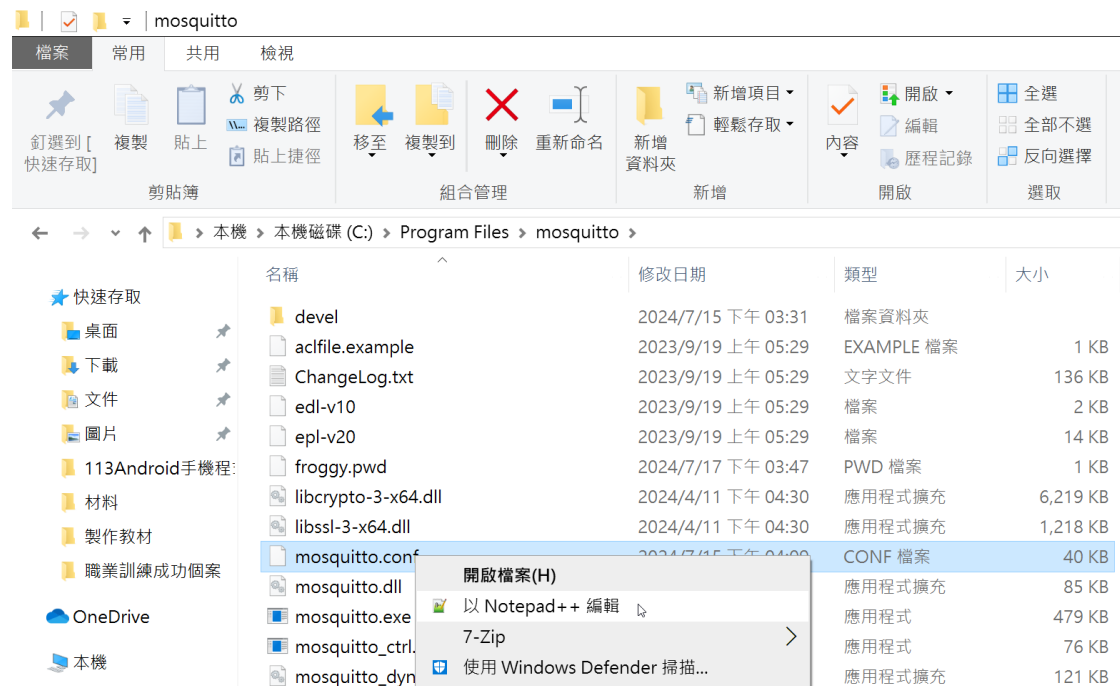
```
C:\Program Files\mosquitto>mosquitto_passwd -c "froggy.pwd" cubie
Password:
Reenter password:
```

系統管理員: Windows PowerShell

```
Windows PowerShell
Copyright (C) Microsoft Corporation. 著作權所有，並保留一切權利。
請嘗試新的跨平台 PowerShell https://aka.ms/pscore6

PS C:\WINDOWS\system32> cd 'C:\Program Files\mosquitto\'
PS C:\Program Files\mosquitto> .\mosquitto_passwd.exe -c "froggy.pwd" cubie
Password:
Reenter password:
PS C:\Program Files\mosquitto>
```

➤ 在 `mosquitto.conf` 中加入使用者帳密檔的路徑



#不允許匿名登入

allow_anonymous false

#指定帳號清單的目錄

password_file C:\Program Files\mosquitto\froggy.pwd

#指定遠端登入時可以使用的 PORT

listener 1883

如果你希望區分本地和外部連線（例如給內部用戶不同的認證方式），你可以定義兩個 listener：

```
conf
# /etc/mosquitto/mosquitto.conf

# Listener for local-only
listener 1883 127.0.0.1
allow_anonymous true

# Listener for external
listener 1884 0.0.0.0
allow_anonymous false
password_file /etc/mosquitto/passwd
```

這樣你就有兩個埠號：

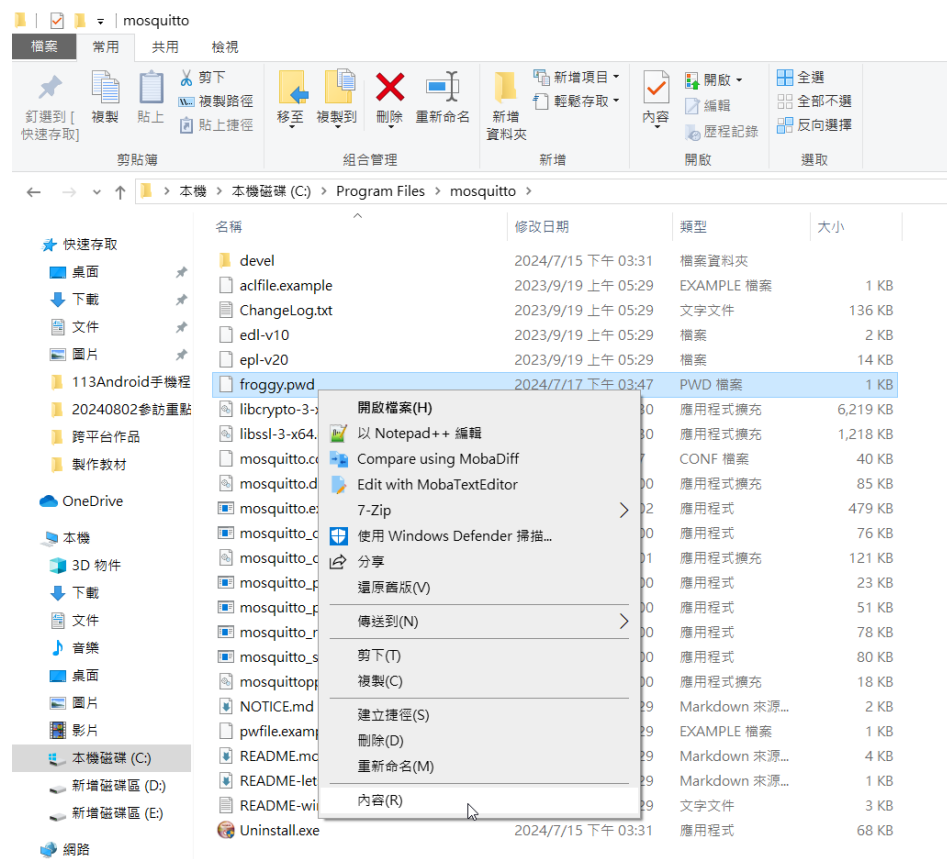
- 本地端使用 127.0.0.1:1883
- 外部使用 0.0.0.0:1884



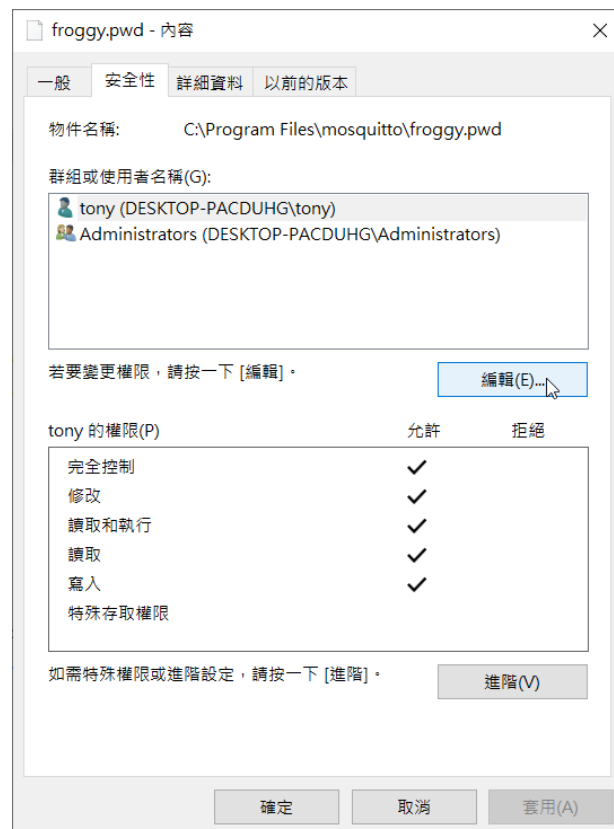
設定帳號「cubie」與密碼，儲存到 froggy.pwd 文件內

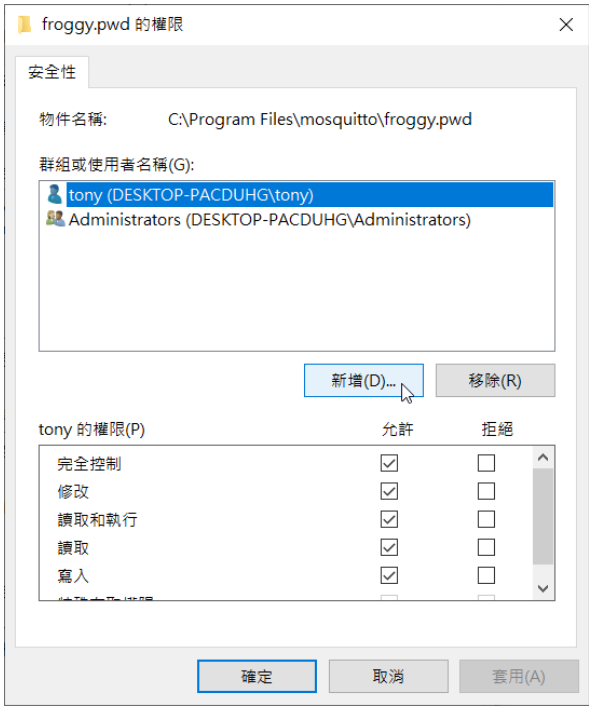
```
# mosquitto_passwd -c "froggy.pwd" cubie
```

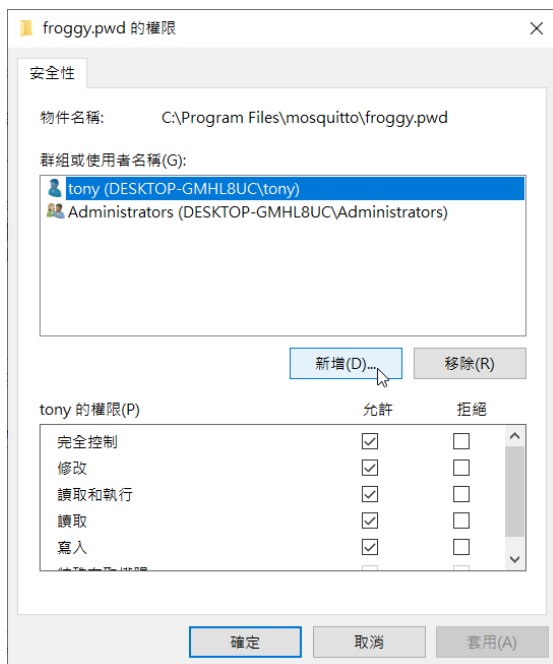
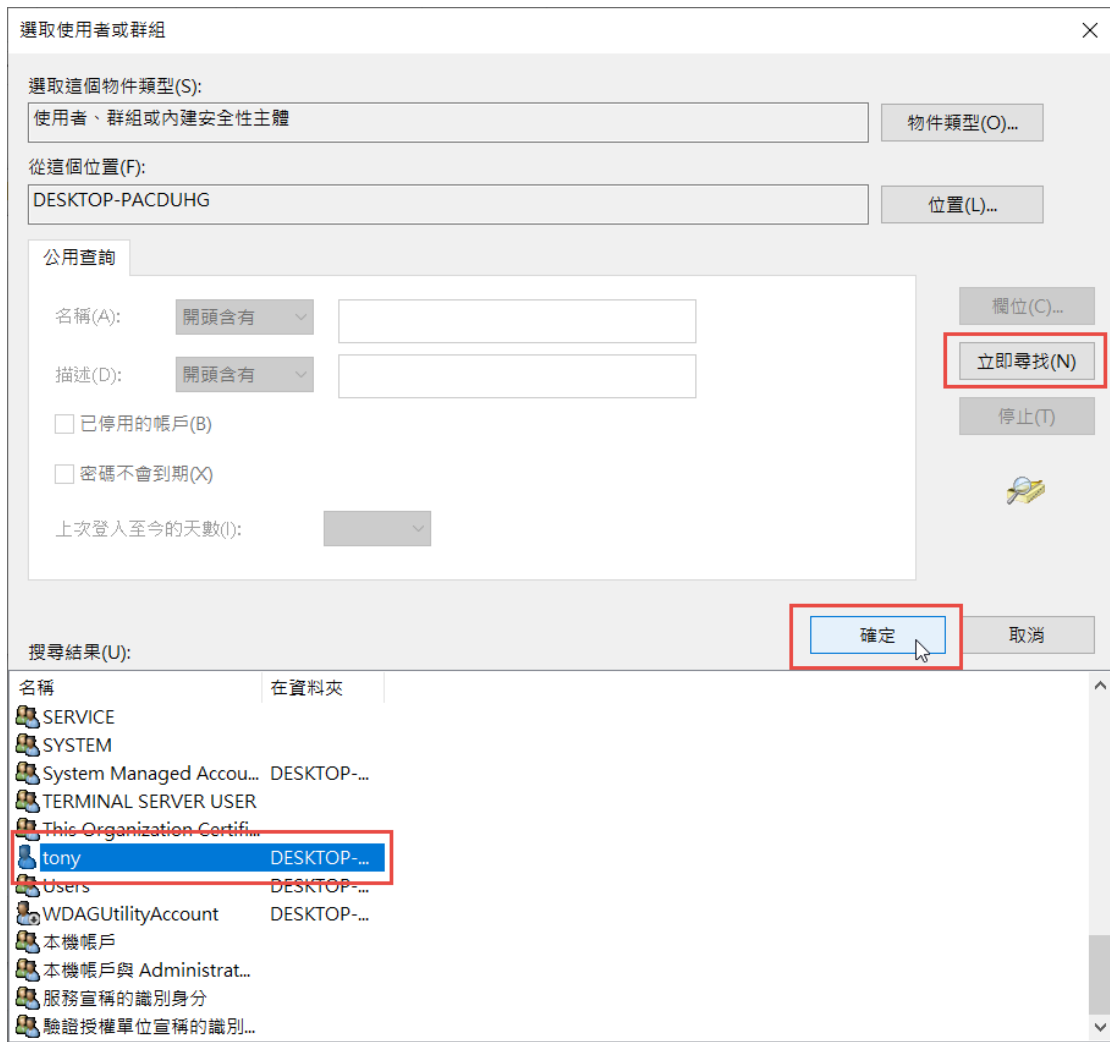
開啟權限：

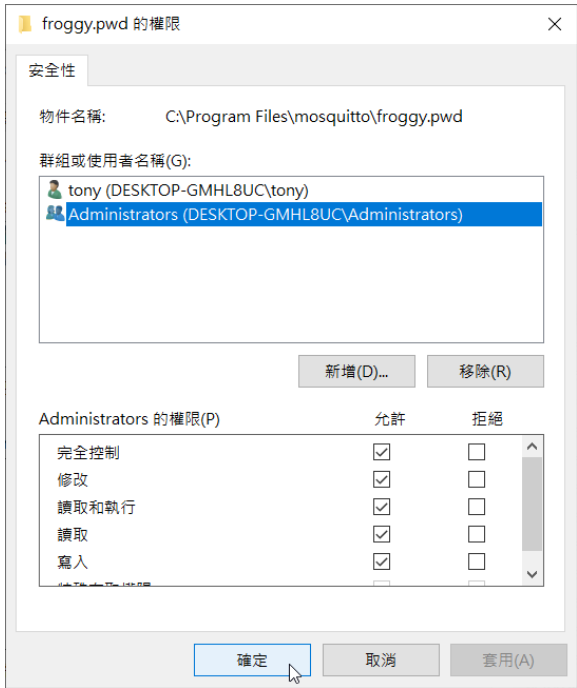
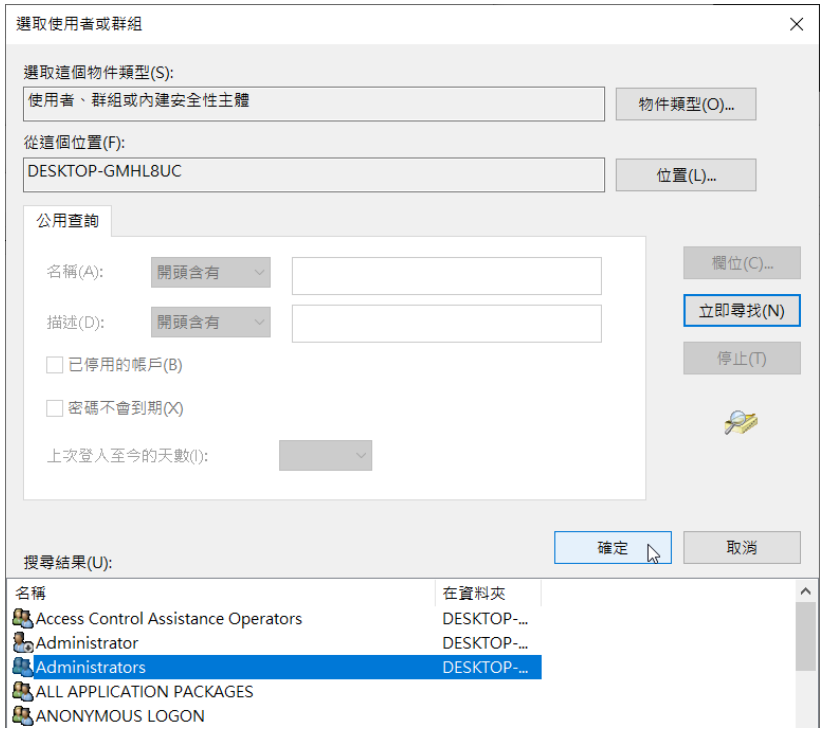


要新增 Administrators 群組，讓有這個權限的使用者都有權限。









netstat -an

(用於顯示網絡連接、路由表、網絡接口和網絡協議統計信息)

```

Microsoft Windows [版本 10.0.18362.239]
(c) 2019 Microsoft Corporation. 著作權所有，並保留一切權利。

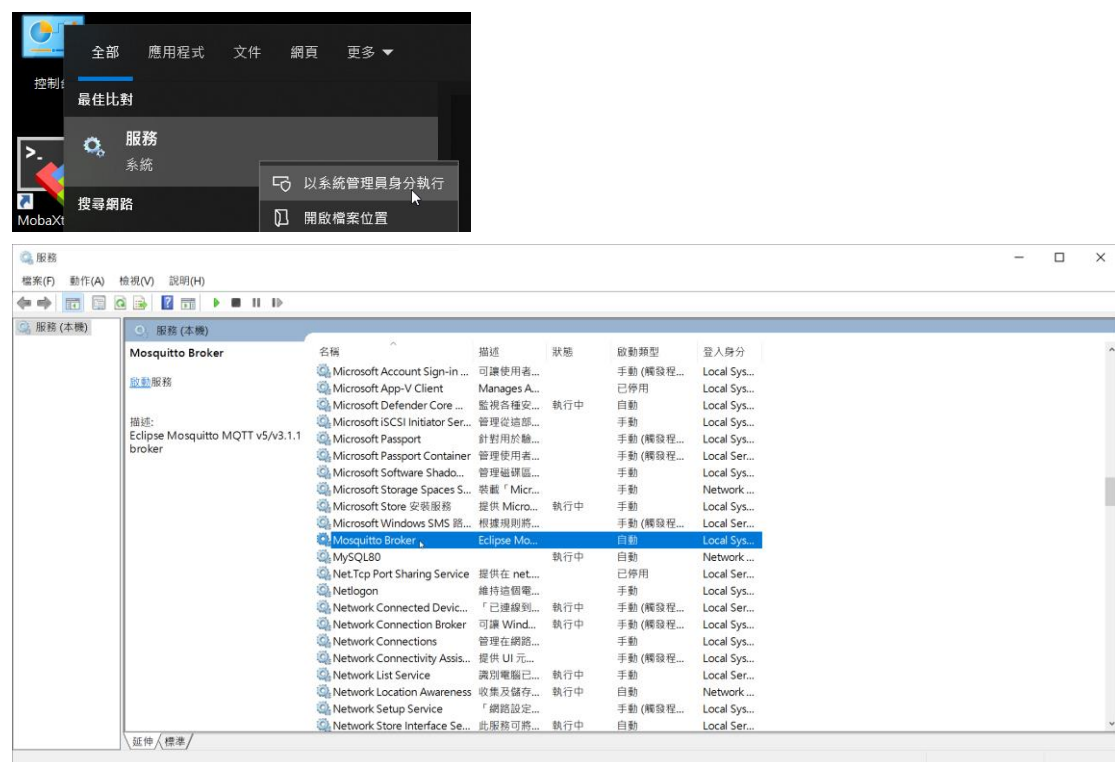
C:\Users\tony>netstat -an

使用中連線

協定    本機位址          外部位址          狀態
TCP     0.0.0.0:135       0.0.0.0:0         LISTENING
TCP     0.0.0.0:445       0.0.0.0:0         LISTENING
TCP     0.0.0.0:623       0.0.0.0:0         LISTENING
TCP     0.0.0.0:1883      0.0.0.0:0         LISTENING
TCP     0.0.0.0:3306      0.0.0.0:0         LISTENING
TCP     0.0.0.0:3389      0.0.0.0:0         LISTENING
TCP     0.0.0.0:5040      0.0.0.0:0         LISTENING
TCP     0.0.0.0:5357      0.0.0.0:0         LISTENING
TCP     0.0.0.0:7070      0.0.0.0:0         LISTENING
TCP     0.0.0.0:16992     0.0.0.0:0         LISTENING
TCP     0.0.0.0:30193     0.0.0.0:0         LISTENING
TCP     0.0.0.0:30194     0.0.0.0:0         LISTENING
TCP     0.0.0.0:30196     0.0.0.0:0         LISTENING
TCP     0.0.0.0:33060     0.0.0.0:0         LISTENING
TCP     0.0.0.0:49664     0.0.0.0:0         LISTENING
TCP     0.0.0.0:49665     0.0.0.0:0         LISTENING
TCP     0.0.0.0:49666     0.0.0.0:0         LISTENING
TCP     0.0.0.0:49667     0.0.0.0:0         LISTENING
TCP     0.0.0.0:49668     0.0.0.0:0         LISTENING
TCP     0.0.0.0:49669     0.0.0.0:0         LISTENING
TCP     0.0.0.0:49671     0.0.0.0:0         LISTENING
TCP     127.0.0.1:30196   127.0.0.1:49674   ESTABLISHED
TCP     127.0.0.1:30196   127.0.0.1:51212   ESTABLISHED

```

使用服務執行：
用系統管理身份打開「服務」





➤ 手動執行 Server

```
# mosquitto -c mosquitto.conf -v
```

其中 `-c` 是指定 `config` 檔名，`-v` 是 `verbose mode` 詳細模式。

如果啟動成功，Powershell 會將所有的 Broker 即時資訊顯示出來。

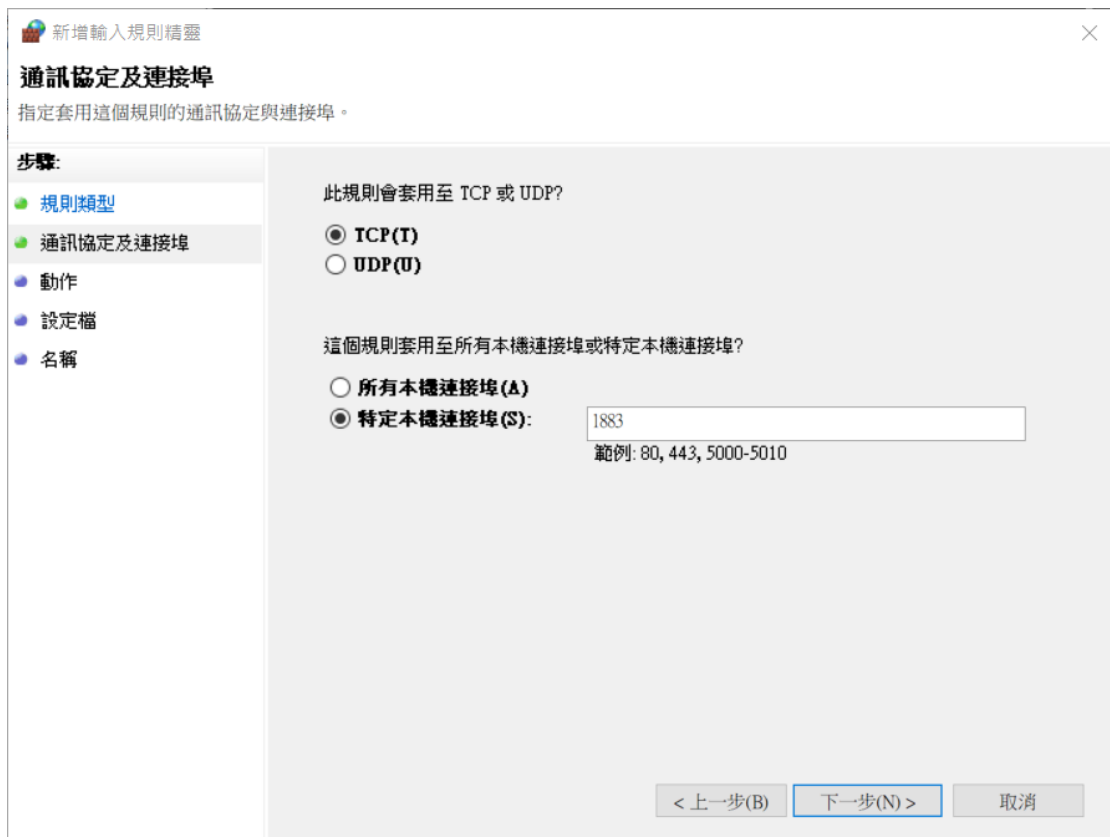
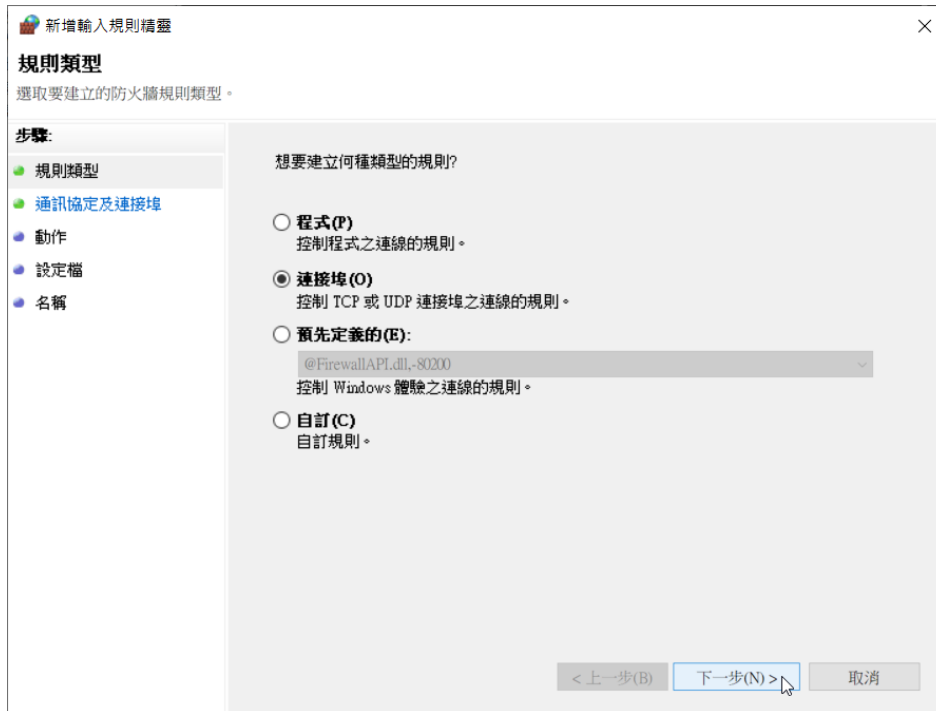
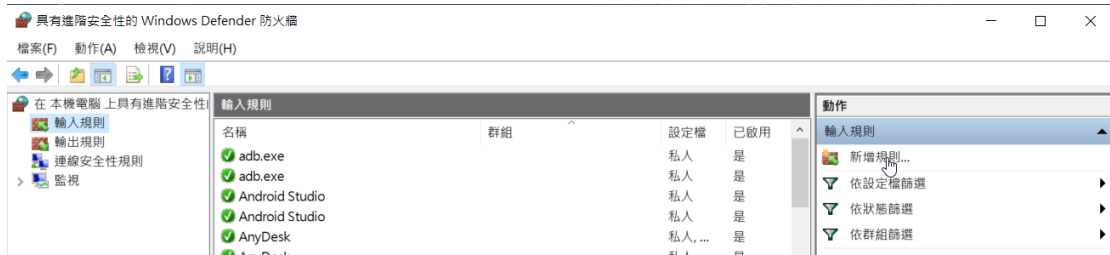
```
c:\Program Files\mosquitto>mosquitto -c mosquitto.conf -v
1721353051: mosquitto version 2.0.18 starting
1721353051: Config loaded from mosquitto.conf.
1721353051: Opening ipv6 listen socket on port 1883.
1721353051: Opening ipv4 listen socket on port 1883.
1721353051: mosquitto version 2.0.18 running
```

命令提示時，要用管理員身份開啟

```
# net stop mosquitto
# net start mosquito
# sc query mosquito (查看狀態)
```

➤ 防火牆設定

輸入規則：




新增輸入規則精靈
✕

設定檔

指定要套用此規則的設定檔。

步驟:

- 規則類型
- 通訊協定及連接埠
- 動作
- 設定檔**
- 名稱

何時會套用此規則?

- ☒ **網域 (D)**
當電腦連線至其公司網域時套用。
- ☒ **私人 (P)**
當電腦連線至私人網路位置時套用，例如住家或工作場所。
- ☒ **公用 (U)**
當電腦連線至公用網路位置時套用。

< 上一步(B)
下一步(N) >
取消


新增輸入規則精靈
✕

名稱

指定此規則的名稱與描述。

步驟:

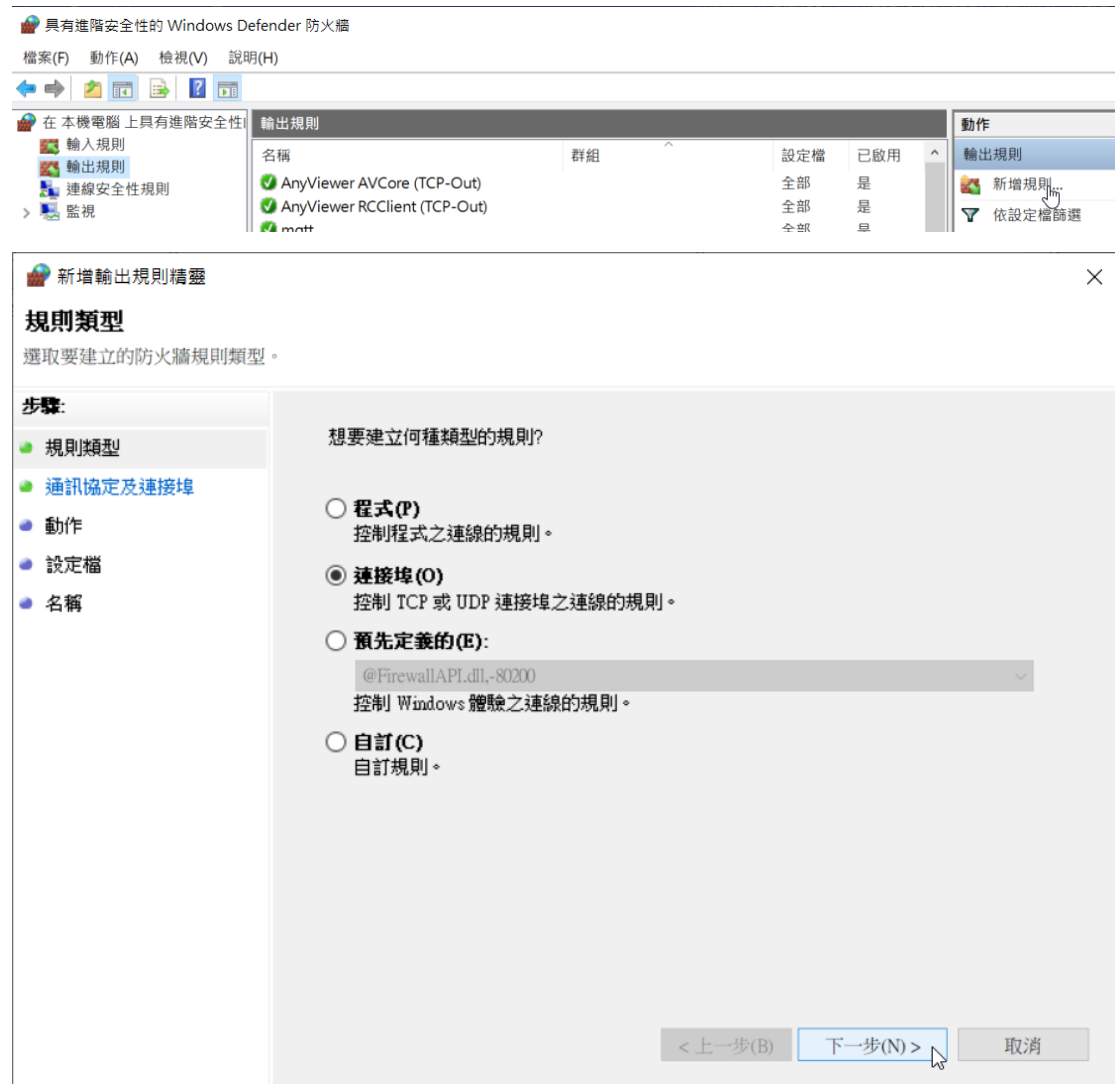
- 規則類型
- 通訊協定及連接埠
- 動作
- 設定檔
- 名稱**

名稱(N):

描述 (可省略)(D):

< 上一步(B)
完成(F)
取消

輸出規則:



新增輸出規則精靈

×

通訊協定及連接埠

指定套用這個規則的通訊協定與連接埠。

步驟:

規則類型

通訊協定及連接埠

動作

設定檔

名稱

此規則會套用到 TCP 或 UDP?

☒ TCP (T)

☐ UDP (U)

要將這個規則套用到所有遠端連接埠或特定遠端連接埠?

☐ 所有遠端連接埠 (A)

☒ 特定遠端連接埠 (S):

1883

範例: 80, 443, 5000-5010

< 上一步(B)

下一步(N) >

取消

新增輸出規則精靈

×

動作

指定要在連線符合規則中指定的條件時採取的動作。

步驟:

規則類型

通訊協定及連接埠

動作

設定檔

名稱

當連線符合指定的條件時，應採取哪些動作?

☒ 允許連線 (A)

這包含使用 IPsec 保護的連線，以及未使用 IPsec 保護的連線。

☐ 僅允許安全連線 (C)

這只包含已使用 IPsec 驗證的連線。會使用 [連線安全性規則] 節點中的 IPsec 內容和規則設定，來確保連線的安全。

自訂(O)...

☐ 封鎖連線 (K)

< 上一步(B)

下一步(N) >

取消

新增輸出規則精靈

設定檔

指定要套用此規則的設定檔。

步驟:

規則類型

通訊協定及連接埠

動作

設定檔

名稱

何時會套用此規則?

☒ 網域(D)

當電腦連線至其公司網域時套用。

☒ 私人(P)

當電腦連線至私人網路位置時套用，例如住家或工作場所。

☒ 公用(U)

當電腦連線至公用網路位置時套用。

< 上一步(B)

下一步(N) >

取消

新增輸出規則精靈

名稱

指定此規則的名稱與描述。

步驟:

規則類型

通訊協定及連接埠

動作

設定檔

名稱

名稱(N):

mqtt

描述 (可省略)(D):

< 上一步(B)

完成(F)

取消

Reference:

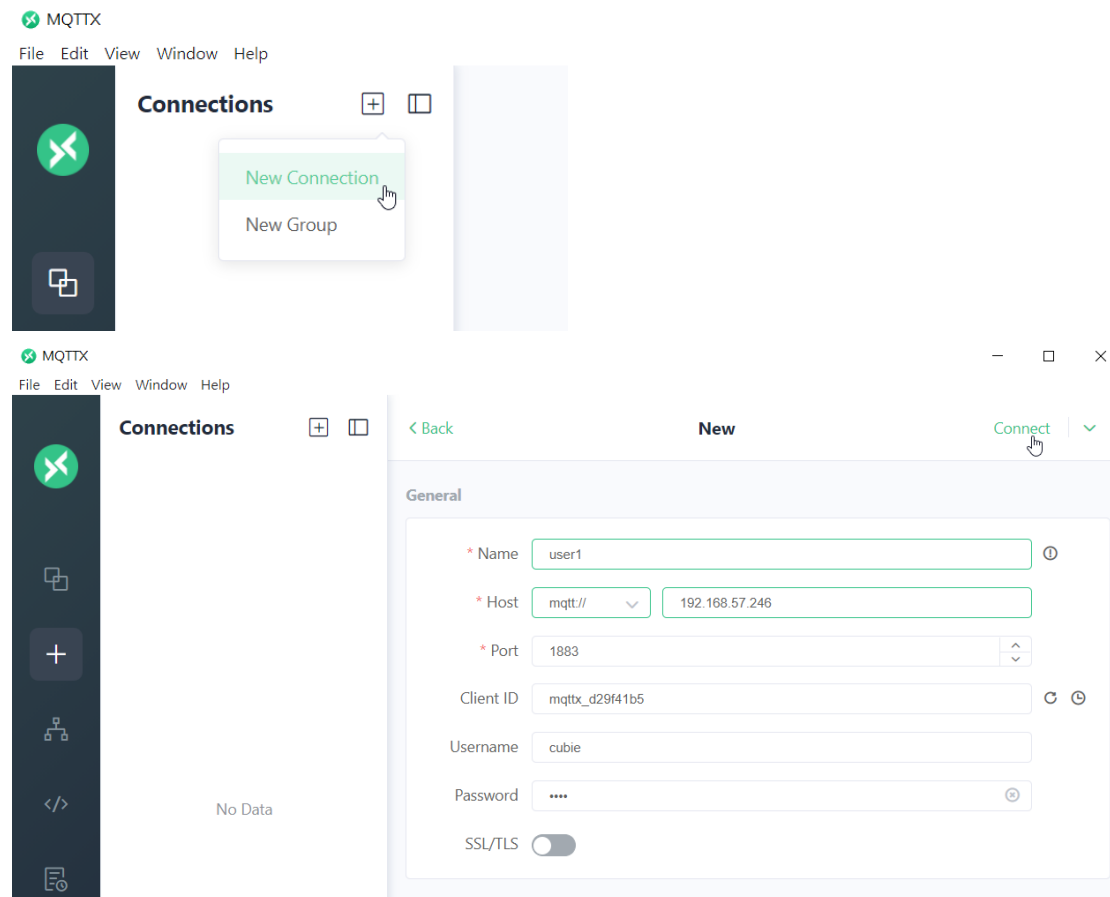
<https://jimirobot.tw/esp32-mosquitto-windows-mqtt-tutorial/>

<https://jimirobot.tw/esp32-mosquitto-conf-mqtt-tutorial/>

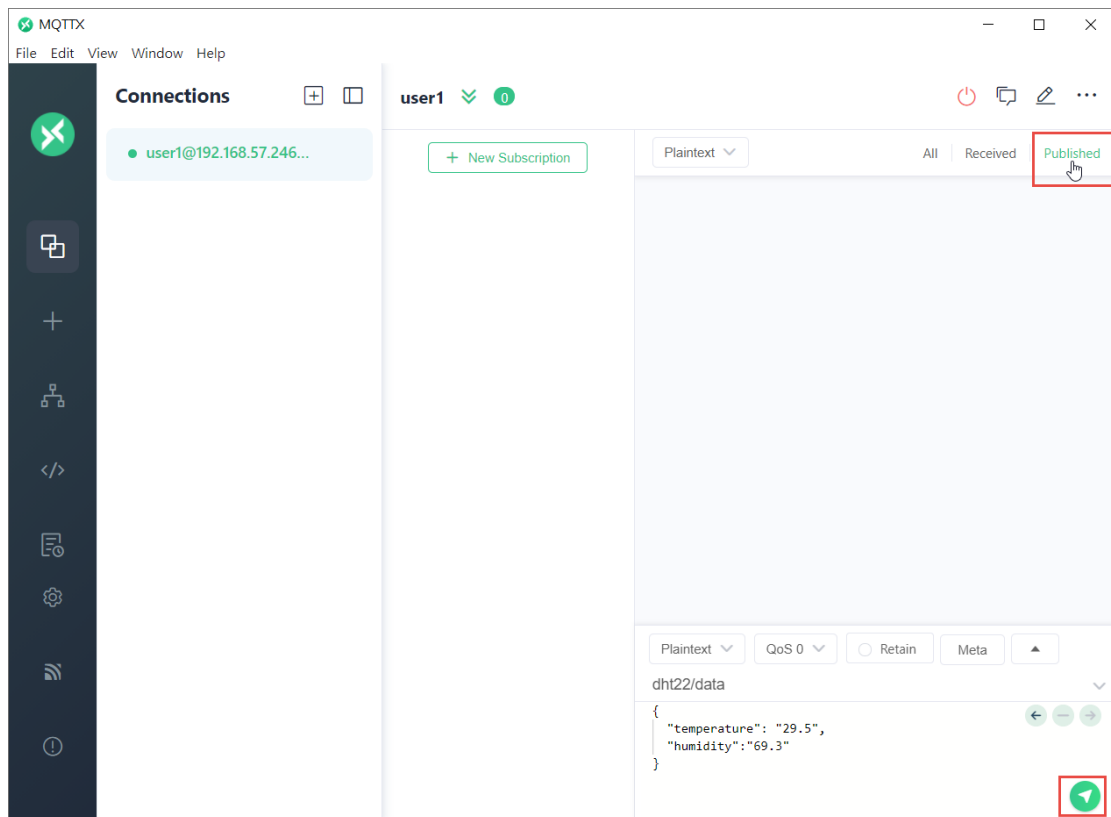
➤ 測試程式 MQTTX

網址: <https://mqttx.app/downloads>

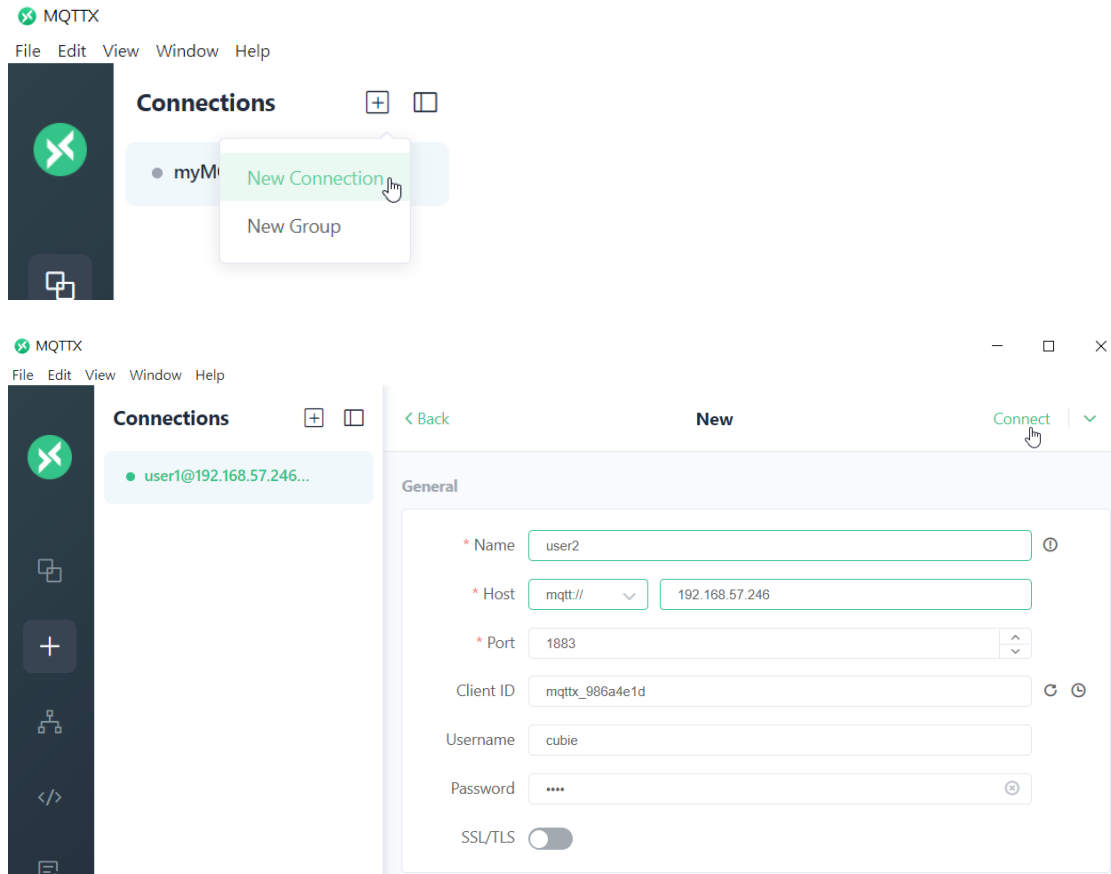
連線側 MQTT Server :

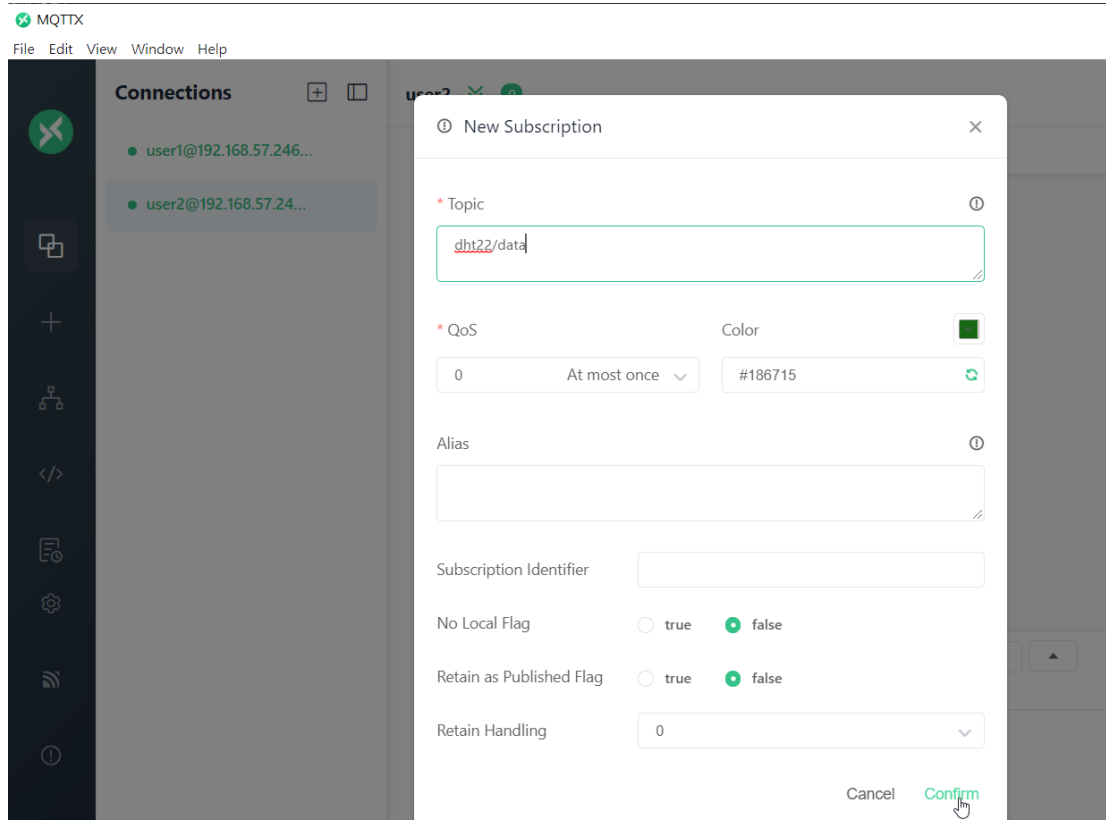
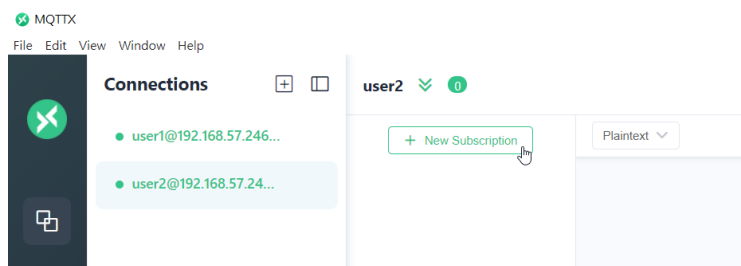


Publisher 訊息發佈者 :

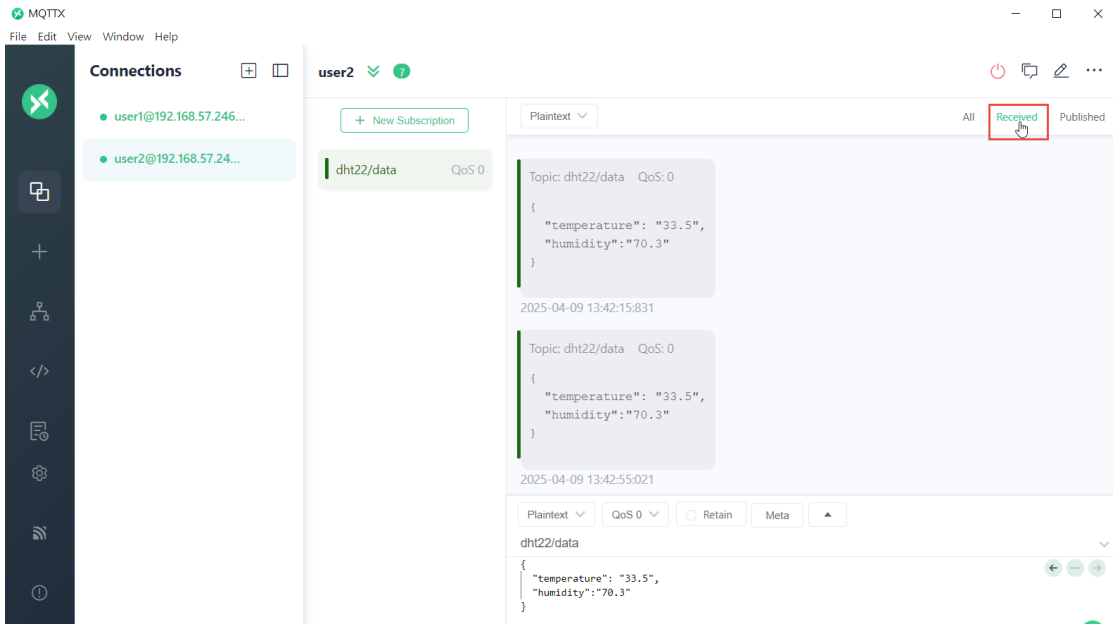


Subscriber 訊息訂閱者





即可查看 Publisher 訊息發佈者的訊息



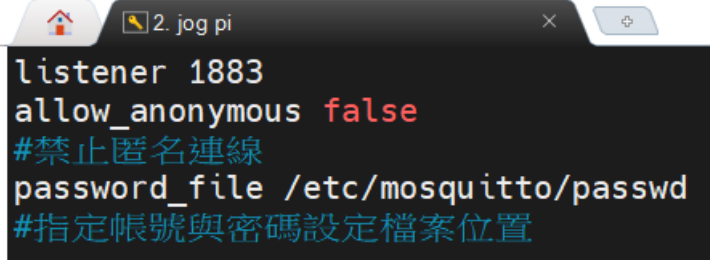
5-3 MQTT 架設 in Linux

```
# sudo apt install mosquitto mosquitto-clients
# sudo systemctl status mosquitto
# sudo systemctl start mosquitto
# sudo systemctl stop mosquitto
# sudo systemctl restart mosquitto
# sudo systemctl disable mosquitto
```

```
# sudo mosquitto_passwd -c /etc/mosquitto/passwd cubie
```

```
pi@raspberrypi:~$ sudo mosquitto_passwd -c /etc/mosquitto/passwd cubie
Password:
Reenter password:
```

```
# sudo vim /etc/mosquitto/conf.d/default.conf
```



```
listener 1883
allow_anonymous false
#禁止匿名連線
password_file /etc/mosquitto/passwd
#指定帳號與密碼設定檔案位置
```

```
# netstat -a | grep 1883
```

```
pi@raspberrypi:~/code$ netstat -a | grep 1883
tcp        0      0 localhost:1883      0.0.0.0:*           LISTEN
tcp6       0      0 localhost:1883      [::]:*              LISTEN
```

開啟一個終端，使用 `mosquitto_sub` 訂閱一個主題：

```
# mosquitto_sub -h 192.168.57.246 -p 1883 -u cubie -P 1qaz@wsx
-t test/topic
```

此命令將會訂閱主題 `test/topic`，並且會顯示發佈到該主題的消息。

開啟另一個終端，使用 `mosquitto_pub` 發佈一條消息到同一主題：

```
# mosquitto_pub -h 192.168.57.246 -p 1883 -u cubie -P 1qaz@wsx
-t test/topic -m "Hello, Mosquitto!"
```

reference:

<https://blog.davidou.org/archives/2735>

如果你希望區分本地和外部連線（例如給內部用戶不同的認證方式），你可以定義兩個 listener：

```
conf
# /etc/mosquitto/mosquitto.conf

# Listener for local-only
listener 1883 127.0.0.1
allow_anonymous true

# Listener for external
listener 1884 0.0.0.0
allow_anonymous false
password_file /etc/mosquitto/passwd
```

這樣你就有兩個埠號：

- 本地端使用 127.0.0.1:1883
- 外部使用 0.0.0.0:1884



第六章 ESP8266

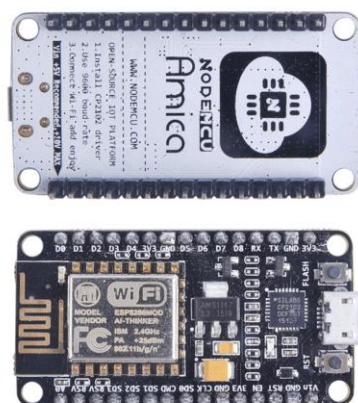
NodeMcu 簡介:

在你的專案裡面，如果 WIFI 是非常重要的一環，其實捨棄 Arduino 系統的開發板，而選擇 NodeMcu，絕對是最佳的選擇。

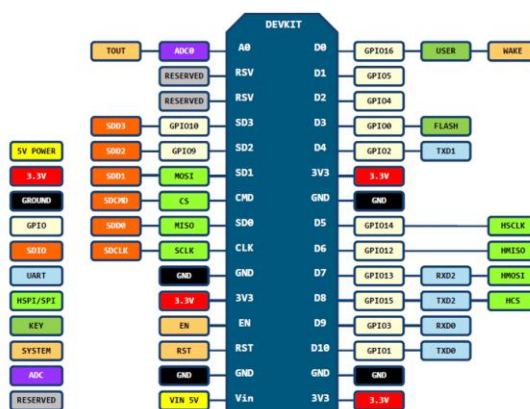
開源，交互式，可編程，低成本，簡單，智能，Wi-Fi 模組。

NodeMCU 本身就是一顆包含 ESP8266 Wi-Fi 的微控制器，本身就可以做到控制、資料處理、運算，不要再用 ESP8266 + Arduino Uno 做應用了 NodeMCU 官方內建的是 eLua 的韌體，為直譯語言，可以用 ESplorer 開發，也可以用 Arduino IDE 進行開發。常用的感測器及模組(如: 溫濕度感測、OLED 螢幕、三軸感測器等...)大多能直接支援，並包含完整的網路協議，能迅速開發進行物聯網相關應用。

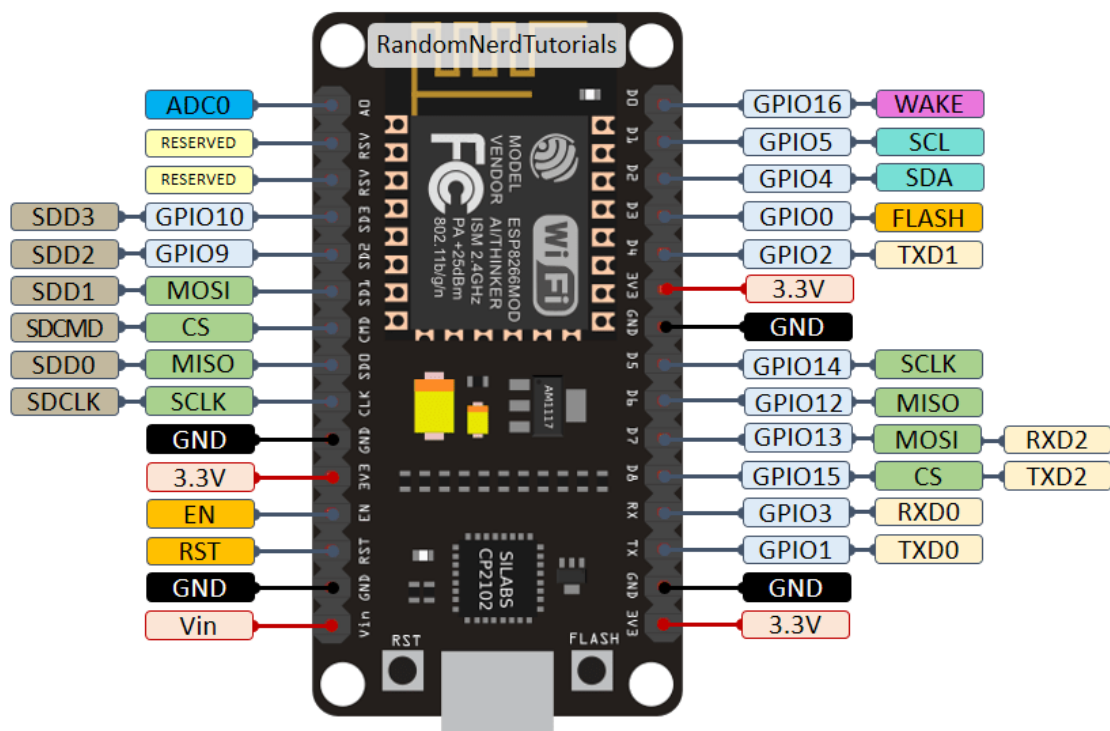
Ai-Thinker **NodeMCU V2**



PIN DEFINITION



D0(GPIO16) can only be used as gpio read/write, no interrupt supported, no pwm/12c/i2c supported.



6-1 環境安裝

下載 Arduino IDE:

首先至下載 arduino IDE: <https://www.arduino.cc/en/Main/Software>

Download the Arduino IDE



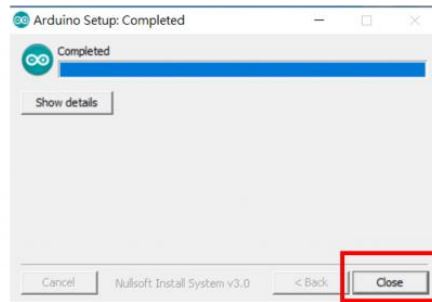
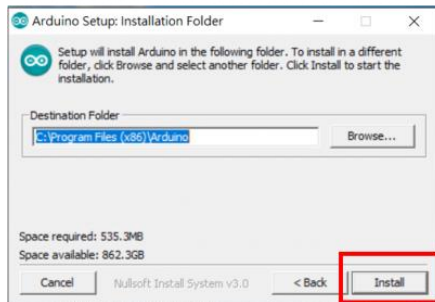
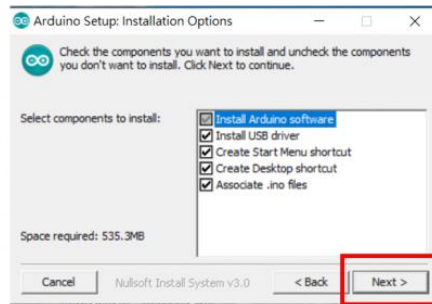
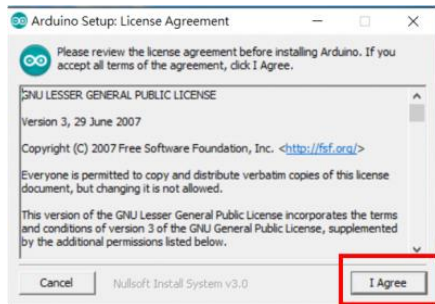
The screenshot shows the Arduino IDE download page. On the left, there is a large teal circle with a white infinity symbol and a plus sign. To its right, the text reads: **ARDUINO 1.8.12**. Below this, it says: "The open-source Arduino Software (IDE) makes it easy to write code and upload it to the board. It runs on Windows, Mac OS X, and Linux. The environment is written in Java and based on Processing and other open-source software. This software can be used with any Arduino board. Refer to the [Getting Started](#) page for Installation instructions." On the right side, there is a teal box with white text. It lists: **Windows** Installer, for Windows XP and up; **Windows** ZIP file for non admin install; **Windows app** Requires Win 8.1 or 10 with a "Get" button; **Mac OS X** 10.8 Mountain Lion or newer; **Linux** 32 bits; **Linux** 64 bits; **Linux** ARM 32 bits; **Linux** ARM 64 bits; Release Notes; Source Code; Checksums (sha512).

Contribute to the Arduino Software

Consider supporting the Arduino Software by contributing to its development. (US tax payers, please note this contribution is not tax deductible). [Learn more on how your contribution will be used.](#)



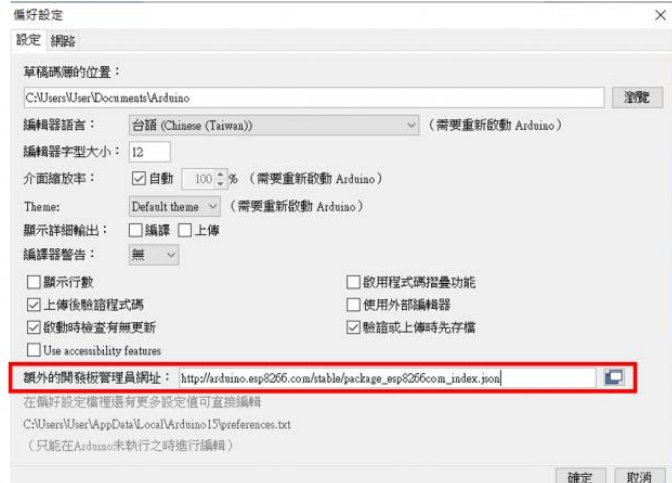
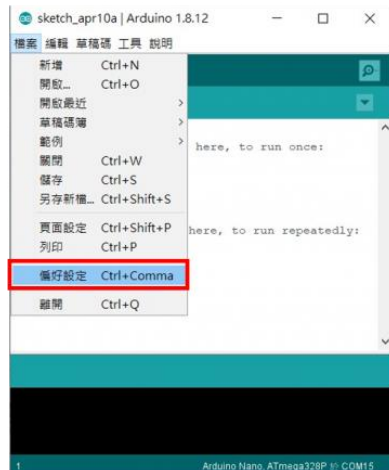
The screenshot shows the Arduino Software contribution page. On the left, there is a cartoon illustration of three characters: a square head, a rectangular head, and a circular head, all with stick-figure bodies. To their right, the text reads: "SINCE MARCH 2015, THE ARDUINO IDE HAS BEEN DOWNLOADED **41,194,697** TIMES. (IMPRESSIVE!) NO LONGER JUST FOR ARDUINO AND GENUINO BOARDS, HUNDREDS OF COMPANIES AROUND THE WORLD ARE USING THE IDE TO PROGRAM THEIR DEVICES, INCLUDING COMPATIBLES, CLONES, AND EVEN COUNTERFEITS. HELP ACCELERATE ITS DEVELOPMENT WITH A SMALL CONTRIBUTION! REMEMBER: OPEN SOURCE IS LOVE!" Below this text, there are six white circles with teal borders, each containing a teal dollar amount: "\$3", "\$5", "\$10", "\$25", "\$50", and "OTHER". At the bottom right, there are two buttons: a white button with a red border labeled "JUST DOWNLOAD" and a teal button labeled "CONTRIBUTE & DOWNLOAD".

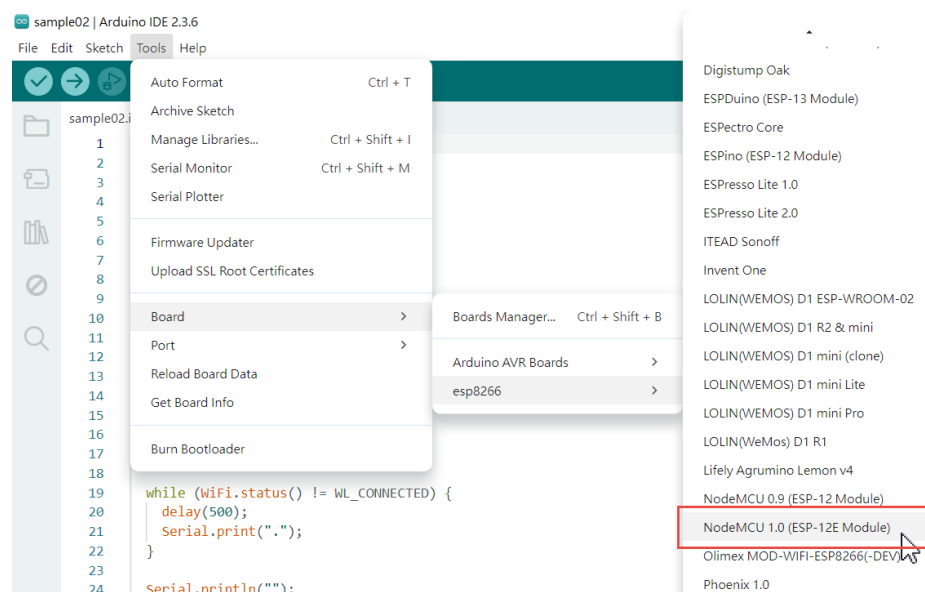
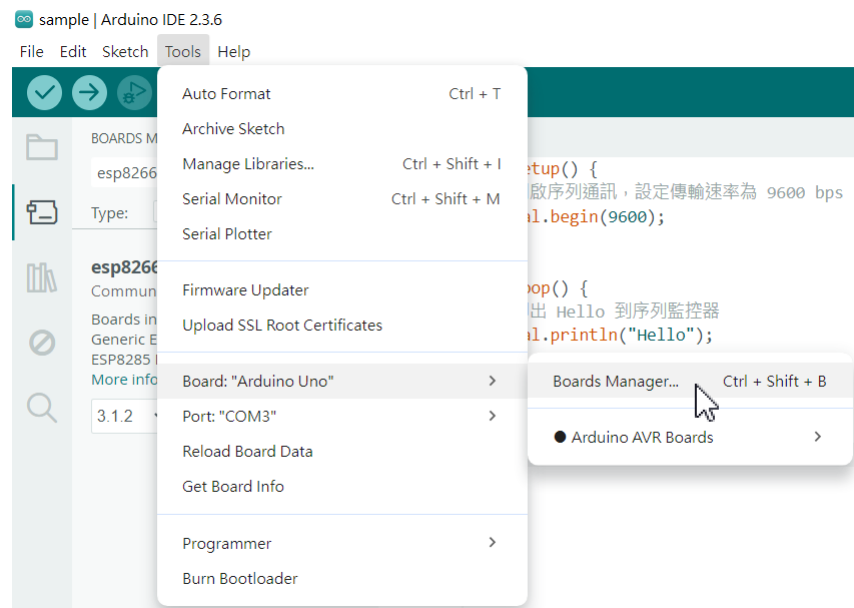


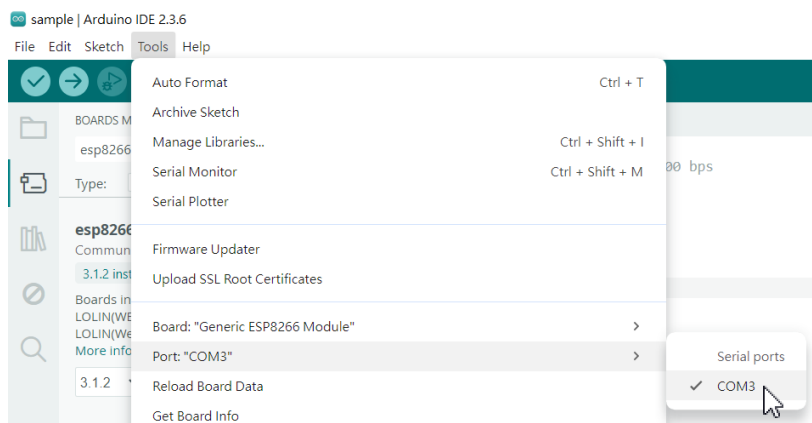
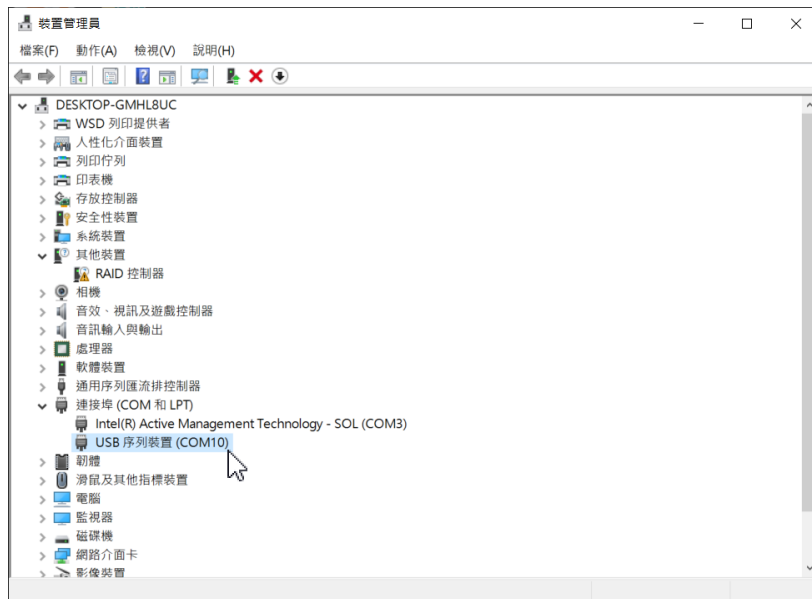
➤ 在 Arduino IDE 設定 NodeMCU:

檔案/偏好設定 => 額外的開發板管理員網址，加入

http://arduino.esp8266.com/stable/package_esp8266com_index.json







Reference:

<https://shop.mirotek.com.tw/iot/iot-start-1/>

6-2 Sample Code

提問:我的版子為 esp8266，請給我每十秒顯示 hello world 的程式

生成式 AI 範例: 1.ESP8266 顯示 hello 與連到 wifi.mhtml

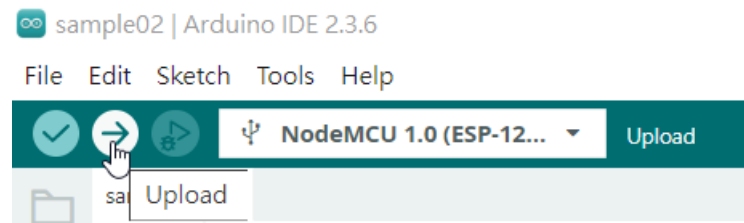
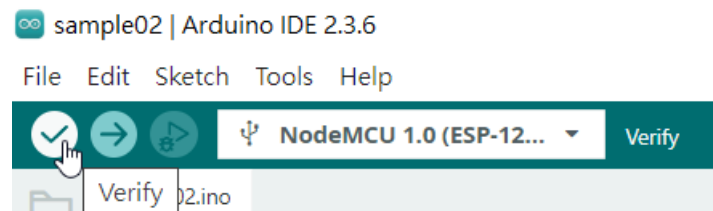


sample01 | Arduino IDE 2.3.6

File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12...

```
sample01.ino
1 void setup() {
2   Serial.begin(115200);
3   delay(1000);
4   Serial.println("Start...");
5 }
6
7 void loop() {
8   Serial.println("Hello World from ESP8266!");
9   delay(10000); // 10 秒 = 10000 毫秒
10 }
```



範例:sample01

sample02 | Arduino IDE 2.3.6

File Edit Sketch Tools Help

```
NodeMCU 1.0 (ESP-12... ▾

sample02.ino
1  #include <ESP8266WiFi.h>
2
3  // WiFi 資訊
4  // const char* ssid = "office_303_ap1";
5  // const char* password = "yvtsyvtsyvts";
6  const char* ssid = "thmr305";
7  const char* password = "thmr305";
8
9
10 // 計數變數
11 int count = 0;
12
13 void setup() {
14   Serial.begin(115200);
15   delay(10);
16
17   Serial.println();
18   Serial.println("開始連接 WiFi...");
19
20   WiFi.begin(ssid, password);
21
22   while (WiFi.status() != WL_CONNECTED) {
23     delay(500);
24     Serial.print(".");
25   }
26
27   Serial.println("");
28   Serial.println("WiFi 連線成功!");
29   Serial.print("IP 位址: ");
30   Serial.println(WiFi.localIP());
31 }
32
33 void loop() {
34   count++; // 次數 +1
35
36   Serial.print("Hello World 第 ");
37   Serial.print(count);
38   Serial.println(" 次!");
39
40   delay(10000); // 每10秒一次
41 }
```

範例:sample02

6-3 Getting Started With the ESP8266 and DHT22 Sensor

提問:我想要 esp8266+dht22，腳位使用 gpio04，請給我使用步驟

生成式 AI 範例: 2.ESP8266 DHT22 接線步驟與讀取資料.mhtml

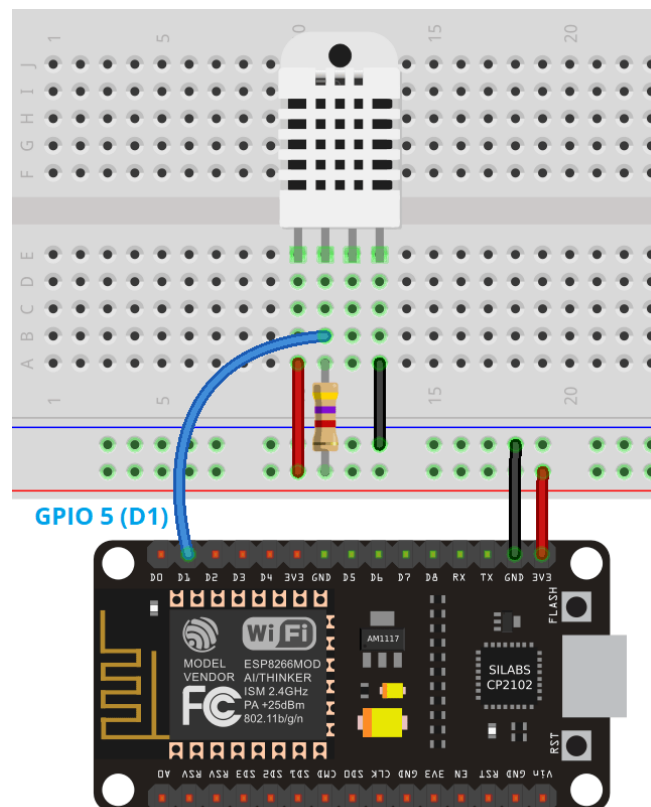
腳位參考如下：

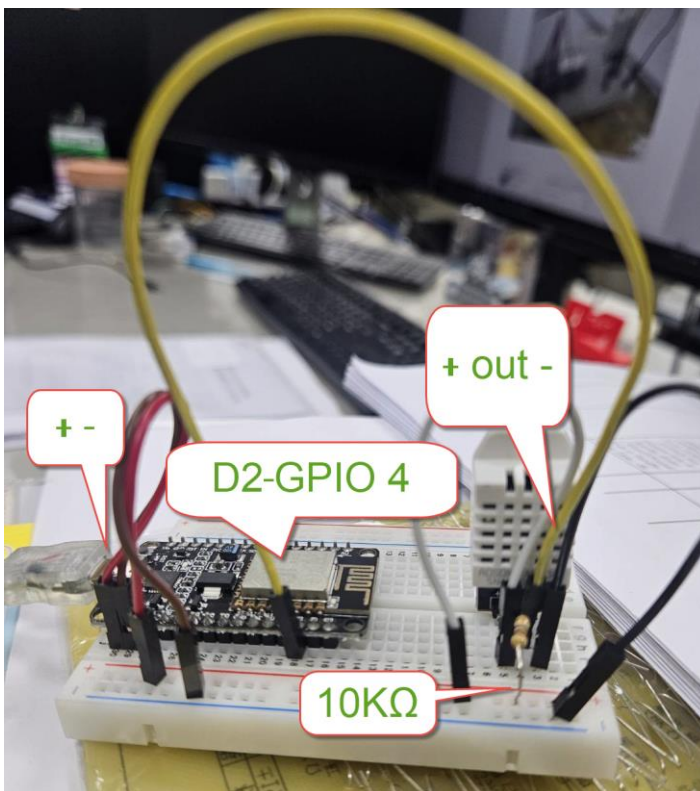
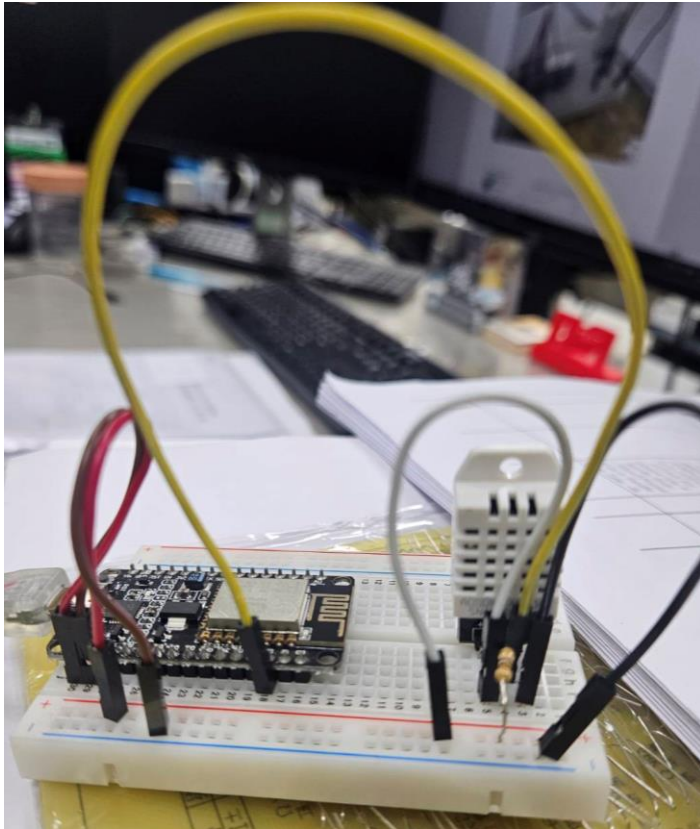
Reference:

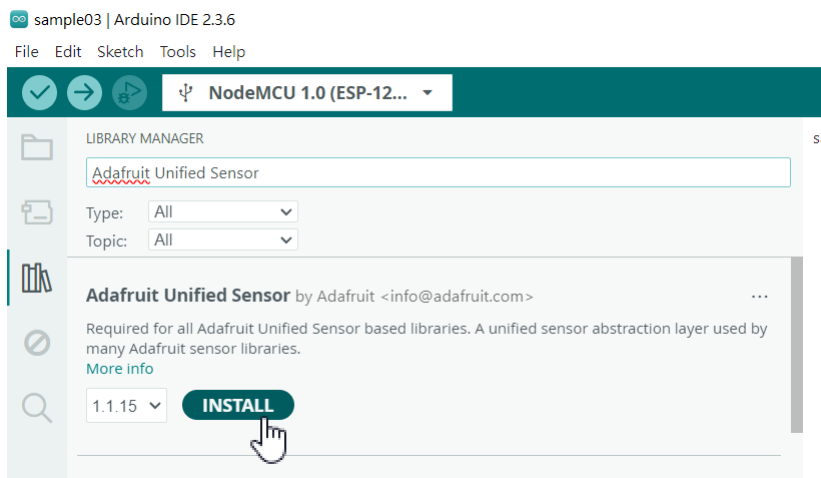
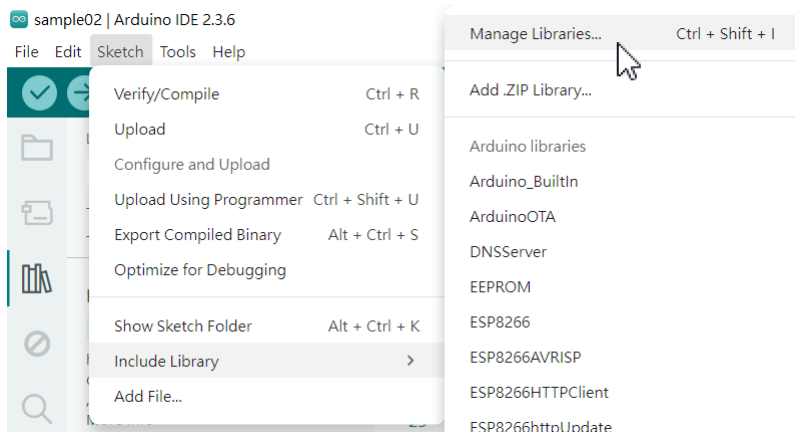
<https://randomnerdtutorials.com/esp8266-dht11dht22-temperature-and-humidity-web-server-with-arduino-ide/>

Reference:

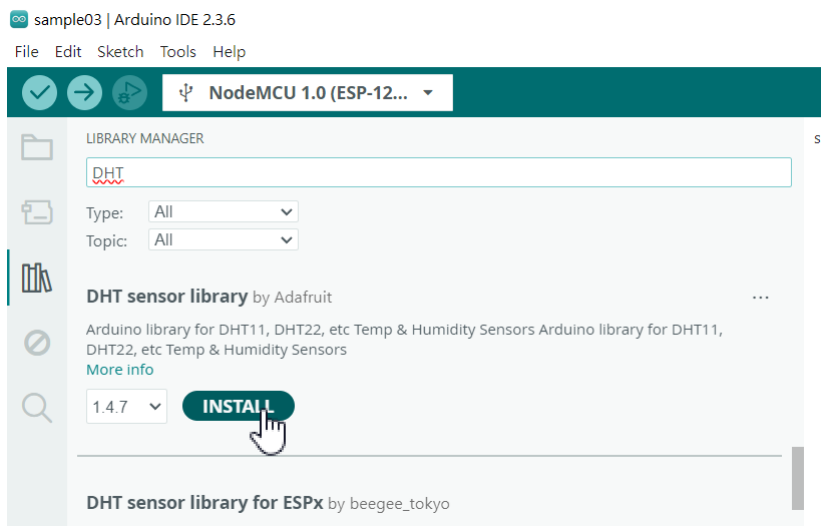
<https://www.losant.com/blog/getting-started-with-the-esp8266-and-dht22-sensor>







adafruit unified sensor



sample03 | Arduino IDE 2.3.6

File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12...)

sample03.ino

```

1  #include <DHT.h>
2
3  #define DHTPIN 4
4  #define DHTTYPE DHT22
5
6  DHT dht(DHTPIN, DHTTYPE);
7
8  void setup() {
9      Serial.begin(115200);
10     delay(1000);
11     dht.begin();
12 }
13
14 void loop() {
15     delay(2000);
16
17     float h = dht.readHumidity();
18     float t = dht.readTemperature();
19
20     if (isnan(h) || isnan(t)) {
21         Serial.println("讀取失敗！");
22         return;
23     }
24
25     Serial.print("濕度: ");
26     Serial.print(h);
27     Serial.print("% 溫度: ");
28     Serial.print(t);
29     Serial.println("C");
30 }

```

sample03 | Arduino IDE 2.3.6

File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12...)

sample03.ino

```

1  #include <DHT.h>
2
3  #define DHTPIN 4
4  #define DHTTYPE DHT22
5
6  DHT dht(DHTPIN, DHTTYPE);
7
8  void setup() {
9      Serial.begin(115200);
10     delay(1000);
11     dht.begin();
12 }
13
14 void loop() {
15     delay(2000);
16

```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM10')

Both NL & CR 115200 baud

```

濕度: 72.50% 溫度: 27.60C
濕度: 72.50% 溫度: 27.60C
濕度: 72.50% 溫度: 27.60C
濕度: 72.40% 溫度: 27.60C
濕度: 72.40% 溫度: 27.60C

```

Ln 8, Col 15 NodeMCU 1.0 (ESP-12E Module) on COM10 9

範例:sample03

6-4 Sending DHT22 Sensor Data from ESP8266 Using

MQTT

提問:

esp8266+dht22，腳位使用 gpio04，已可使顯示溫溼度在 serial monitor。

wifi:office_303_ap1

wifi password:yvtsyvtsyvts

我想要連接到

mqtt broker ip:20.194.175.52

mqtt broker port:1883

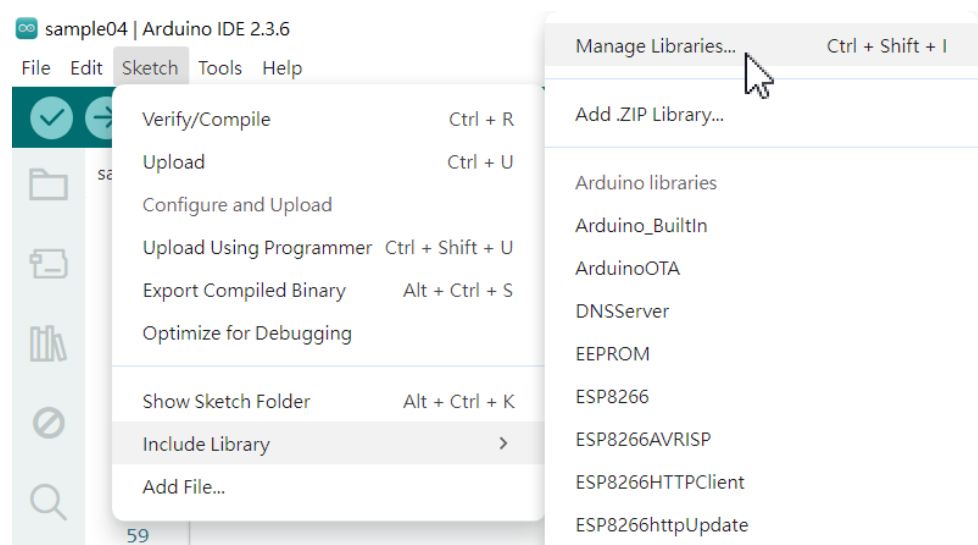
mqtt broker username:cubie

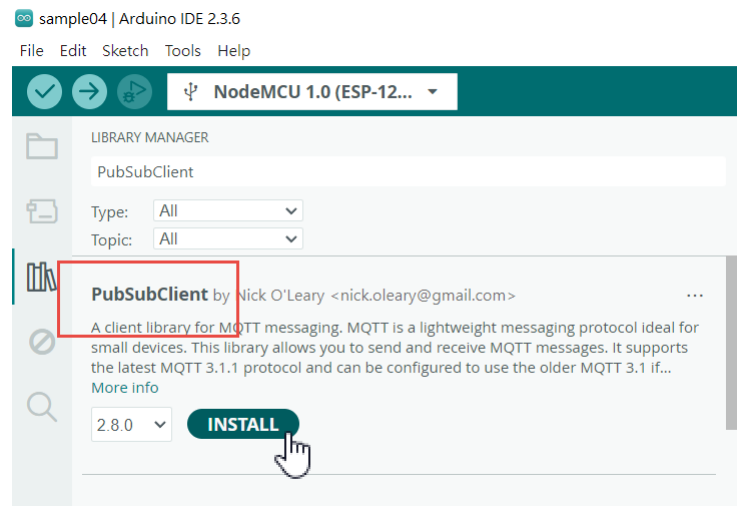
mqtt broker password:1qaz2wsx

主題為: dht22/number40

請給我步驟

生成式 AI 範例: 3.ESP8266 and DHT22 by MQTT 連接.mhtml







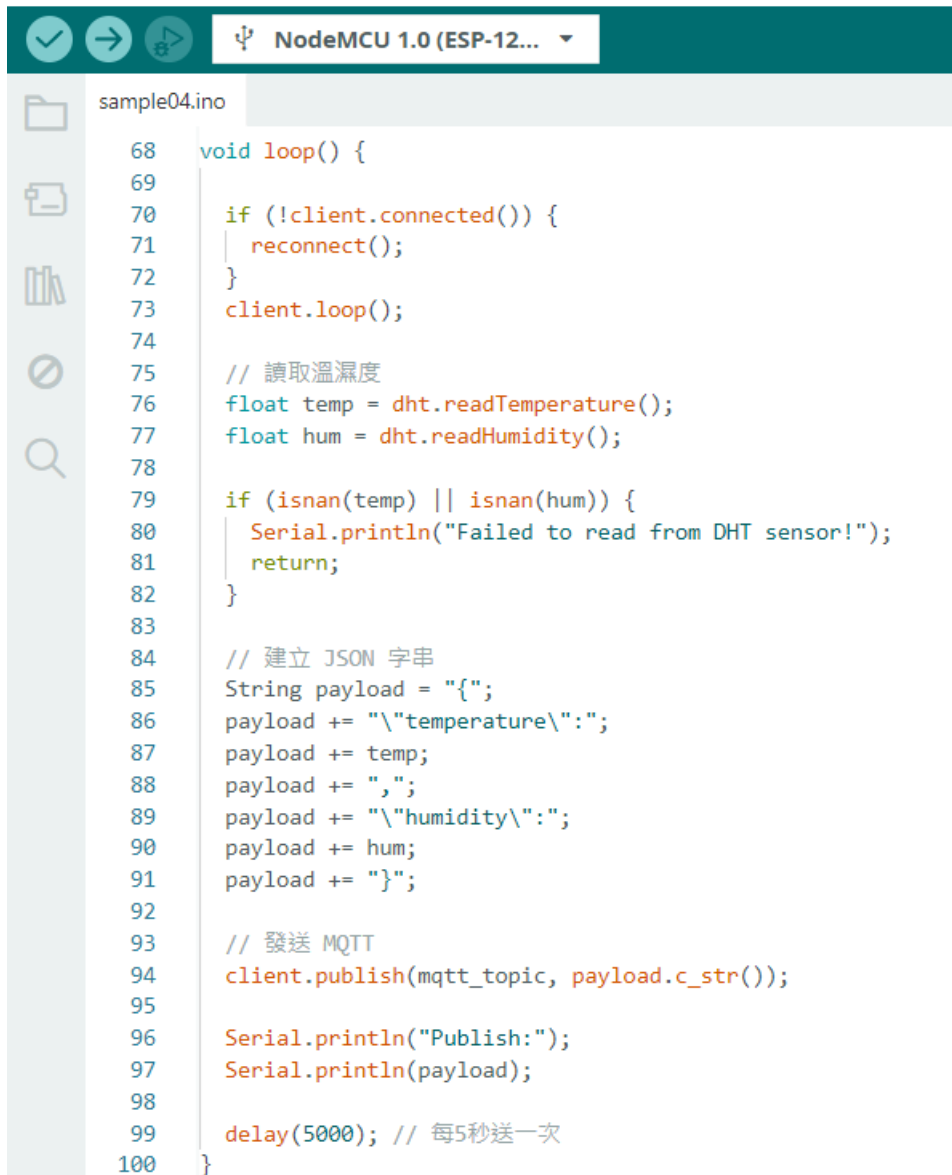
```
1  #include <ESP8266WiFi.h>
2  #include <PubSubClient.h>
3  #include <DHT.h>
4
5  // ===== WiFi 設定 =====
6  const char* ssid = "thmr305";
7  const char* password = "thmr305";
8
9  // ===== MQTT 設定 =====
10 const char* mqtt_server = "20.194.175.52";
11 const int mqtt_port = 1883;
12 const char* mqtt_user = "cubie";
13 const char* mqtt_password = "1qaz2wsx";
14 const char* mqtt_topic = "dht22/number40";
15
16 // ===== DHT 設定 =====
17 #define DHTPIN 4 // GPIO4
18 #define DHTTYPE DHT22
19 DHT dht(DHTPIN, DHTTYPE);
20
21 // ===== WiFi + MQTT =====
22 WiFiClient espClient;
23 PubSubClient client(espClient);
24
25 // ===== 接続 WiFi =====
26 void setup_wifi() {
27   delay(10);
28   Serial.println();
29   Serial.print("Connecting to ");
30   Serial.println(ssid);
31
32   WiFi.begin(ssid, password);
33
34   while (WiFi.status() != WL_CONNECTED) {
35     delay(500);
36     Serial.print(".");
37   }
38
39   Serial.println("\nWiFi connected");
40   Serial.println(WiFi.localIP());
41 }
```

sample04 | Arduino IDE 2.3.6

File Edit Sketch Tools Help



```
42
43 // ===== MQTT 重新連線 =====
44 void reconnect() {
45     while (!client.connected()) {
46         Serial.print("Attempting MQTT connection...");
47
48         if (client.connect("ESP8266Client40", mqtt_user, mqtt_password)) {
49             Serial.println("connected");
50         } else {
51             Serial.print("failed, rc=");
52             Serial.print(client.state());
53             Serial.println(" try again in 5 seconds");
54             delay(5000);
55         }
56     }
57 }
58
59 void setup() {
60     Serial.begin(115200);
61     dht.begin();
62
63     setup_wifi();
64
65     client.setServer(mqtt_server, mqtt_port);
66 }
67
```



```
68 void loop() {
69
70     if (!client.connected()) {
71         reconnect();
72     }
73     client.loop();
74
75     // 讀取溫濕度
76     float temp = dht.readTemperature();
77     float hum = dht.readHumidity();
78
79     if (isnan(temp) || isnan(hum)) {
80         Serial.println("Failed to read from DHT sensor!");
81         return;
82     }
83
84     // 建立 JSON 字串
85     String payload = "{";
86     payload += "\"temperature\":";
87     payload += temp;
88     payload += ",";
89     payload += "\"humidity\":";
90     payload += hum;
91     payload += "}";
92
93     // 發送 MQTT
94     client.publish(mqtt_topic, payload.c_str());
95
96     Serial.println("Publish:");
97     Serial.println(payload);
98
99     delay(5000); // 每5秒送一次
100 }
```

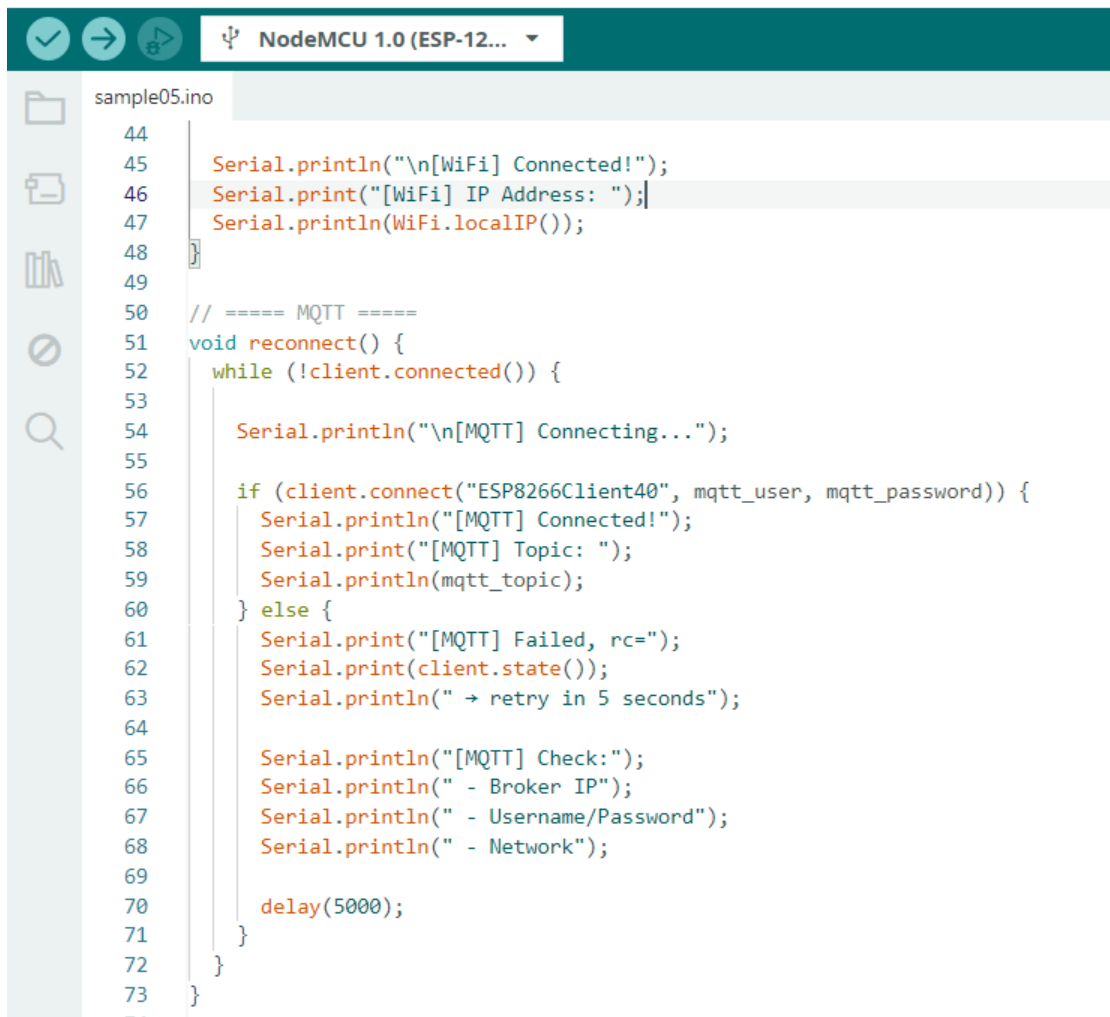
範例:sample04

sample05 | Arduino IDE 2.3.6

File Edit Sketch Tools Help



```
1  #include <ESP8266WiFi.h>
2  #include <PubSubClient.h>
3  #include <DHT.h>
4
5  // ===== WiFi =====
6  const char* ssid = "thmrb305";
7  const char* password = "thmrbthmrbthmrb";
8
9  // ===== MQTT =====
10 const char* mqtt_server = "20.194.175.52";
11 const int mqtt_port = 1883;
12 const char* mqtt_user = "cubie";
13 const char* mqtt_password = "1qaz2wsx";
14 const char* mqtt_topic = "dht22/305";
15
16 // ===== DHT =====
17 #define DHTPIN 4
18 #define DHTTYPE DHT22
19 DHT dht(DHTPIN, DHTTYPE);
20
21 // ===== Client =====
22 WiFiClient espClient;
23 PubSubClient client(espClient);
24
25 // ===== WiFi =====
26 void setup_wifi() {
27   Serial.println("\n[WiFi] Connecting...");
28   Serial.print("SSID: ");
29   Serial.println(ssid);
30
31   WiFi.begin(ssid, password);
32
33   int retry = 0;
34   while (WiFi.status() != WL_CONNECTED) {
35     delay(500);
36     Serial.print(".");
37     retry++;
38
39     if (retry > 20) {
40       Serial.println("\n[WiFi] Connection FAILED");
41       return;
42     }
43   }
```

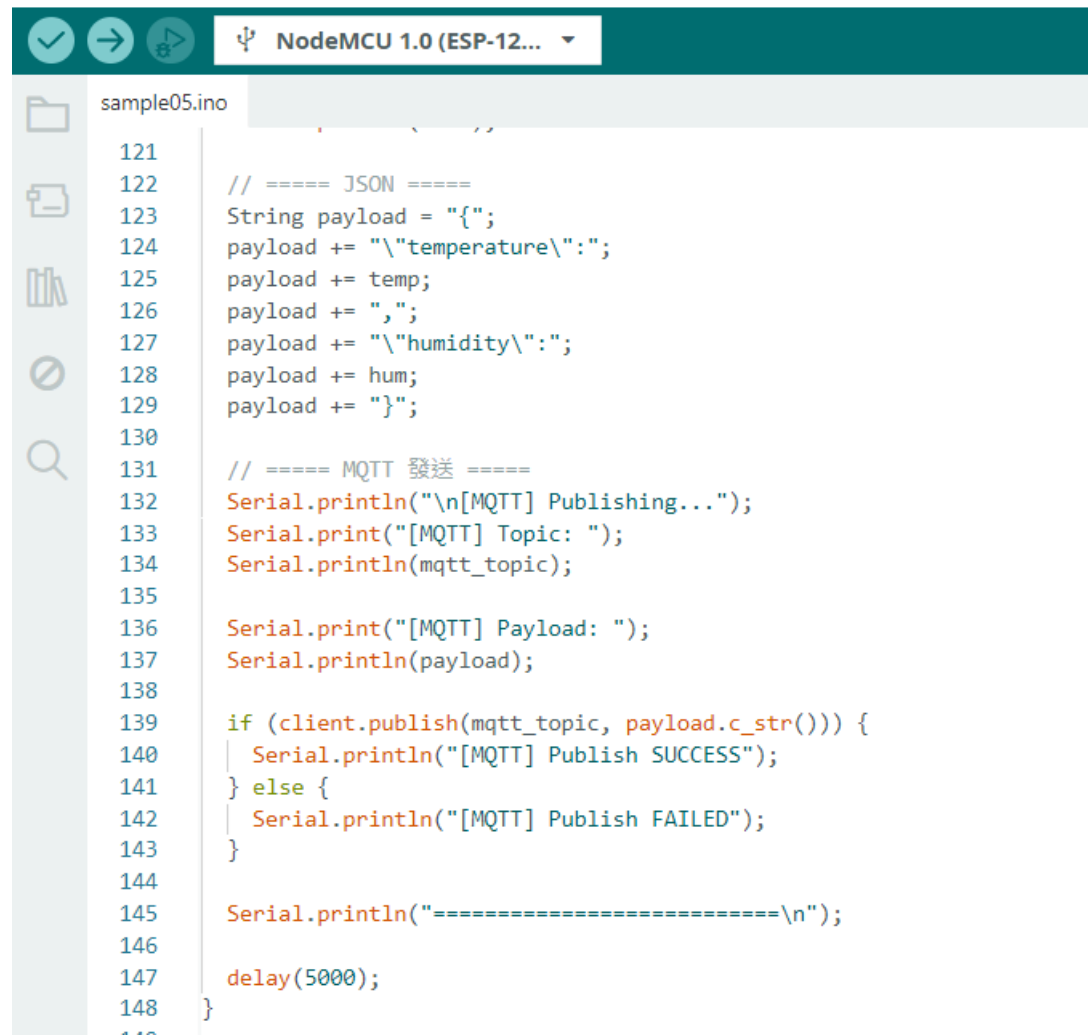


```
44
45   Serial.println("\n[WiFi] Connected!");
46   Serial.print("[WiFi] IP Address: ");
47   Serial.println(WiFi.localIP());
48 }
49
50 // ===== MQTT =====
51 void reconnect() {
52   while (!client.connected()) {
53
54     Serial.println("\n[MQTT] Connecting...");
55
56     if (client.connect("ESP8266Client40", mqtt_user, mqtt_password)) {
57       Serial.println("[MQTT] Connected!");
58       Serial.print("[MQTT] Topic: ");
59       Serial.println(mqtt_topic);
60     } else {
61       Serial.print("[MQTT] Failed, rc=");
62       Serial.print(client.state());
63       Serial.println(" → retry in 5 seconds");
64
65       Serial.println("[MQTT] Check:");
66       Serial.println(" - Broker IP");
67       Serial.println(" - Username/Password");
68       Serial.println(" - Network");
69
70       delay(5000);
71     }
72   }
73 }
```

sample05 | Arduino IDE 2.3.6

File Edit Sketch Tools Help

```
sample05.ino
/*
75 // ===== SETUP =====
76 void setup() {
77   Serial.begin(115200);
78   Serial.println("\n===== ESP8266 DHT22 MQTT START =====");
79
80   dht.begin();
81
82   setup_wifi();
83
84   client.setServer(mqtt_server, mqtt_port);
85 }
86
87 // ===== LOOP =====
88 void loop() {
89
90   // WiFi 狀態監控
91   if (WiFi.status() != WL_CONNECTED) {
92     Serial.println("[WiFi] Disconnected! Reconnecting...");
93     setup_wifi();
94   }
95
96   // MQTT 狀態監控
97   if (!client.connected()) {
98     reconnect();
99   }
100
101   client.loop();
102
103   // ===== 讀取感測器 =====
104   float temp = dht.readTemperature();
105   float hum = dht.readHumidity();
106
107   Serial.println("\n===== Sensor Data =====");
108
109   if (isnan(temp) || isnan(hum)) {
110     Serial.println("[DHT22] Read FAILED!");
111     return;
112   }
113
114   Serial.print("[DHT22] Temperature: ");
115   Serial.print(temp);
116   Serial.println(" °C");
117
118   Serial.print("[DHT22] Humidity: ");
119   Serial.print(hum);
120   Serial.println(" %");
}
```



```
121
122 // ===== JSON =====
123 String payload = "{";
124 payload += "\"temperature\":";
125 payload += temp;
126 payload += ",";
127 payload += "\"humidity\":";
128 payload += hum;
129 payload += "}";
130
131 // ===== MQTT 發送 =====
132 Serial.println("\n[MQTT] Publishing...");
133 Serial.print("[MQTT] Topic: ");
134 Serial.println(mqtt_topic);
135
136 Serial.print("[MQTT] Payload: ");
137 Serial.println(payload);
138
139 if (client.publish(mqtt_topic, payload.c_str())) {
140 | Serial.println("[MQTT] Publish SUCCESS");
141 } else {
142 | Serial.println("[MQTT] Publish FAILED");
143 }
144
145 Serial.println("=====\n");
146
147 delay(5000);
148 }
```

範例:sample05

6-5 建立 Django 顯示溫濕度系統

提問：

我已架 mqtt:20.194.175.52，

mqtt 帳號:cubie mqtt 密碼:1qaz2wsx

有三個主題，分別為:dht22/311,dht22/305,dht22/306

得到的資料格式如下: { "temperature": 27.1, "humidity": 77.9 }

我想要寫一個 django 網站，可以顯示這三個教室的即時溫濕度。

把溫濕度存到 SQLite，固定刪除，做成 Bootstrap 美化版頁面，並加入折線圖顯示溫濕度變化。

我 django 專案名為:iotclass

app 名為:myapp

請給我程式

生成式 AI 範例: 建立 Django 顯示溫濕度系統.mhtml

11. 啟動方式

先啟動 Django：

```
Bash
python manage.py runserver
```

再開另一個終端機，啟動 MQTT 訂閱：

```
Bash
python manage.py startmqtt
```

然後瀏覽器開：

```
Bash
http://127.0.0.1:8000/
```

問題:顯示時間的時域不對



問題:在 API 時間就有錯

```
{
  "305": {
    "temperature": 26.7,
    "humidity": 63.4,
    "created_at": "2026-04-20 14:09:39"
  },
  "306": {
    "temperature": null,
    "humidity": null,
    "created_at": ""
  },
  "311": {
    "temperature": null,
    "humidity": null,
    "created_at": ""
  }
}
```

尋問:圖表不要濕度

尋問:如下

時間方面我在settings.py
 TIME_ZONE = 'Asia/Taipei'

但顯示時

```
<div class="mt-3 text-muted">
  更新時間：
  <span id="time-{{ room }}">
    {% if data %}{{ data.created_at|date:"Y-m-d H:i:s" }}{%
else %}--{% endif %}
  </span>
</div>
```

少6小時

```
def latest_sensor_data(request):
    rooms = ['311', '305', '306']
    data = {}

    for room in rooms:
        latest = SensorData.objects.filter(room=room).order_by('-
created_at').first()
        if latest:
            data[room] = {
                'temperature': latest.temperature,
                'humidity': latest.humidity,
                'created_at': latest.created_at.strftime("%Y-%m-%d
%H:%M:%S")
            }
        else:
            data[room] = {
                'temperature': None,
                'humidity': None,
                'created_at': ""
            }

    return JsonResponse(data)
時間就沒顯示
TIME_ZONE = 'Asia/Taipei'
```

生成式 AI 範例：修正時間顯示.mhtml

你現在要一起改的兩個 view

1. latest_sensor_data

Python

執行

```
def latest_sensor_data(request):
    rooms = ['311', '305', '306']
    data = {}

    for room in rooms:
        latest = SensorData.objects.filter(room=room).order_by('-created_at').first()
        if latest:
            data[room] = {
                'temperature': latest.temperature,
                'humidity': latest.humidity,
                'created_at': localtime(latest.created_at).strftime('%Y-%m-%d %H:%M:%S')
            }
        else:
            data[room] = {
                'temperature': None,
                'humidity': None,
                'created_at': ''
            }

    return JsonResponse(data)
```

2. chart_data

Python

執行

```
from django.http import JsonResponse
from django.utils.timezone import localtime
from .models import SensorData

def chart_data(request, room):
    qs = SensorData.objects.filter(room=room).order_by('-created_at')[:20]
    qs = list(qs)[:1]

    labels = [localtime(obj.created_at).strftime('%H:%M:%S') for obj in qs]
    temperatures = [obj.temperature for obj in qs]

    return JsonResponse({
        'labels': labels,
        'temperatures': temperatures
    })
```

settings.py 也要確認

Python

執行

```
TIME_ZONE = 'Asia/Taipei'
USE_TZ = True
```

尋問:我想要將圖表顯示最新 500 筆

答:

⚠ 2. 你要注意的效能問題 (很重要)

直接拉 500 筆是可以，但會有這幾個影響：

🔥 (1) Chart.js 會變慢

500個點 × 每5秒更新 → 會卡

🔥 (2) 瀏覽器負擔變大

尤其在手機上會 lag

尋問: 500 筆，但可以一個區間

答:

做法 2：用時間區間顯示

例如你想要：

- 每 1 分鐘取一筆
- 或每 5 分鐘取一筆

這種是「真正的區間」。

如果你的感測器資料很多，這種方式會更漂亮。

例如每分鐘只保留最後一筆資料。

選用：

我建議你先用這個版本

最簡單，也最穩定：

Python

```
def chart_data(request, room):
    qs = SensorData.objects.filter(room=room).order_by('-created_at')[:500]
    qs = list(qs)[::-1]

    interval = 10
    qs = qs[::interval]

    labels = [localtime(obj.created_at).strftime('%H:%M:%S') for obj in qs]
    temperatures = [obj.temperature for obj in qs]

    return JsonResponse({
        'labels': labels,
        'temperatures': temperatures
    })
```

生成式 AI 範例: 修改圖表顯示資料量.mhtml

範例:iotclass

6-6 建立 Android 顯示溫溼度

➤ 顯示一個主題的溫溼度

提問：

我已架 mqtt:20.194.175.52，

mqtt 帳號:cubie mqtt 密碼:1qaz2wsx

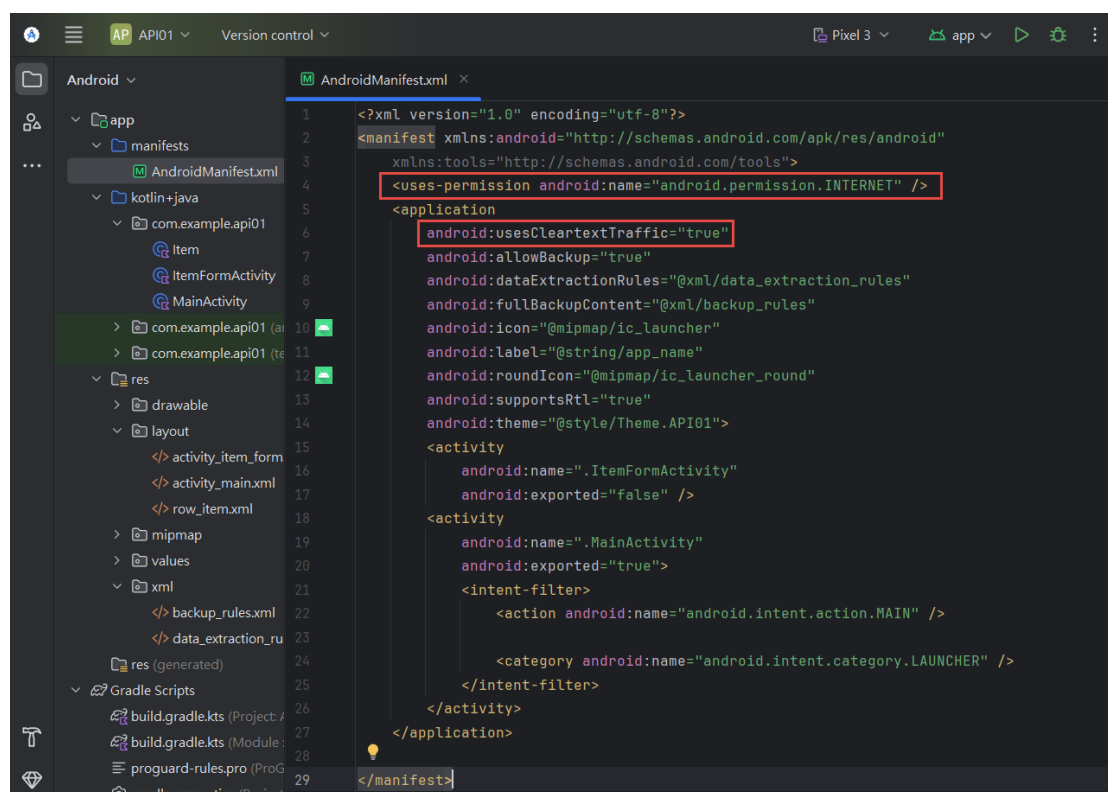
有一個主題，分別為:dht22/305

得到的資料格式如下: { "temperature": 27.1, "humidity": 77.9 }

我想要寫一個 Android 手機程式，語言為 kotlin，可以顯示這個設備的即時溫溼度。

生成式 AI 範例:建立 Android 顯示溫溼度

設定權限：



讓 App 能使用網路存取權限：

<manifest>

<uses-permission android:name="android.permission.INTERNET" />

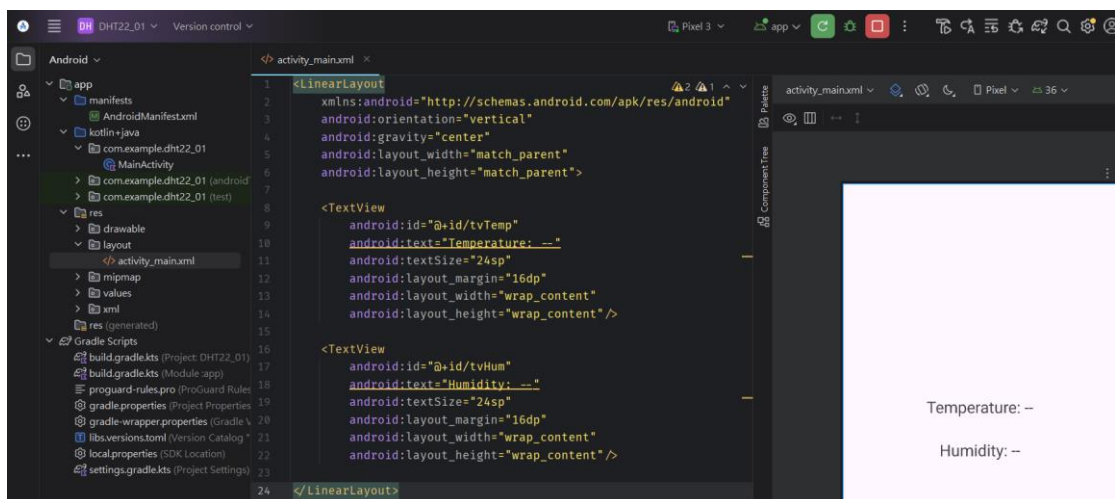
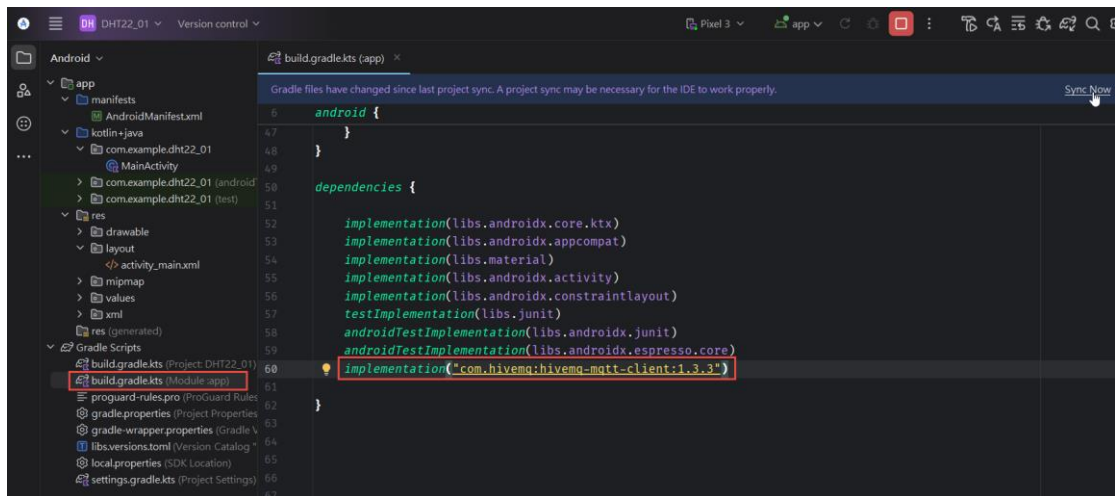
</manifest>

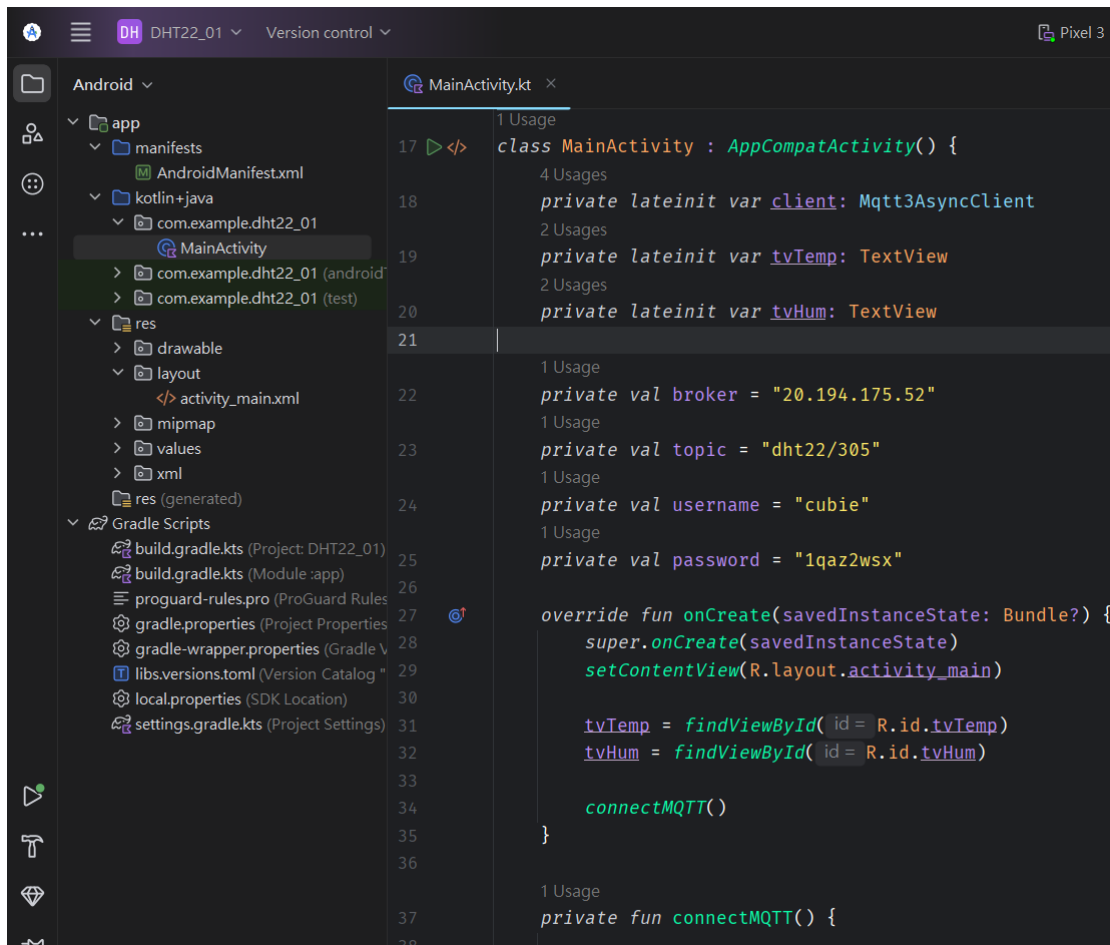
允許 HTTP 明文請求（避免被系統封鎖）：

<application>

android:usesCleartextTraffic="true"

</application>





提問：

6 files found with path 'META-INF/INDEX.LIST' from inputs:
- io.netty:netty-handler:4.1.99.Final/netty-handler-4.1.99.Final.jar
- io.netty:netty-codec:4.1.99.Final/netty-codec-4.1.99.Final.jar
- io.netty:netty-transport-native-unix-common:4.1.99.Final/netty-transport-native-unix-common-4.1.99.Final.jar
- io.netty:netty-transport:4.1.99.Final/netty-transport-4.1.99.Final.jar
- io.netty:netty-buffer:4.1.99.Final/netty-buffer-4.1.99.Final.jar
- io.netty:netty-resolver:4.1.99.Final/netty-resolver-4.1.99.Final.jar
Adding a packaging block may help, please refer to <https://developer.android.com/reference/tools/gradle-api/com/android/build/api/dsl/Packaging> for more information

這個錯誤是 Android Gradle 打包衝突 (duplicate META-INF/INDEX.LIST) ，
HiveMQ 依賴了 Netty，而 Netty 多個 JAR 都包含同一個檔案 📁 META-INF/INDEX.LIST

📁 這是常見問題，解法很簡單：排除它

🔥 如果還有其他 META-INF 衝突 (建議一次處理)

你可以用「強化版」：

```
<> gradle
android {
    packagingOptions {
        resources {
            excludes += [
                "META-INF/INDEX.LIST",
                "META-INF/io.netty.versions.properties",
                "META-INF/DEPENDENCIES",
                "META-INF/LICENSE",
                "META-INF/LICENSE.txt",
                "META-INF/NOTICE",
                "META-INF/NOTICE.txt"
            ]
        }
    }
}
```

```
android {
    packagingOptions {
        resources {
            excludes += [
                "META-INF/INDEX.LIST",
                "META-INF/io.netty.versions.properties",
                "META-INF/DEPENDENCIES",
                "META-INF/LICENSE",
                "META-INF/LICENSE.txt",
                "META-INF/NOTICE",
                "META-INF/NOTICE.txt"
            ]
        }
    }
}格式有錯
```

你這段其實邏輯是對的，但錯在 Gradle 語法版本差異 🍌

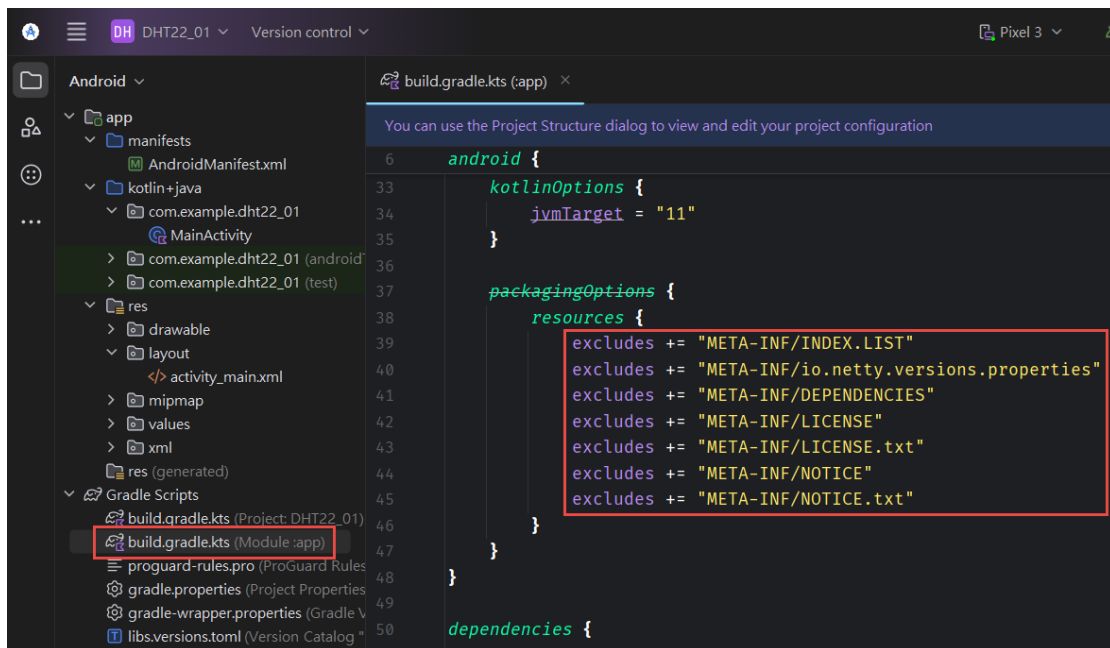
(很多人會卡在這裡)

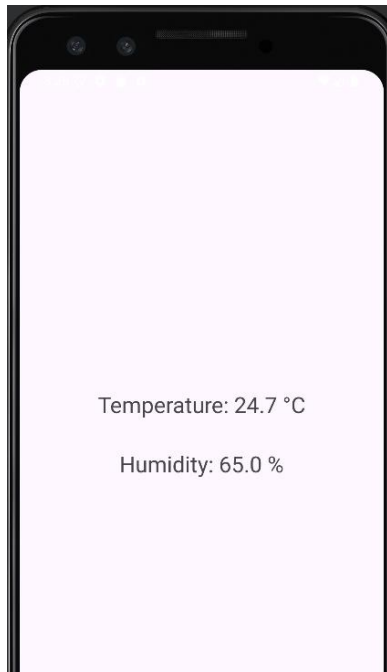
◆ 情況 2：如果你用 Kotlin DSL (build.gradle.kts)

才可以用這種👉

```
Kotlin

android {
    packaging {
        resources {
            excludes += setOf(
                "META-INF/INDEX.LIST",
                "META-INF/io.netty.versions.properties",
                "META-INF/DEPENDENCIES",
                "META-INF/LICENSE",
                "META-INF/LICENSE.txt",
                "META-INF/NOTICE",
                "META-INF/NOTICE.txt"
            )
        }
    }
}
```





範例: DHT22_01

➤ 顯示一個主題的溫溼度+歷史訊息

提問：

我已架 mqtt:20.194.175.52，

mqtt 帳號:cubie mqtt 密碼:1qaz2wsx

有一個主題，分別為:dht22/305

得到的資料格式如下: { "temperature": 27.1, "humidity": 77.9 }

有一個 API: <http://192.168.57.246:8080/api/chart/305/>，

可以得到如下的歷史訊息:

`{"labels":`

`["09:34:15","09:35:36","09:38:04","09:39:00","09:40:06","09:41:12","09:42:28","09:43:55","09:44:46","09:45:37","09:46:28","09:47:19","09:48:10","09:49:11","09:50:28","09:51:19","09:52:15"],`

`"temperatures":
[24.3,24.5,24.7,24.7,24.7,24.8,24.9,24.9]}`

我想要寫一個 Android 手機程式，

語言為 kotlin，

使用 `implementation("com.hivemq:hivemq-mqtt-client:1.3.3")`連 mqtt

可以顯示這個設備的即時溫濕度與歷史訊息(用圖表顯示)。

請給我程式碼與 layout code

生成式 AI 範例:建立 Android 顯示溫溼度



```
{
  "labels": [ ... ], // 50 items
  "temperatures": [ ... ] // 50 items
}
```

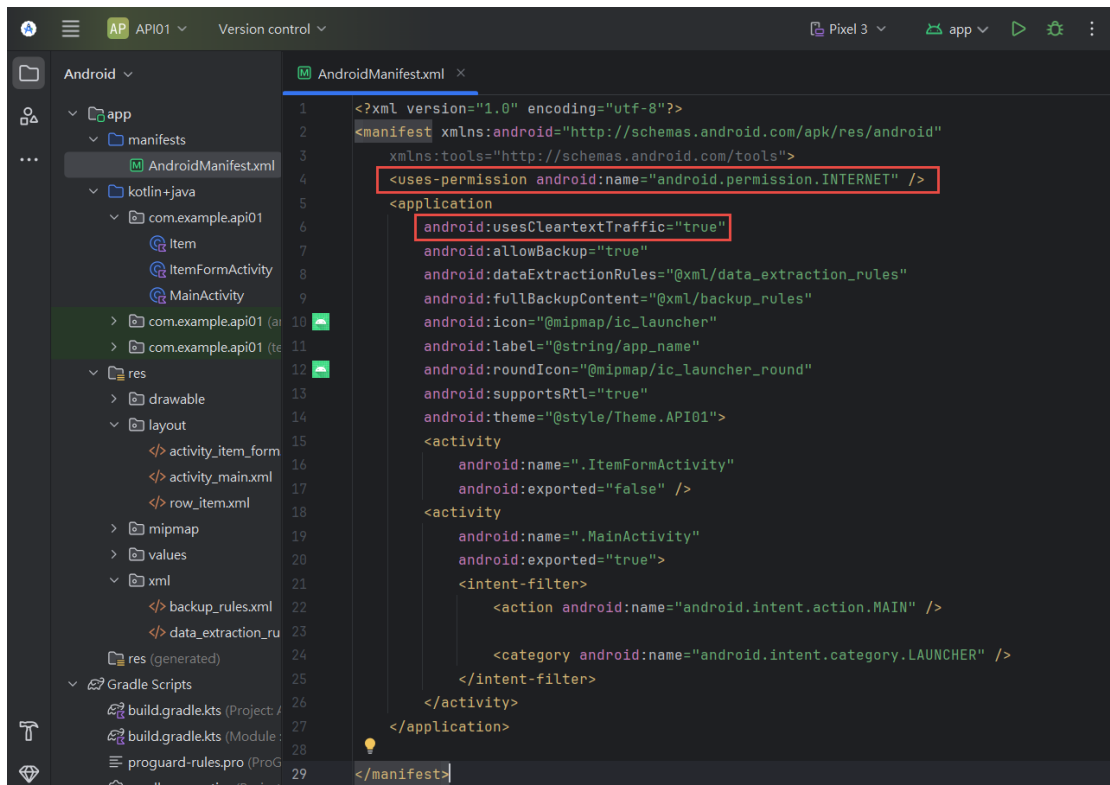


```
{
  "labels": [
    "09:34:15",
    "09:35:36",
    "09:38:04",
    "09:39:00",
    "09:40:06",
    "09:41:12",
    "09:42:28",
    "09:43:55",
    "09:44:46",
  ]
}
```



```
{
  "labels": [ ... ], // 50 items
  "temperatures": [
    24.3,
    24.5,
    24.7,
    24.7,
    24.7,
    24.8,
    24.9,
  ]
}
```

設定權限：



讓 App 能使用網路存取權限：

<manifest>

<uses-permission android:name="android.permission.INTERNET" />

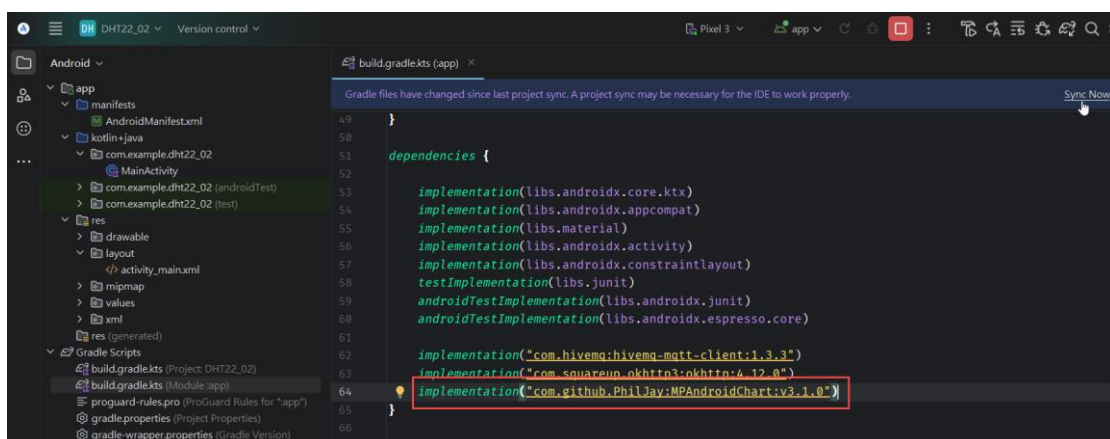
</manifest>

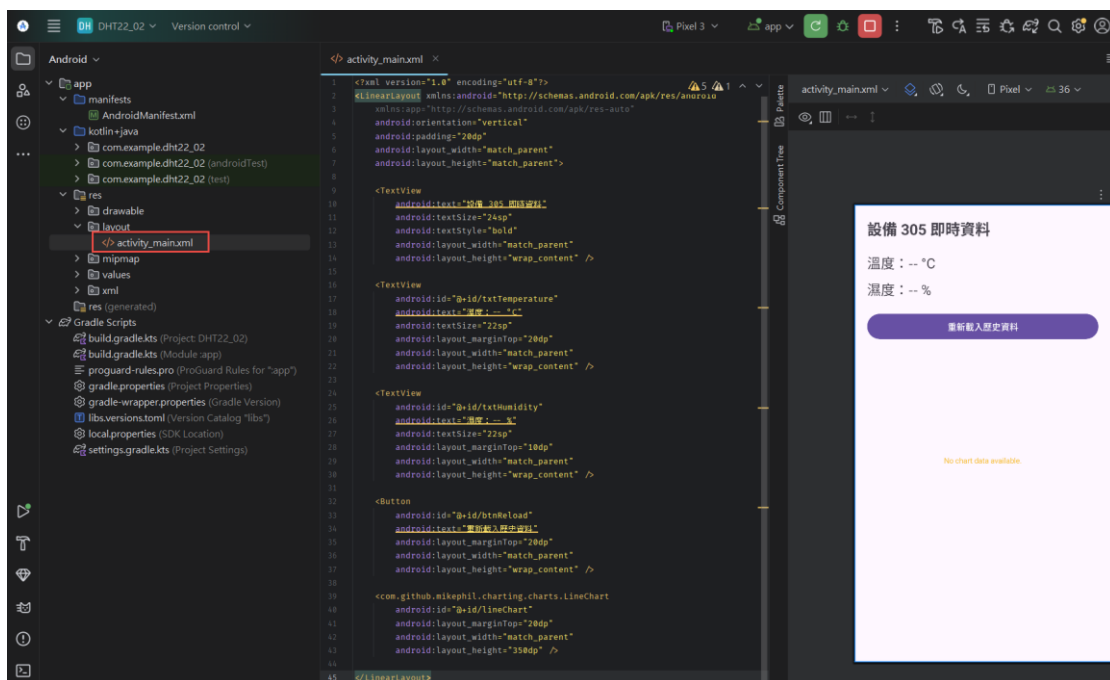
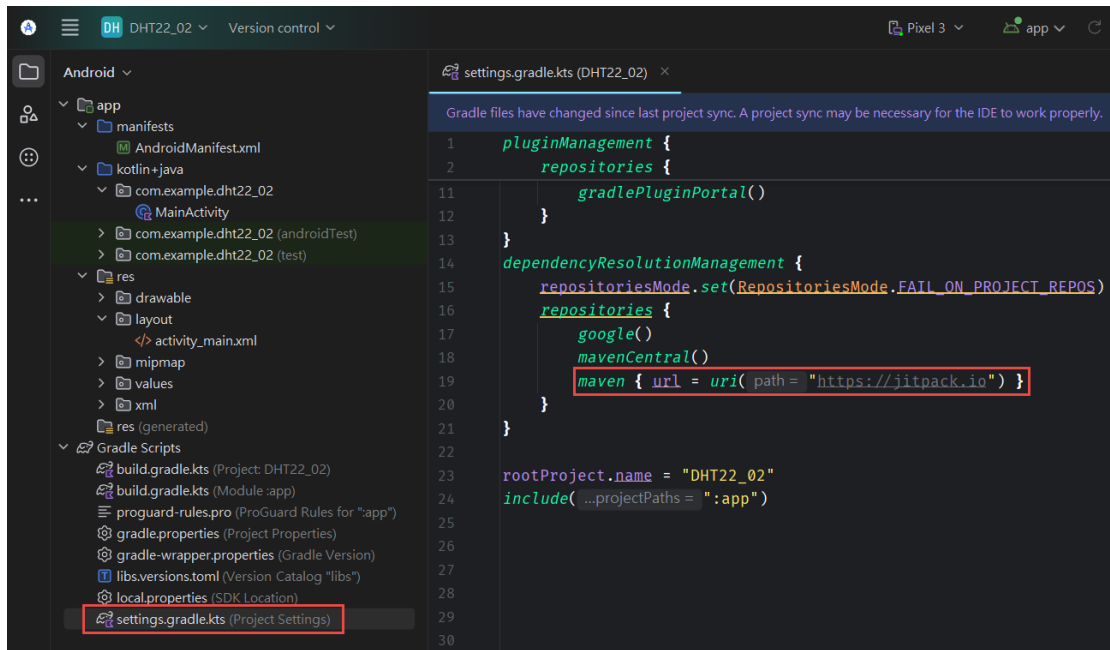
允許 HTTP 明文請求（避免被系統封鎖）：

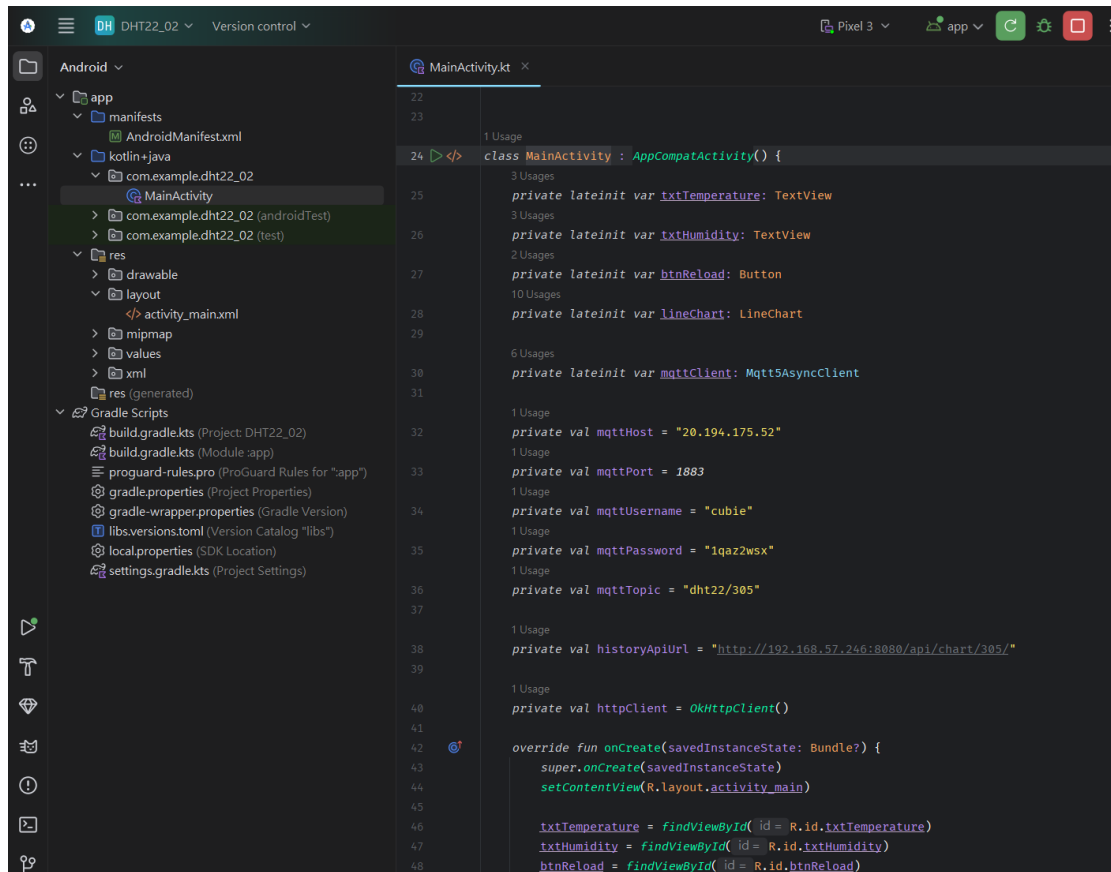
<application>

android:usesCleartextTraffic="true"

</application>







提問：

6 files found with path 'META-INF/INDEX.LIST' from inputs:
 - io.netty:netty-handler:4.1.99.Final/netty-handler-4.1.99.Final.jar
 - io.netty:netty-codec:4.1.99.Final/netty-codec-4.1.99.Final.jar
 - io.netty:netty-transport-native-unix-common:4.1.99.Final/netty-transport-native-unix-common-4.1.99.Final.jar
 - io.netty:netty-transport:4.1.99.Final/netty-transport-4.1.99.Final.jar
 - io.netty:netty-buffer:4.1.99.Final/netty-buffer-4.1.99.Final.jar
 - io.netty:netty-resolver:4.1.99.Final/netty-resolver-4.1.99.Final.jar
 Adding a packaging block may help, please refer to <https://developer.android.com/reference/tools/gradle-api/com/android/build/api/dsl/Packaging> for more information

這個錯誤是 **Android 常見的 Netty 衝突問題** (HiveMQ MQTT client 會用到 Netty)，原因是多個 `.jar` 裡都有同一個檔案：

META-INF/INDEX.LIST



👉 Android build 時不允許重複資源，所以要「排除」

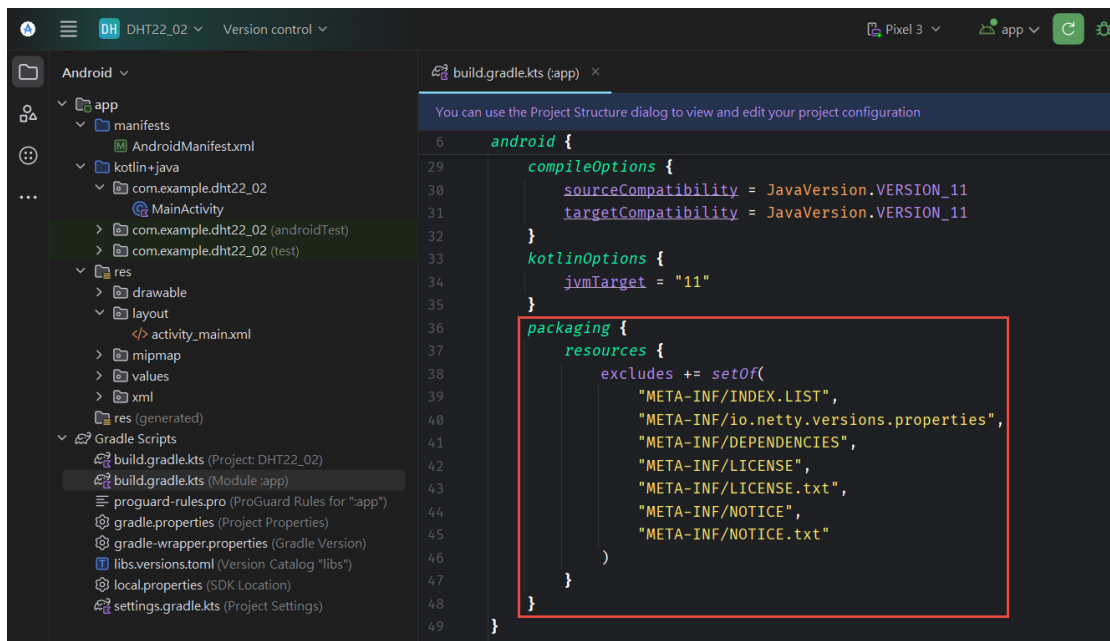


🔧 如果還有錯 (建議完整版)

有時不只一個 META-INF 衝突，可以直接用這個比較完整的版本：

```
 Kotlin

android {
    packaging {
        resources {
            excludes += setOf(
                "META-INF/INDEX.LIST",
                "META-INF/io.netty.versions.properties",
                "META-INF/DEPENDENCIES",
                "META-INF/LICENSE",
                "META-INF/LICENSE.txt",
                "META-INF/NOTICE",
                "META-INF/NOTICE.txt"
            )
        }
    }
}
```





範例: DHT22_02

- 顯示三個主題的溫溼度
- 顯示三個主題的溫溼度+歷史訊息

6-7 ESP8266 and DHT22 Sensor Using HTTP POST Request

提問:

生成式 AI 範例: 4.ESP8266 and DHT22 POST 傳送資料.mhtml

```
sample06 | Arduino IDE 2.3.6
File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12...

sample06.ino
1  #include <ESP8266WiFi.h>
2  #include <ESP8266HTTPClient.h>
3  #include <WiFiClient.h>
4  #include <DHT.h>
5
6  // ===== WiFi 設定 =====
7  const char* ssid = "thmr305";
8  const char* password = "thmr305";
9
10 // ===== API 設定 =====
11 const char* serverUrl = "http://192.168.57.246:8080/add_temperature_API/";
12
13 // ===== DHT22 設定 =====
14 #define DHTPIN 4 // GPIO4 = NodeMCU D2
15 #define DHTTYPE DHT22
16
17 DHT dht(DHTPIN, DHTTYPE);
18
19 void connectWiFi() {
20   WiFi.begin(ssid, password);
21   Serial.print("正在連線 WiFi");
22
23   while (WiFi.status() != WL_CONNECTED) {
24     delay(500);
25     Serial.print(".");
26   }
27
28   Serial.println();
29   Serial.println("WiFi 已連線");
30   Serial.print("IP 位址: ");
31   Serial.println(WiFi.localIP());
32 }
```

sample06 | Arduino IDE 2.3.6

File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12...)

sample06.ino

```

33
34 void setup() {
35   Serial.begin(115200);
36   delay(1000);
37
38   dht.begin();
39   connectWifi();
40 }
41
42 void loop() {
43   // 若 WiFi 斷線，自動重連
44   if (WiFi.status() != WL_CONNECTED) {
45     Serial.println("WiFi 斷線，重新連線中...");
46     connectWifi();
47   }
48
49   float humidity = dht.readHumidity();
50   float temperature = dht.readTemperature(); // 攝氏
51
52   // 檢查是否讀取失敗
53   if (isnan(humidity) || isnan(temperature)) {
54     Serial.println("DHT22 讀取失敗，5 秒後重試");
55     delay(5000);
56     return;
57   }
58
59   Serial.print("溫度: ");
60   Serial.print(temperature);
61   Serial.print(" °C, 濕度: ");
62   Serial.print(humidity);
63   Serial.println(" %");

```

sample06 | Arduino IDE 2.3.6

File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12...)

sample06.ino

```

64
65   WiFiClient client;
66   HTTPClient http;
67
68   http.begin(client, serverUrl);
69   http.addHeader("Content-Type", "application/x-www-form-urlencoded");
70
71   String postData = "sensor_id=2";
72   postData += "&temperature=" + String(temperature, 2);
73   postData += "&humidity=" + String(humidity, 2);
74
75   int httpResponseCode = http.POST(postData);
76
77   Serial.print("HTTP 回應碼: ");
78   Serial.println(httpResponseCode);
79
80   if (httpResponseCode > 0) {
81     String response = http.getString();
82     Serial.println("伺服器回應內容:");
83     Serial.println(response);
84   } else {
85     Serial.print("POST 失敗，錯誤: ");
86     Serial.println(http.errorToString(httpResponseCode));
87   }
88
89   http.end();
90
91   // 每 10 秒送一次
92   delay(10000);
93 }

```

範例:sample06

6-8 ESP8266 with 4-Channel Relay and Sensor Integration using MQTT

提問：

我的開發板是 ESP8266，我使用的模組有 4 路繼電器模組，以及一個雙路 5V/3.3V 電源模組。

我想要將 4 路繼電器模組電源接到雙路 5V/3.3V 電源模組。

請問這些元件要如何接線？

程式要怎麼寫，才能讓我分別控制 第 1 路到第 4 路繼電器的開啟與關閉？

希望能提供完整的接線方式與範例程式。

生成式 AI 範例: ESP8266 繼電器接線與程式



2. ESP8266 與 4 路繼電器控制腳接法

建議使用這 4 個 GPIO：

- D1 → Relay IN1
- D2 → Relay IN2
- D5 → Relay IN3
- D6 → Relay IN4

也就是：

- ESP8266 D1 (GPIO5) → 繼電器模組 IN1
- ESP8266 D2 (GPIO4) → 繼電器模組 IN2
- ESP8266 D5 (GPIO14) → 繼電器模組 IN3
- ESP8266 D6 (GPIO12) → 繼電器模組 IN4

四、接線示意圖

雙路 5V/3.3V 電源模組

```

-----
3.3V -----> ESP8266 3V3
GND -----> ESP8266 GND
5V -----> 4路繼電器模組 VCC
GND -----> 4路繼電器模組 GND

ESP8266
-----
D1(GPIO5) -----> Relay IN1
D2(GPIO4) -----> Relay IN2
D5(GPIO14) -----> Relay IN3
D6(GPIO12) -----> Relay IN4
GND -----> Relay GND

```

六、Arduino IDE 範例程式

這份程式可讓你依序控制第 1 路到第 4 路繼電器開關。

範例 1：依序開啟與關閉 4 路繼電器

```

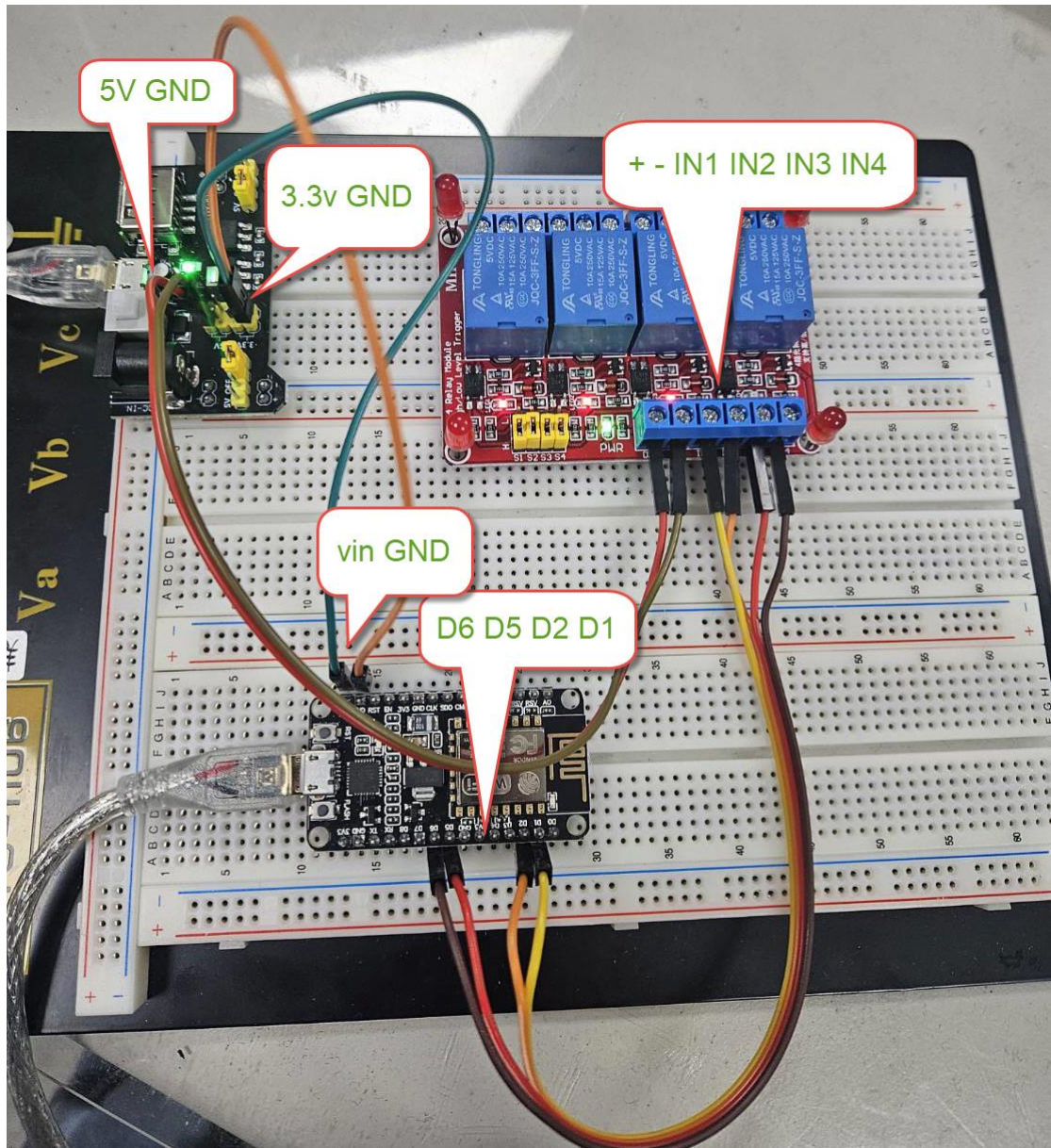
<> C++
// ESP8266 Relay Control
// 假設繼電器模組為「低電位觸發」
// LOW = 開啟繼電器
// HIGH = 關閉繼電器

#define RELAY1 D1 // GPIO5
#define RELAY2 D2 // GPIO4
#define RELAY3 D5 // GPIO14
#define RELAY4 D6 // GPIO12

void setup() {
  Serial.begin(115200);

  pinMode(RELAY1, OUTPUT);
  pinMode(RELAY2, OUTPUT);
  pinMode(RELAY3, OUTPUT);
  pinMode(RELAY4, OUTPUT);
}

```



realy01 | Arduino IDE 2.3.6

File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12E M...

realy01.ino

```
1 // ESP8266 Relay Control
2 // 假設繼電器模組為「低電位觸發」
3 // LOW = 開啟繼電器
4 // HIGH = 關閉繼電器
5
6 #define RELAY1 D1 // GPIO5
7 #define RELAY2 D2 // GPIO4
8 #define RELAY3 D5 // GPIO14
9 #define RELAY4 D6 // GPIO12
10
11 void setup() {
12     Serial.begin(115200);
13
14     pinMode(RELAY1, OUTPUT);
15     pinMode(RELAY2, OUTPUT);
16     pinMode(RELAY3, OUTPUT);
17     pinMode(RELAY4, OUTPUT);
18
19     // 一開始全部關閉
20     digitalWrite(RELAY1, HIGH);
21     digitalWrite(RELAY2, HIGH);
22     digitalWrite(RELAY3, HIGH);
23     digitalWrite(RELAY4, HIGH);
24
25     Serial.println("Relay control start...");
26 }
27
28 void loop() {
29     // 第1路開
30     digitalWrite(RELAY1, LOW);
31     Serial.println("Relay 1 ON");
32     delay(2000);
33
34     // 第1路關
35     digitalWrite(RELAY1, HIGH);
36     Serial.println("Relay 1 OFF");
37     delay(1000);
38 }
```

範例:relay01

八、如果你想用序列埠手動控制 1~4 路

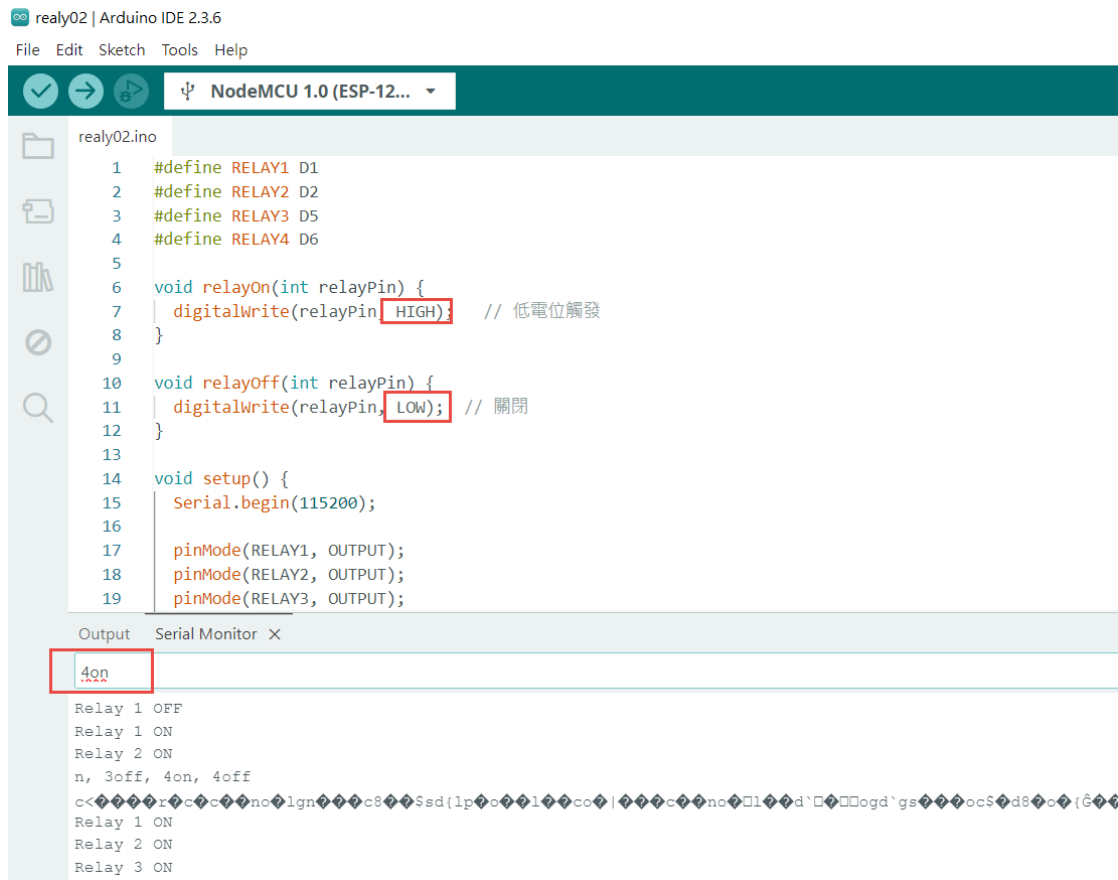
你可以在序列監控視窗輸入指令：

- 1on
- 1off
- 2on
- 2off
- 3on
- 3off
- 4on
- 4off

範例 3：序列埠個別控制

```
</> C++  
  
#define RELAY1 D1  
#define RELAY2 D2  
#define RELAY3 D5  
#define RELAY4 D6  
  
void relayOn(int relayPin) {  
    digitalWrite(relayPin, LOW); // 低電位觸發  
}
```

將程式中 HIGH 與 LOW 交換 (高電位:開、低電位:關)



```

1  #define RELAY1 D1
2  #define RELAY2 D2
3  #define RELAY3 D5
4  #define RELAY4 D6
5
6  void relayOn(int relayPin) {
7      digitalWrite(relayPin, HIGH); // 低電位觸發
8  }
9
10 void relayOff(int relayPin) {
11     digitalWrite(relayPin, LOW); // 關閉
12 }
13
14 void setup() {
15     Serial.begin(115200);
16
17     pinMode(RELAY1, OUTPUT);
18     pinMode(RELAY2, OUTPUT);
19     pinMode(RELAY3, OUTPUT);
20     pinMode(RELAY4, OUTPUT);
21
22     relayOff(RELAY1);
23     relayOff(RELAY2);
24     relayOff(RELAY3);
25     relayOff(RELAY4);
26
27     Serial.println("請輸入指令: 1on, 1off, 2on, 2off, 3on, 3off, 4on, 4off");
28 }
29
30 void loop() {
31     if (Serial.available()) {
32         String cmd = Serial.readStringUntil('\n');
33         cmd.trim();
34
35         if (cmd == "1on") {
36             relayOn(RELAY1);
37             Serial.println("Relay 1 ON");

```

範例:relay02

提問:

我的版本為 esp8266 連接到 4 pin relay 繼電器，

連接腳位為:D1、D2、D5、D6

我想要將 esp8266 當作 subscribe 角色連到

mqtt_server ip :20.194.175.52

port = 1883

user = cubie

password=1qaz2wsx

topic = "relay_control/teacher"

published 傳送的格式為

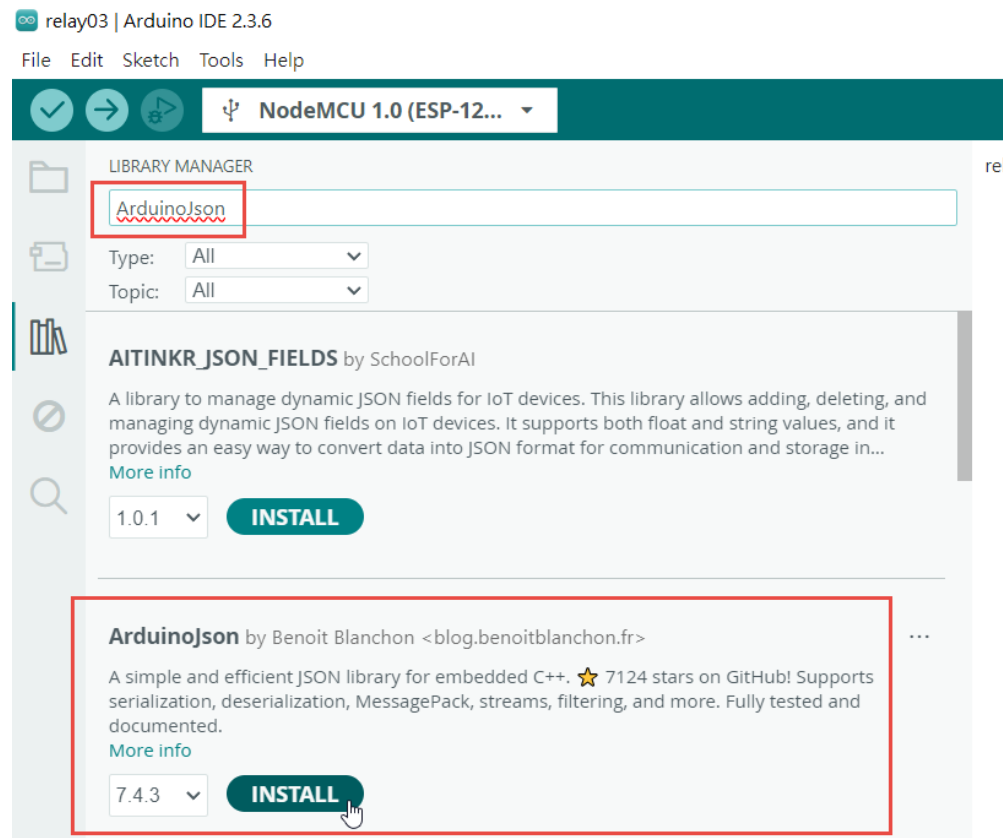
開: {"relay1": "on", "relay2": "on", "relay3": "on", "relay4": "on"}

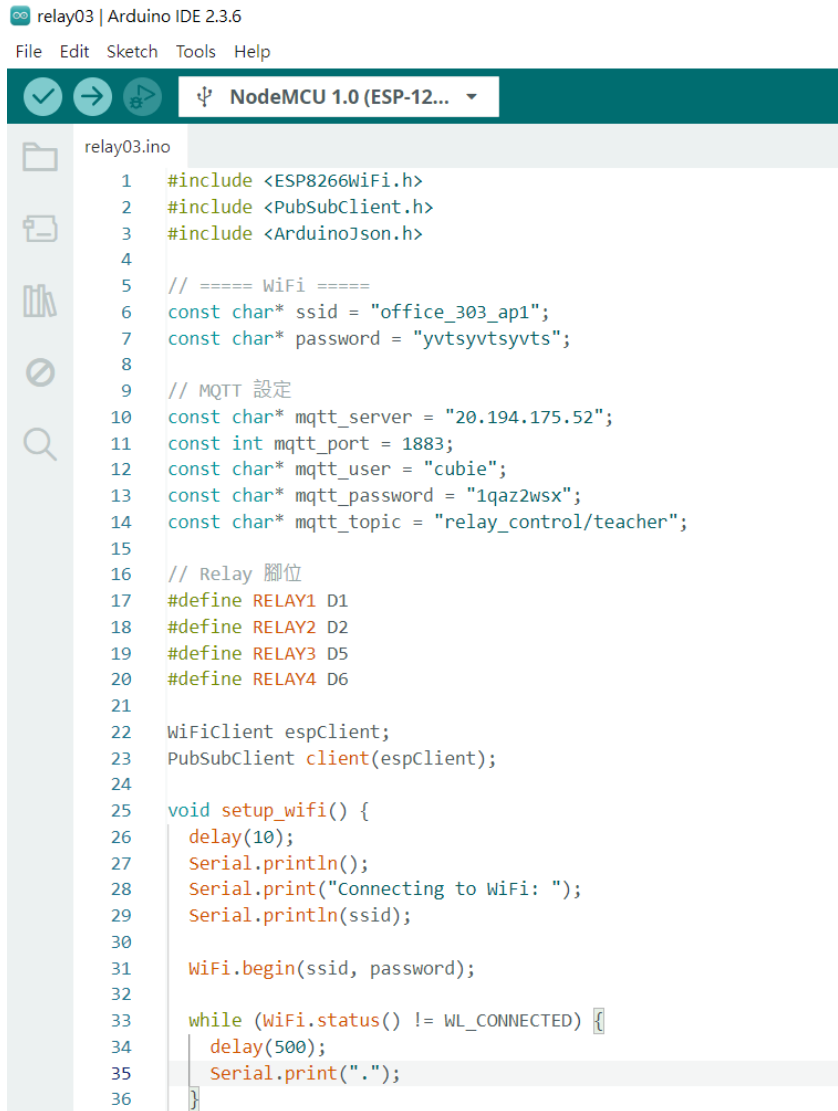
關: {"relay1": "off", "relay2": "off", "relay3": "off", "relay4": "off"}

低電位為關、高電位為開

請給我程式

生成式 AI 範例：ESP8266 with 4-Channel Relay and Sensor Integration using MQTT.mhtml

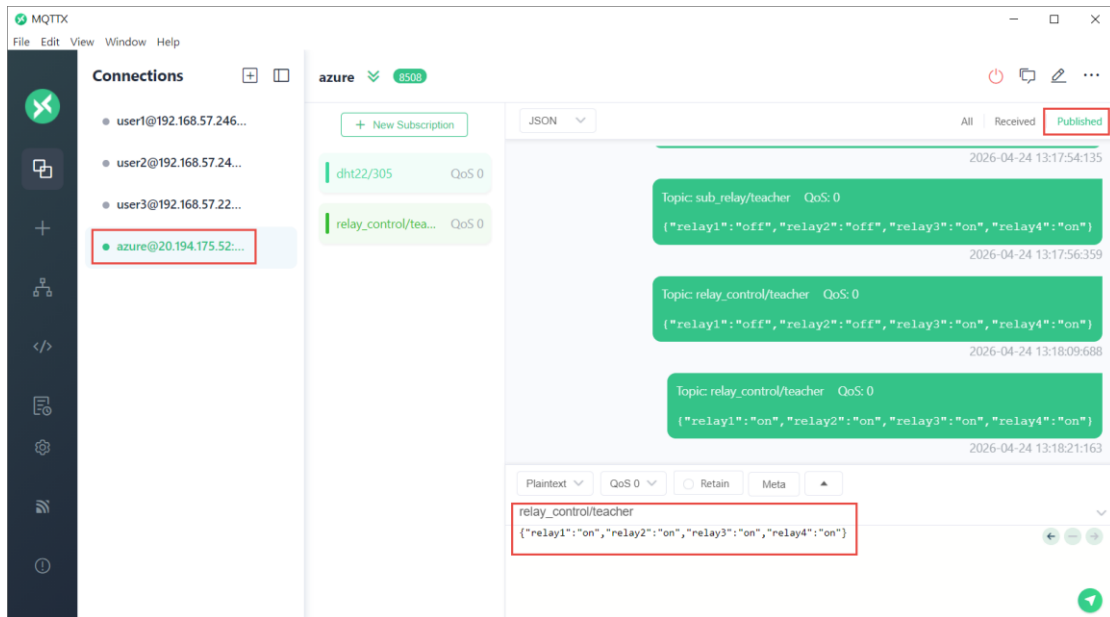




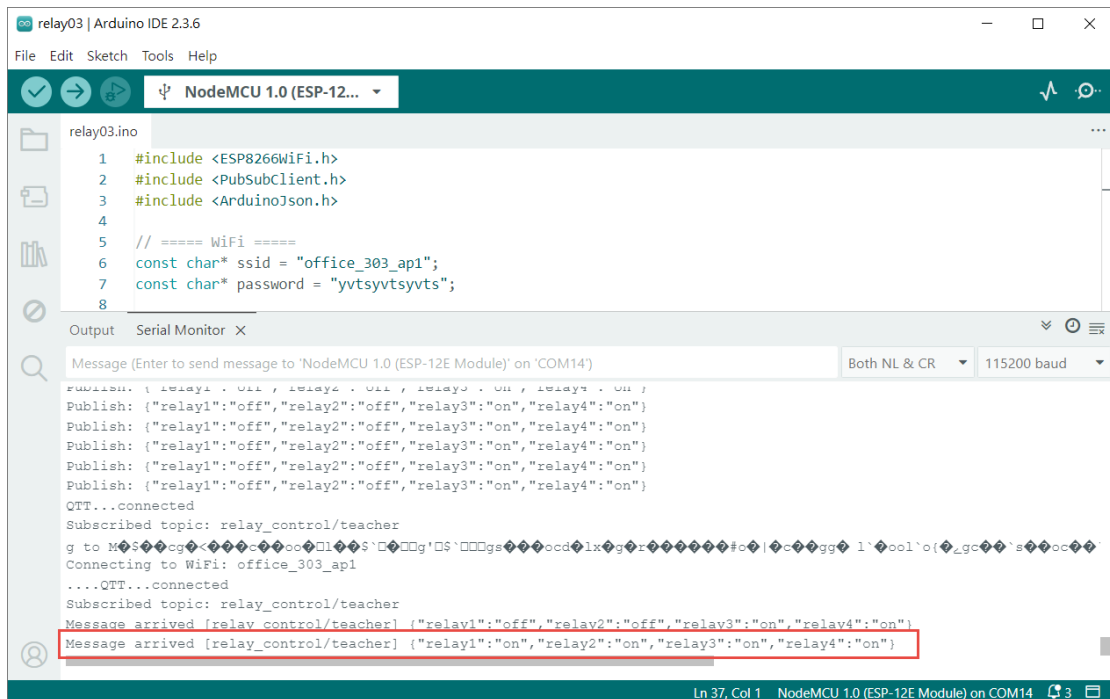
```
1 #include <ESP8266WiFi.h>
2 #include <PubSubClient.h>
3 #include <ArduinoJson.h>
4
5 // ===== WiFi =====
6 const char* ssid = "office_303_ap1";
7 const char* password = "yvtsyvtsyvts";
8
9 // MQTT 設定
10 const char* mqtt_server = "20.194.175.52";
11 const int mqtt_port = 1883;
12 const char* mqtt_user = "cubie";
13 const char* mqtt_password = "1qaz2wsx";
14 const char* mqtt_topic = "relay_control/teacher";
15
16 // Relay 腳位
17 #define RELAY1 D1
18 #define RELAY2 D2
19 #define RELAY3 D5
20 #define RELAY4 D6
21
22 WiFiClient espClient;
23 PubSubClient client(espClient);
24
25 void setup_wifi() {
26   delay(10);
27   Serial.println();
28   Serial.print("Connecting to WiFi: ");
29   Serial.println(ssid);
30
31   WiFi.begin(ssid, password);
32
33   while (WiFi.status() != WL_CONNECTED) {
34     delay(500);
35     Serial.print(".");
36   }
```

範例:relay03

使用 MQTTX 當作 published



esp8266 為 subscribe 角色



提問:

我的板子為 esp8266，此設備連接到 4 pin relay 繼電器，
連接腳位為:D1、D2、D5、D6

我想要將 esp8266 當作 subscribe 角色同時也是 published 角色

subscribe 角色(接收訊號，將 relay 開或關):

接收到其它設備的訊號格式如下:

開: {"relay1":"on","relay2":"on","relay3":"on", "relay4":"on"}

關: {"relay1":"off","relay2":"off","relay3":"off", "relay4":"off"}

連到

mqtt_server ip :20.194.175.52

port = 1883

user = cubie

password=1qaz2wsx

topic = " relay_ control/teacher"

published 角色(將現階段的四個 relay 開與關發送出去)

發送訊號格式如下，要固定時間發布:

開: {"relay1":"on","relay2":"on","relay3":"on", "relay4":"on"}

關: {"relay1":"off","relay2":"off","relay3":"off", "relay4":"off"}

連到

mqtt_server ip :20.194.175.52

port = 1883

user = cubie

password=1qaz2wsx

topic = "relay_status/teacher"

請給我程式

我的 relay 是高電位觸發

生成式 AI 範例：ESP8266 with 4-Channel Relay and Sensor Integration using MQTT(sub and pub).mhtml

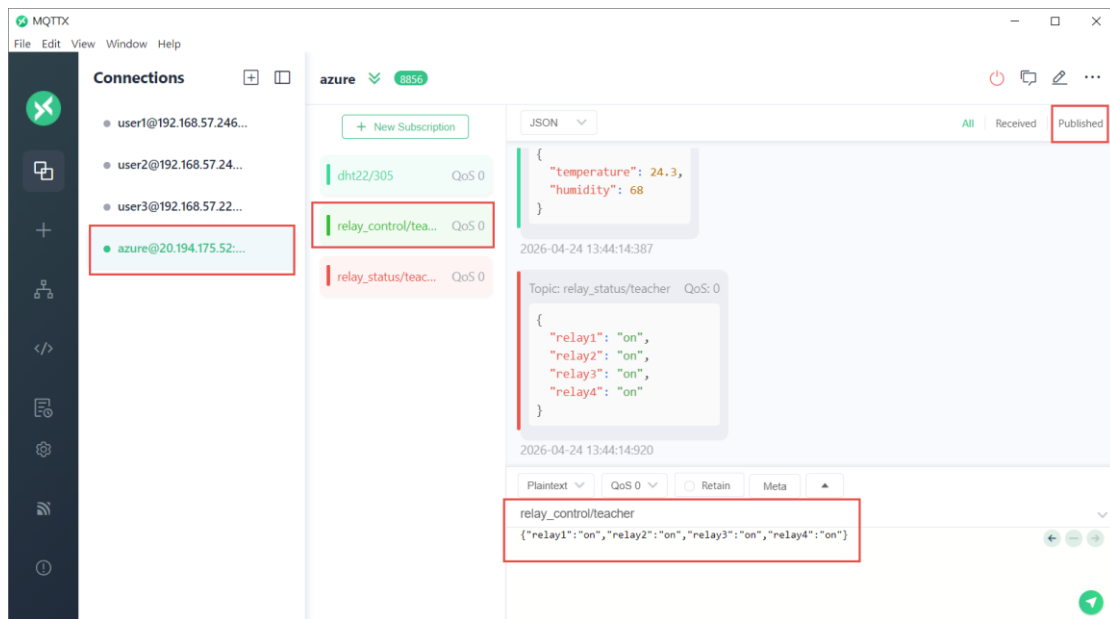
```

relay04.ino
1  #include <ESP8266WiFi.h>
2  #include <PubSubClient.h>
3  #include <ArduinoJson.h>
4
5  // ===== WiFi =====
6  const char* ssid = "office_303_ap1";
7  const char* password = "yvtsyvtsyvts";
8
9  // MQTT 設定
10 const char* mqtt_server = "20.194.175.52";
11 const int mqtt_port = 1883;
12 const char* mqtt_user = "cubie";
13 const char* mqtt_password = "1qaz2wsx";
14
15 // Topic
16 const char* sub_topic = "relay_control/teacher";
17 const char* pub_topic = "relay_status/teacher";
18
19 // Relay 腳位
20 #define RELAY1 D1
21 #define RELAY2 D2
22 #define RELAY3 D5
23 #define RELAY4 D6
24
25 WiFiClient espClient;
26 PubSubClient client(espClient);
27
28 // 高電位觸發
29 String relay1State = "off";
30 String relay2State = "off";
31 String relay3State = "off";
32 String relay4State = "off";
33
34 unsigned long lastPublishTime = 0;
35 const unsigned long publishInterval = 5000; // 每 5 秒發布一次

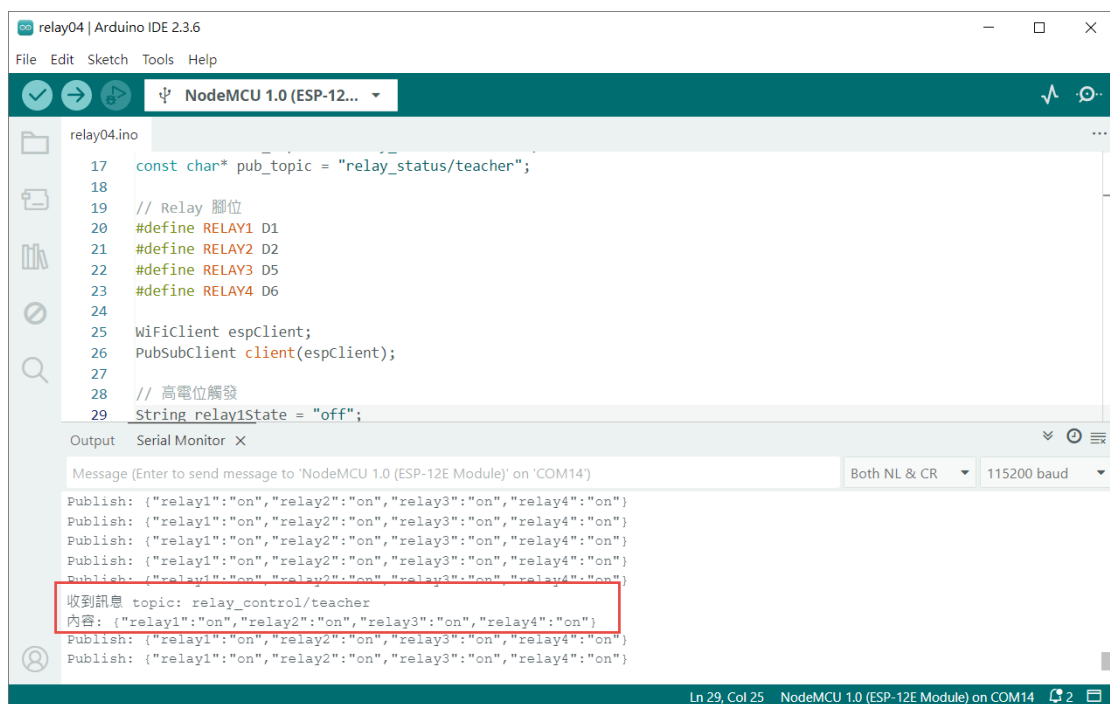
```

範例:relay04

MQTTX 當作 published 角色，發送控制四個 relay 的 on or off



esp8266 為 subscribe 角色(接收訊號，將 relay 開或關)



esp8266 為 subscribe 角色(接收訊號，將 relay 開或關)

同時也是

esp8266 為 publish 角色(將現階段的四個 relay 開與關發送出去)

6-9 建立 Django 控制 4 pin relay

提問：

esp8266 當作 subscribe 角色同時也是 published 角色

subscribe 角色(接收訊號，將 relay 開或關)：

接收到其它設備的訊號格式如下：

開: {"relay1": "on", "relay2": "on", "relay3": "on", "relay4": "on"}

關: {"relay1": "off", "relay2": "off", "relay3": "off", "relay4": "off"}

mqtt_server ip :20.194.175.52

port = 1883

user = cubie

password=1qaz2wsx

topic = "relay_control/teacher"

published 角色(將現階段的四個 relay 開與關發送出去)

發送訊號格式如下：

開: {"relay1": "on", "relay2": "on", "relay3": "on", "relay4": "on"}

關: {"relay1": "off", "relay2": "off", "relay3": "off", "relay4": "off"}

mqtt_server ip :20.194.175.52

port = 1883

user = cubie

password=1qaz2wsx

topic = "relay_status/teacher"

我想要設計一個 django 網站，他是 published 角色(發送訊號)，同時也是 subscribe 角色(接收訊號)

發送訊號格式如下：

開: {"relay1": "on", "relay2": "on", "relay3": "on", "relay4": "on"}

關: {"relay1": "off", "relay2": "off", "relay3": "off", "relay4": "off"}

mqtt_server ip :20.194.175.52

port = 1883

user = cubie

password=1qaz2wsx

topic = "relay_control/teacher"

建立一個頁面，有四個按鈕，會顯示當下即時的狀態，假設當下為開，當使用者按關，即會關閉。

接收到其它設備的訊號格式如下：

開: {"relay1": "on", "relay2": "on", "relay3": "on", "relay4": "on"}

關: {"relay1": "off", "relay2": "off", "relay3": "off", "relay4": "off"}

mqtt_server ip :20.194.175.52

port = 1883

user = cubie

password=1qaz2wsx

topic = "relay_status/teacher"

將

mqtt_server ip :20.194.175.52

port = 1883

user = cubie

password=1qaz2wsx

topic = "relay_control/teacher"

topic = "relay_status/teacher"

資訊存到一個檔案 mqtt_config.py

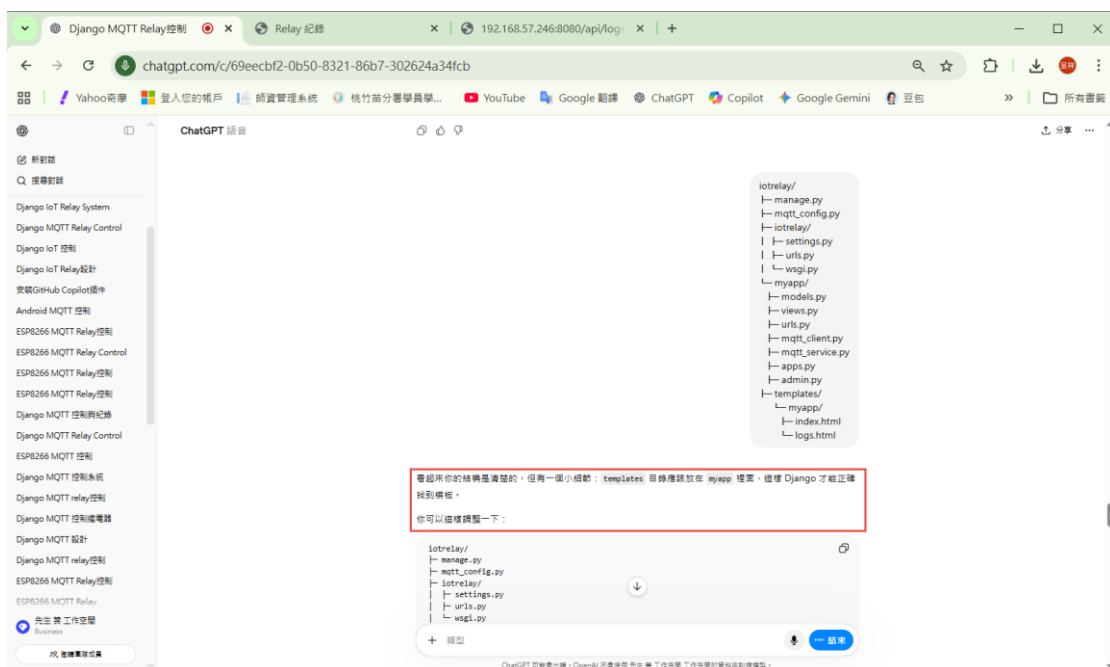
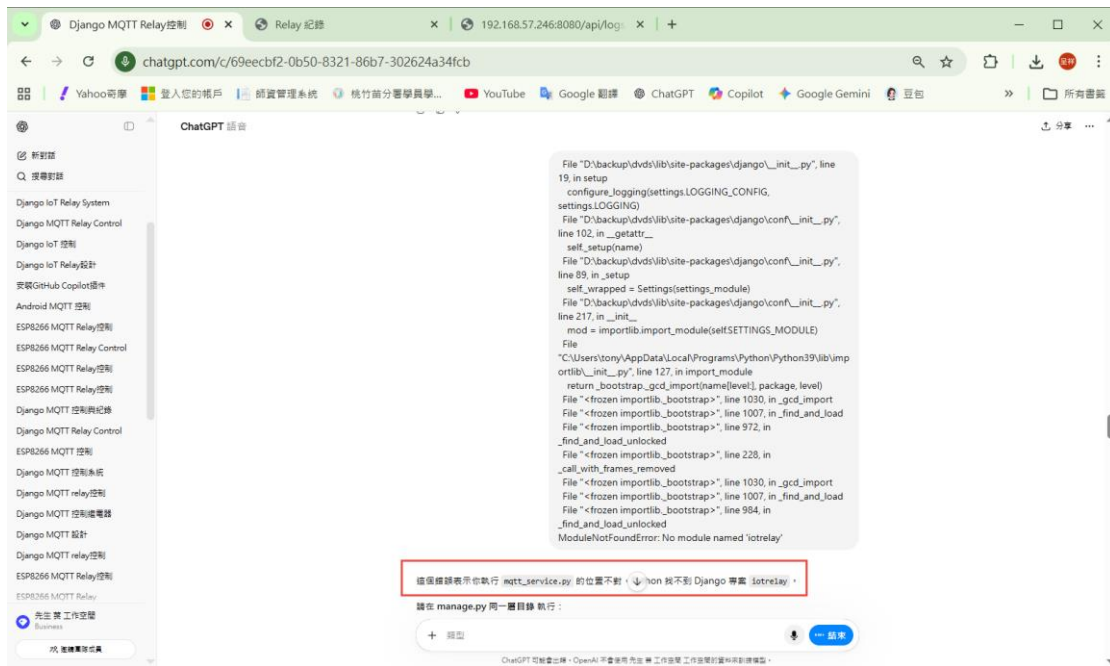
當使用者按下網頁開或關，或是透過 mqtt，topic = "relay_control/teacher"讓 relay 開或關，都將此紀錄到 sqlite 內，此紀錄會區分是網頁觸發(web)，或是其它設備(mqtt)，資料庫記錄，最多 1000 筆，若超過將舊的刪除，也建立一個頁面，可以觀看紀錄。

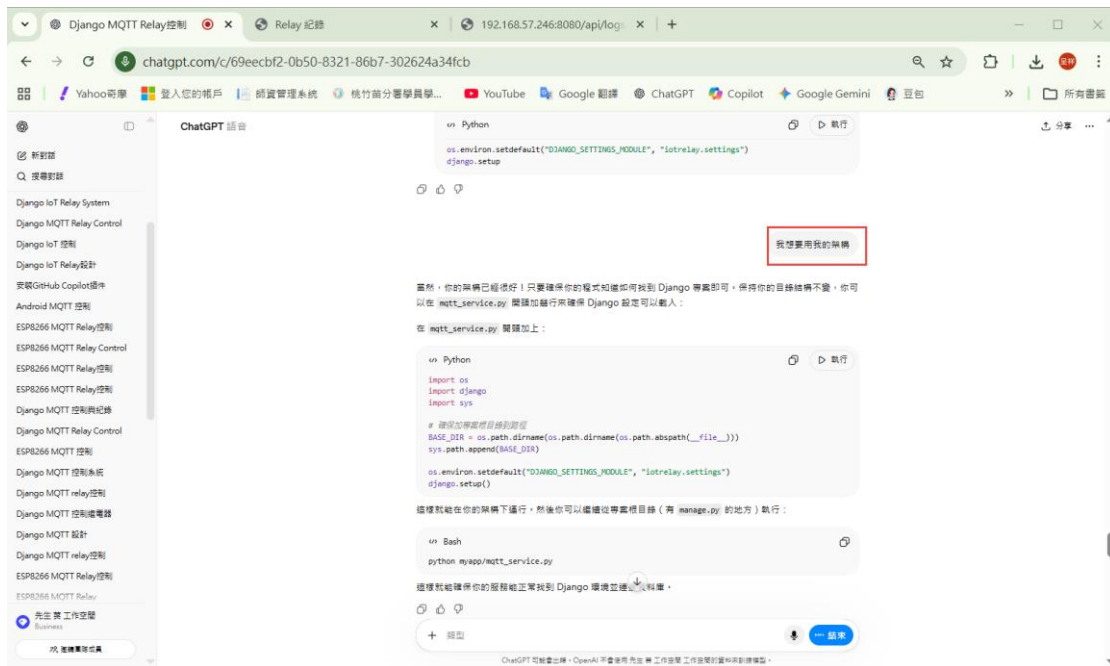
建立一個 api，可以將 relay 的紀錄，分享給其它 sensor，如手機程式使用。

前端可使用 bootstrap 與 jquery 撰寫，其中請將 mqtt_service 手動執行。

Django 專案名為 iotrelay，專案 app 名稱為 myapp，請給程式。

生成式 AI 範例: 建立 Django 控制 4 pin relay





請給我更改後的程式

好的，我們來調整一下。關鍵是在 Django 發送 MQTT 訊號時加上 "source" 標記，並在 MQTT 服務端接收時忽略這個標記。我會給你兩處主要改動。

第一，在 Django 發布訊號時，加上 "source": "web"。

第二，在 MQTT 的 on_message 事件中，檢查是否有 "source": "web"，如果有，就不記錄。

接下來是簡化後的改動範例：

1. 在 myapp/mqtt_client.py 中：

```
Python

import json
import paho.mqtt.client as mqtt
from mqtt_config import (
    MQTT_SERVER, MQTT_PORT, MQTT_USERNAME, MQTT_PASSWORD, MQTT_TOPIC_CONTROL
)

def publish_relay_control(data):
    client = mqtt.Client()
    client.username_pw_set(MQTT_USERNAME, MQTT_PASSWORD)
    client.connect(MQTT_SERVER, MQTT_PORT, 60)

    # 加上 source 標記
    data_with_source = data.copy()
    data_with_source["source"] = "web"

    payload = json.dumps(data_with_source)
    client.publish(MQTT_TOPIC_CONTROL, payload)

    client.disconnect()
```

ChatGPT 語音

```
Python
MQTT_SERVER, MQTT_PORT, MQTT_USERNAME, MQTT_PASSWORD, MQTT_TOPIC_CONTROL
)

def publish_relay_control(data):
    client = mqtt.Client()
    client.username_pw_set(MQTT_USERNAME, MQTT_PASSWORD)
    client.connect(MQTT_SERVER, MQTT_PORT, 60)

    # 加上 source 標記
    data_with_source = data.copy()
    data_with_source["source"] = "web"

    payload = json.dumps(data_with_source)
    client.publish(MQTT_TOPIC_CONTROL, payload)

    client.disconnect()
```

2. 在 `myapp/mqtt_service.py` 中：

```
Python
def on_message(client, userdata, msg):
    try:
        payload = msg.payload.decode("utf-8")
        data = json.loads(payload)

        # 如果有 source 標記是 web，跳過記錄
        if data.get("source") == "web":
            print("跳過來自 web 的記錄")
            return

        update_state(data)

    if msg.topic == MQTT_TOPIC_CONTROL:
        save_log(data, "mqtt", MQTT_TOPIC_CONTROL)
    elif msg.topic == MQTT_TOPIC_STATUS:
        update_state(data)
    except Exception as e:
        print("MQTT message error:", e)
```

這樣一調整，網頁觸發的指令只會記錄一次，其他設備的指令仍會正常記錄。





12. 建立資料庫

Bash

```
python manage.py makemigrations
python manage.py migrate
```

13. 執行 Django

Bash

```
python manage.py runserver 0.0.0.0:8000
```

14. 手動執行 MQTT Service

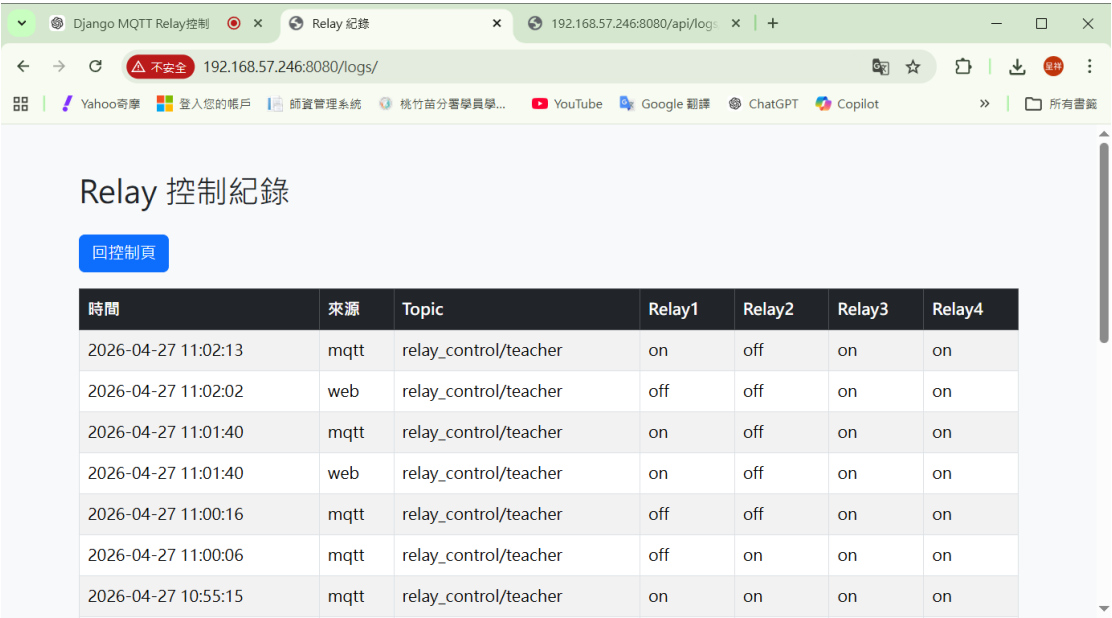
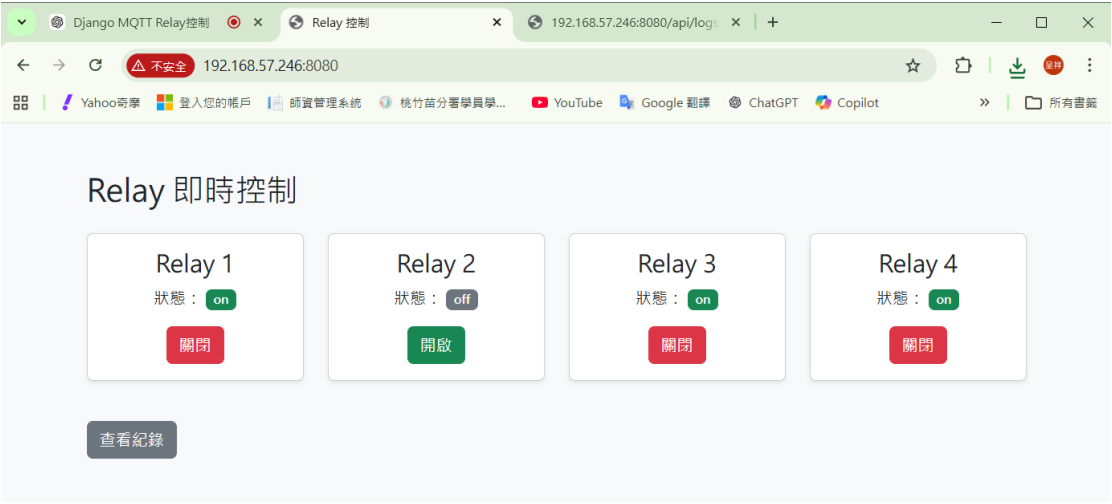
另開一個終端機，在 `manage.py` 同一層執行：

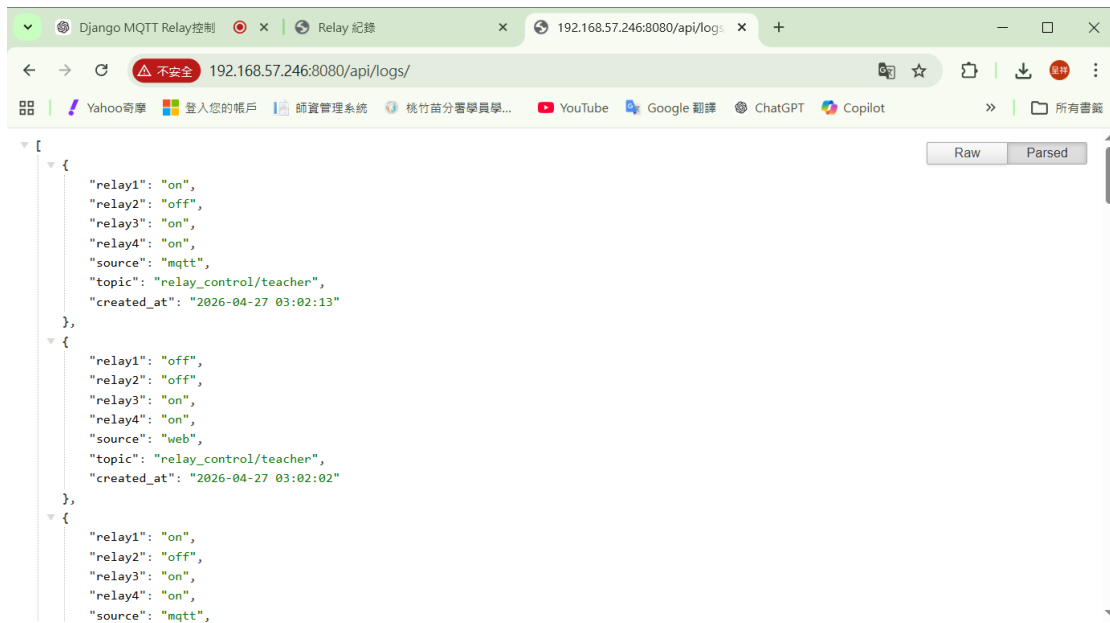
Bash

```
python myapp/mqtt_service.py
```

這樣 Django 網頁會負責發布控制訊號，MQTT service 會負責接收 `relay_status/teacher` 和 `relay_control/teacher` 的狀態更新。

結果如下:

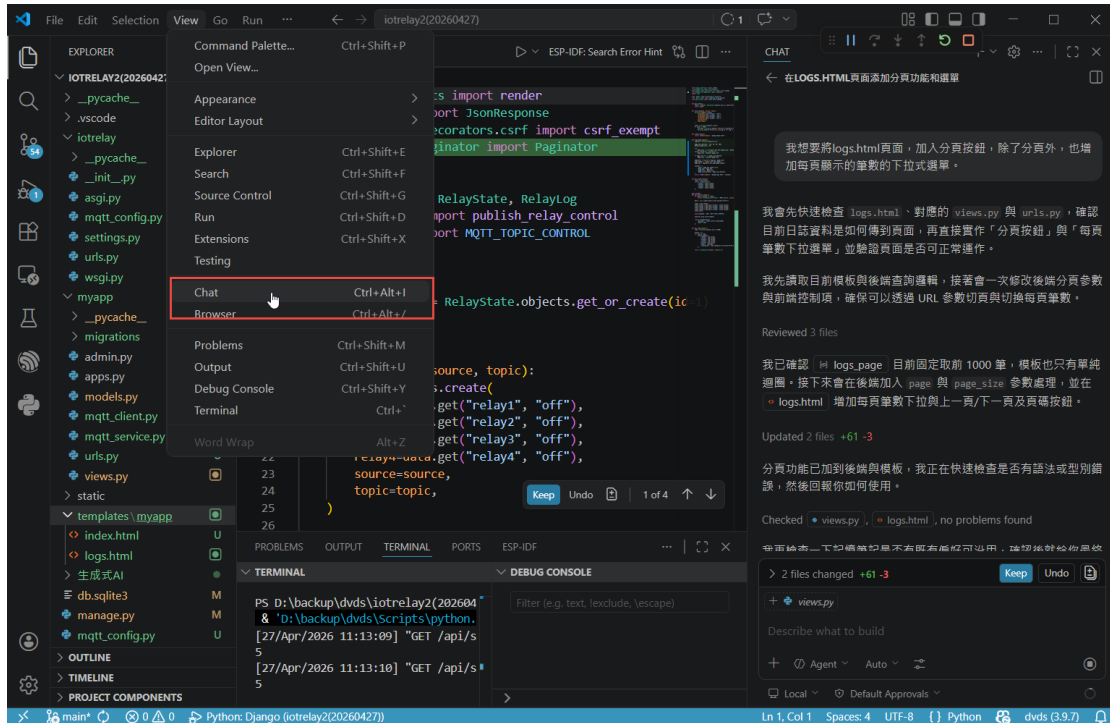


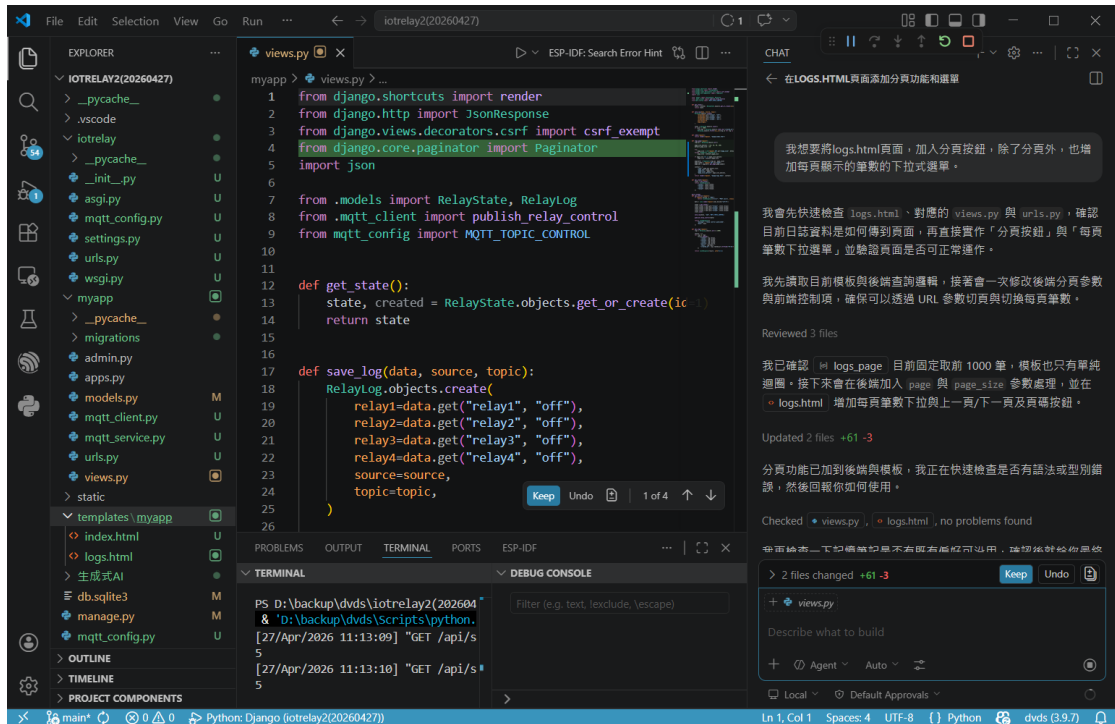


範例: iotrelay

尋問 by github copilot chat:

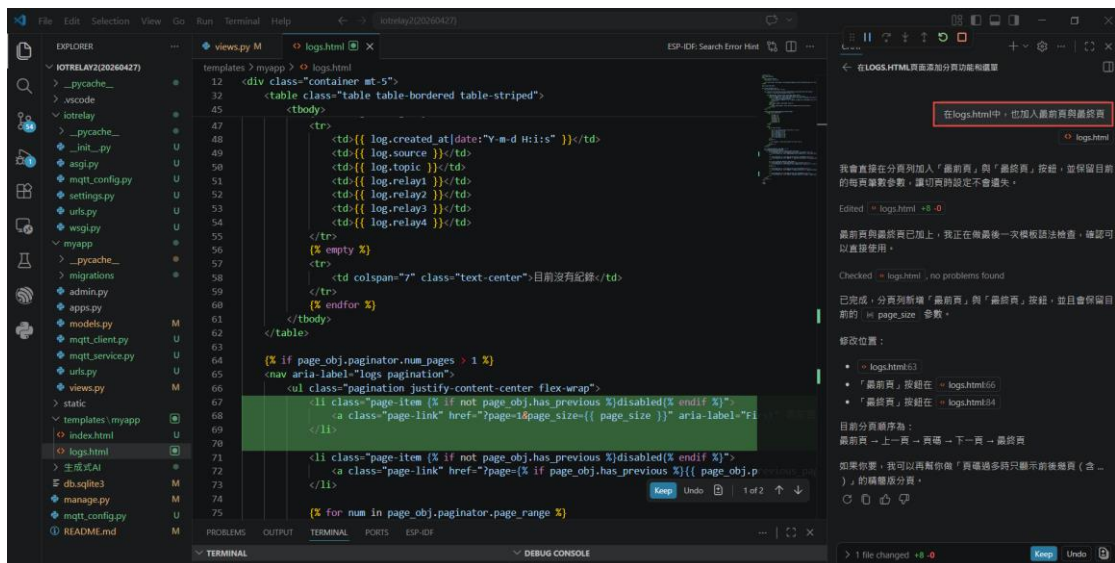
我想要將 logs.html 頁面，加入分頁按鈕，除了分頁外，也增加每頁顯示的筆數的下拉式選單。





尋問 by github copilot chat:

在 logs.html 中，也加入最前頁與最終頁



結果如下：

Django MQTT Relay 控制

Relay 紀錄

Relay 控制

192.168.57.246:8080/logs/?page=1&page_size=10

Yahoo 奇摩登入您的帳戶師資管理系統桃竹苗分署學員學...YouTubeGoogle 翻譯ChatGPTCopilotGoogle Gemini豆包所有書籤

Relay 控制紀錄

回控制頁

每頁顯示10

時間	來源	Topic	Relay1	Relay2	Relay3	Relay4
2026-04-27 11:02:13	mqtt	relay_control/teacher	on	off	on	on
2026-04-27 11:02:02	web	relay_control/teacher	off	off	on	on
2026-04-27 11:01:40	mqtt	relay_control/teacher	on	off	on	on
2026-04-27 11:01:40	web	relay_control/teacher	on	off	on	on
2026-04-27 11:00:16	mqtt	relay_control/teacher	off	off	on	on
2026-04-27 11:00:06	mqtt	relay_control/teacher	off	on	on	on
2026-04-27 10:55:15	mqtt	relay_control/teacher	on	on	on	on
2026-04-27 10:55:03	mqtt	relay_control/teacher	off	off	on	on
2026-04-27 10:53:58	mqtt	relay_control/teacher	off	off	on	on
2026-04-27 10:53:57	web	relay_control/teacher	off	off	on	on

最前頁上一頁123下一頁最終頁

結果如下:

Relay 操作紀錄

Relay 操作紀錄

192.168.57.246:8080/logs/?per_page=10

Yahoo 奇摩登入您的帳戶師資管理系統桃竹苗分署學員學...YouTubeGoogle 翻譯ChatGPTCopilot所有書籤

Relay 操作紀錄

回控制頁

每頁顯示10筆

時間	Relay	狀態	來源
2026-04-24 14:22:25	relay1	off	web
2026-04-24 14:22:24	relay2	off	web
2026-04-24 14:22:23	relay4	off	web
2026-04-24 14:22:22	relay3	off	web
2026-04-24 14:22:18	relay4	on	web
2026-04-24 14:22:17	relay3	on	web
2026-04-24 14:22:16	relay2	on	web
2026-04-24 14:22:15	relay1	on	mqtt
2026-04-24 14:22:15	relay1	off	mqtt
2026-04-24 14:22:15	relay1	on	web

上一頁123下一頁

範例: iotrelay2

6-10 建立 Android 控制 4 pin relay

esp8266 當作 subscribe 角色同時也是 published 角色

subscribe 角色(接收訊號，將 relay 開或關):

接收到其它設備的訊號格式如下:

開: {"relay1":"on","relay2":"on","relay3":"on", "relay4":"on"}

關: {"relay1":"off","relay2":"off","relay3":"off", "relay4":"off"}

mqtt_server ip :20.194.175.52

port = 1883

user = cubie

password=1qaz2wsx

topic = "relay_control/teacher"

published 角色(將現階段的四個 relay 開與關發送出去)

發送訊號格式如下:

開: {"relay1":"on","relay2":"on","relay3":"on", "relay4":"on"}

關: {"relay1":"off","relay2":"off","relay3":"off", "relay4":"off"}

mqtt_server ip :20.194.175.52

port = 1883

user = cubie

password=1qaz2wsx

topic = "relay_status/teacher"

我想要設計一個 Andoird 手機程式，語言為為 kotlin，他是 published 角色(發送訊號)也是，subscribe 角色(接收訊號)

發送訊號格式如下:

開: {"relay1":"on","relay2":"on","relay3":"on", "relay4":"on"}

關: {"relay1":"off","relay2":"off","relay3":"off", "relay4":"off"}

mqtt_server ip :20.194.175.52

port = 1883

user = cubie

password=1qaz2wsx

topic = "relay_control/teacher"

接收到其它設備的訊號格式如下:

開: {"relay1":"on","relay2":"on","relay3":"on", "relay4":"on"}

關: {"relay1":"off","relay2":"off","relay3":"off", "relay4":"off"}

mqtt_server ip :20.194.175.52

```
port = 1883
user = cubie
password=1qaz2wsx
topic = "relay_status/teacher"
```

設計一個檔案，可以存儲，以方便管理

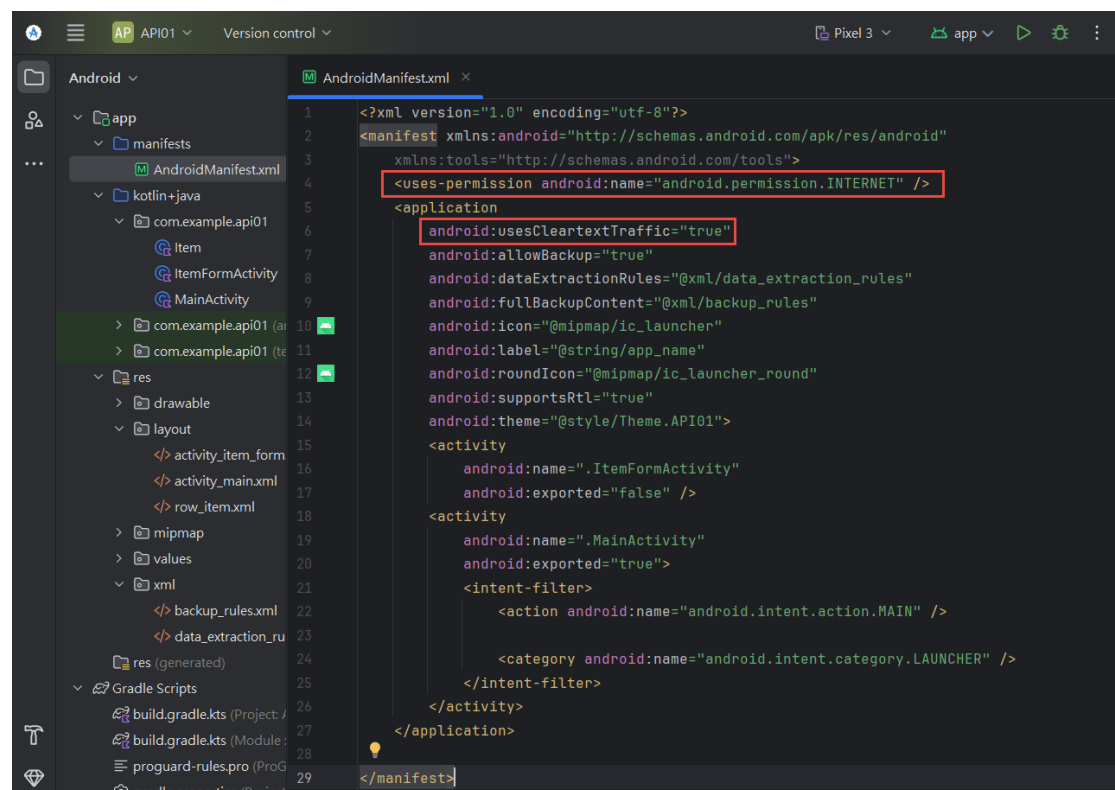
```
mqtt_server ip :20.194.175.52
port = 1883
user = cubie
password=1qaz2wsx
topic = "relay_status/teacher"
topic = "relay_status/teacher"
```

建立一個頁面，有四個按鈕，標題為: Relay 即時控制、四個按鈕有名稱 relay1、狀態 on or off，按鈕上會顯示關閉或開啟，讓使用者控制。

Android 專案名為 Iotrelay，請給程式。

生成式 AI 範例: 建立 Django 控制 4 pin relay

設定權限：



讓 App 能使用網路存取權限:

<manifest>

<uses-permission android:name="android.permission.INTERNET" />

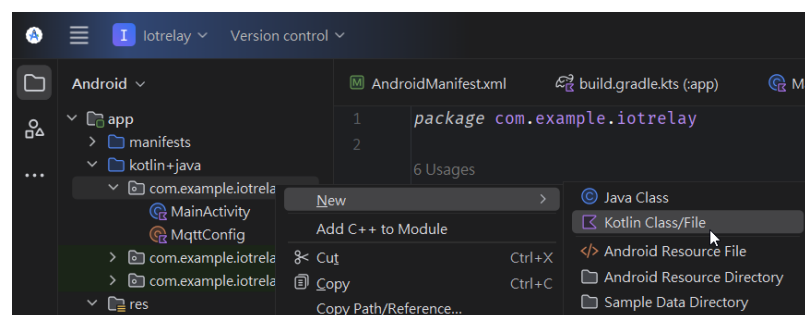
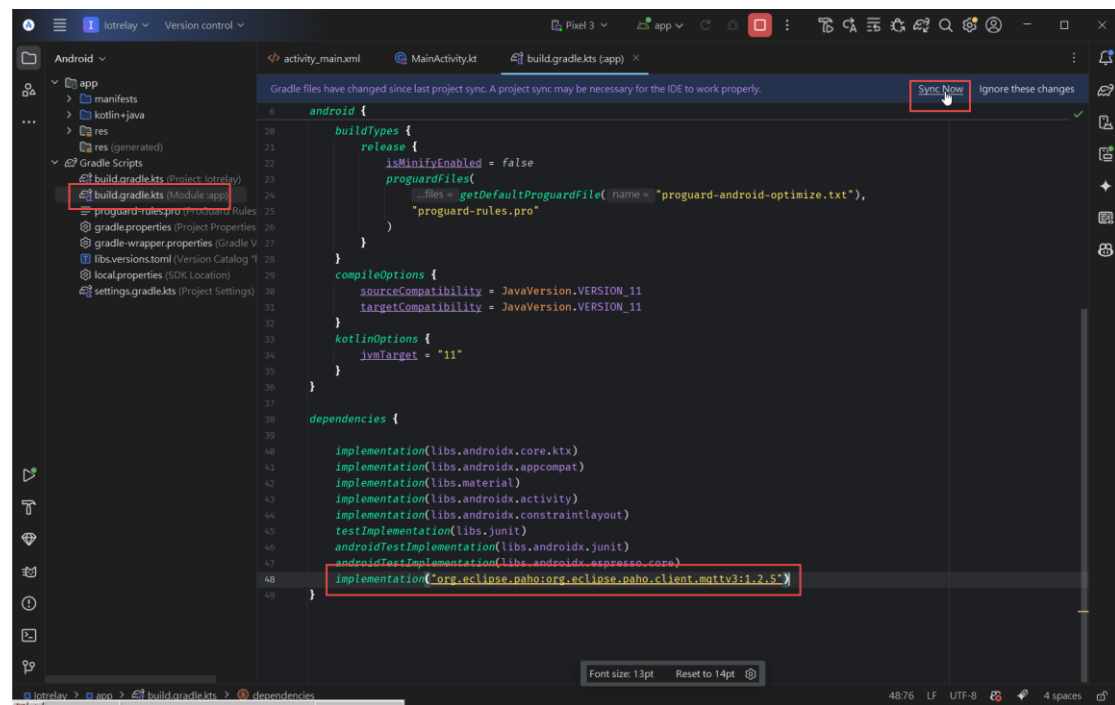
</manifest>

允許 HTTP 明文請求（避免被系統封鎖）:

<application>

android:usesCleartextTraffic="true"

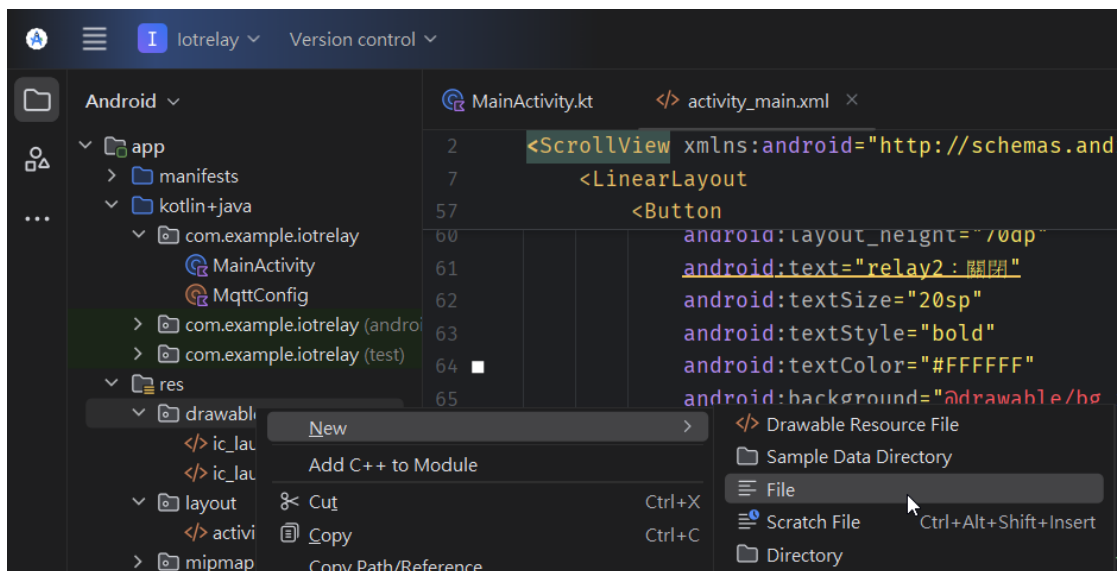
</application>

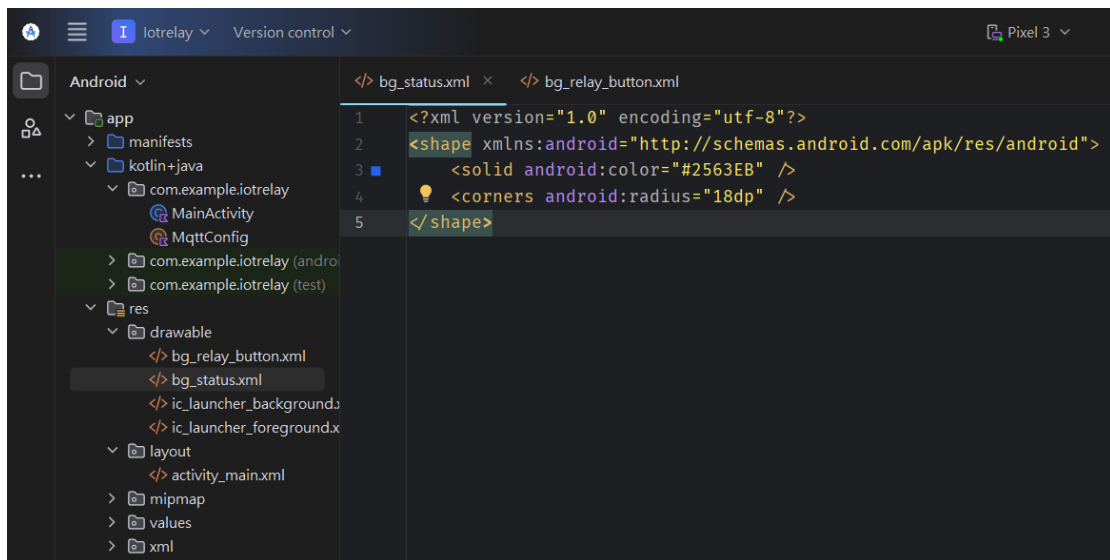
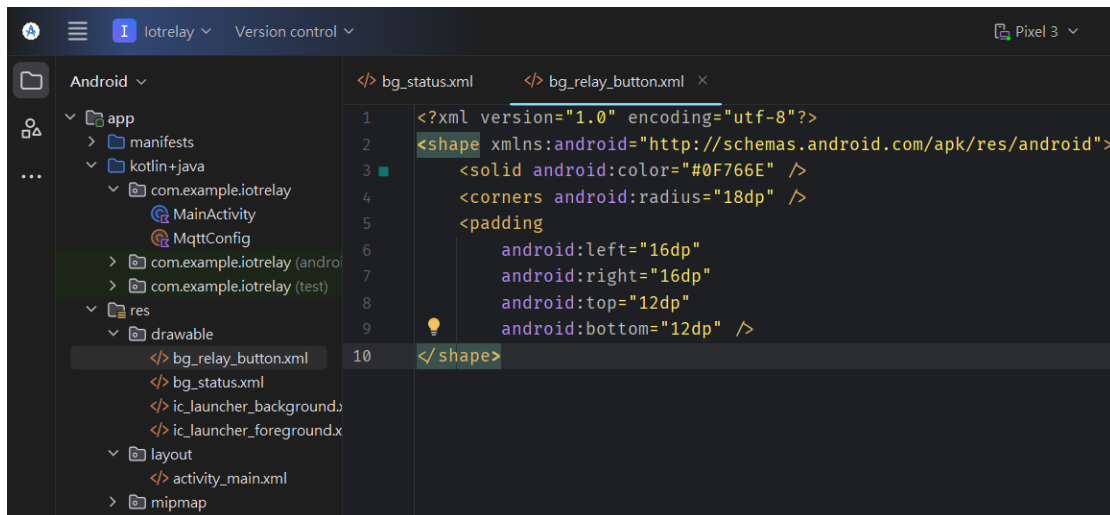




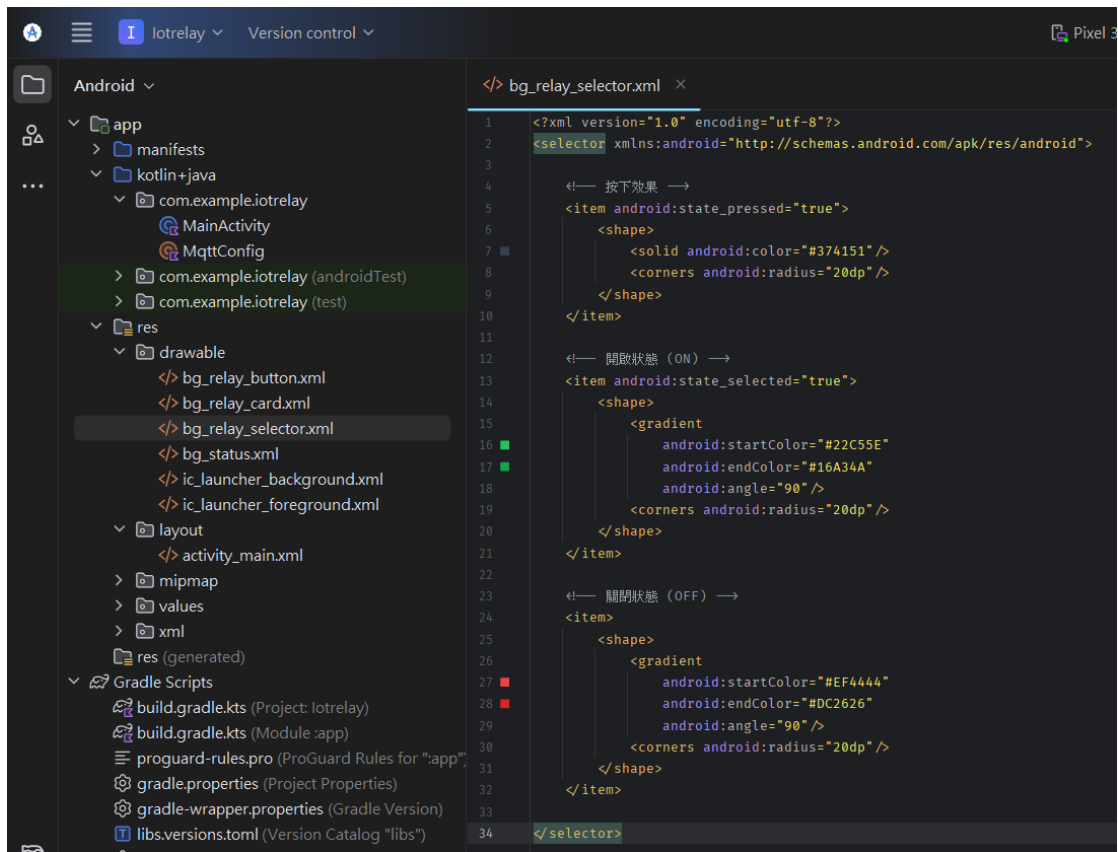
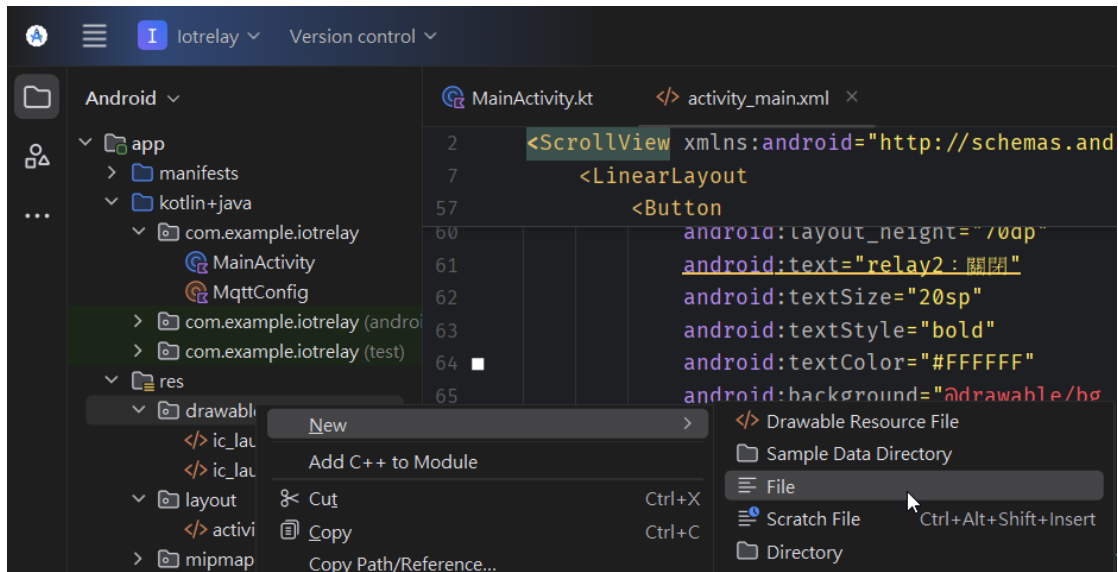
提問:

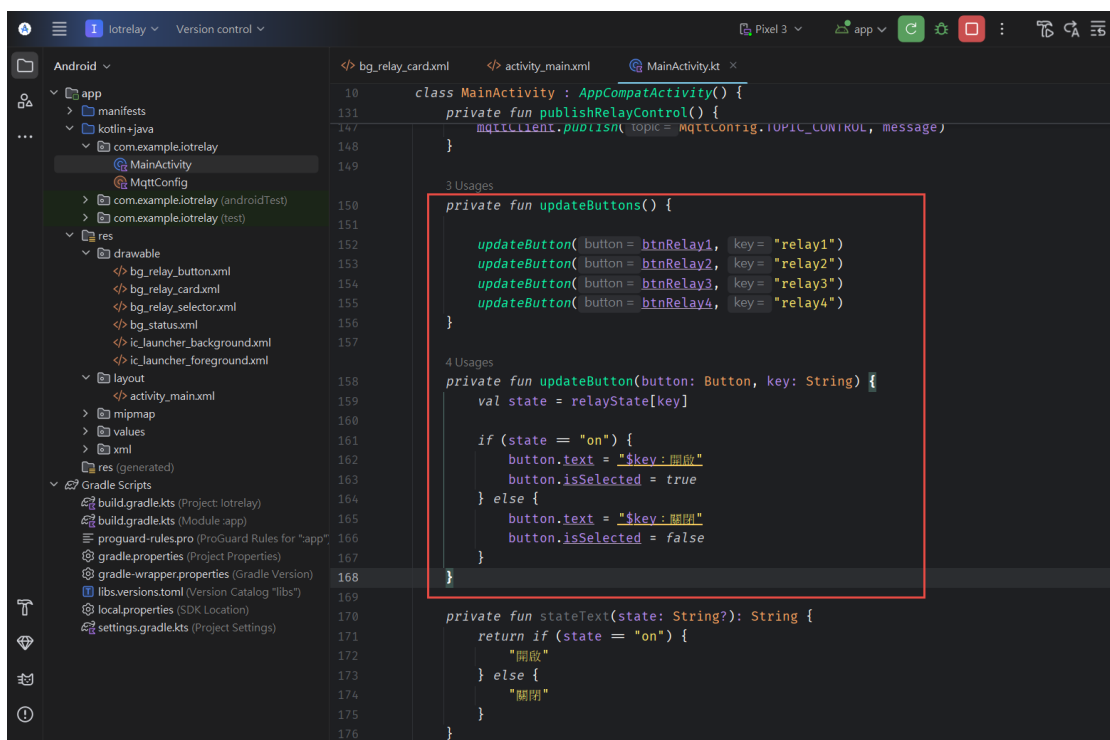
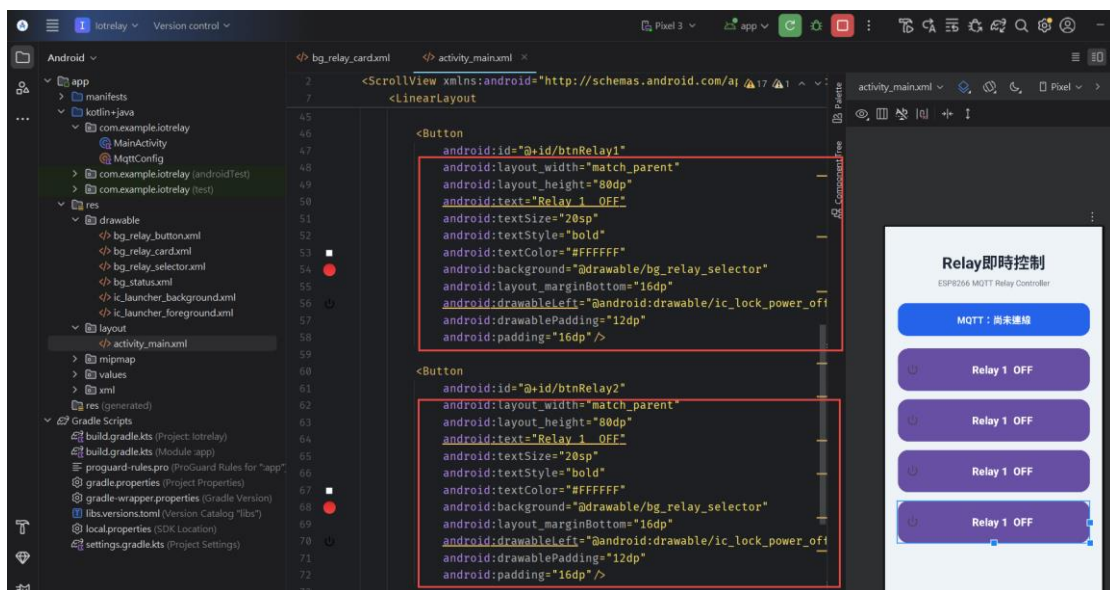
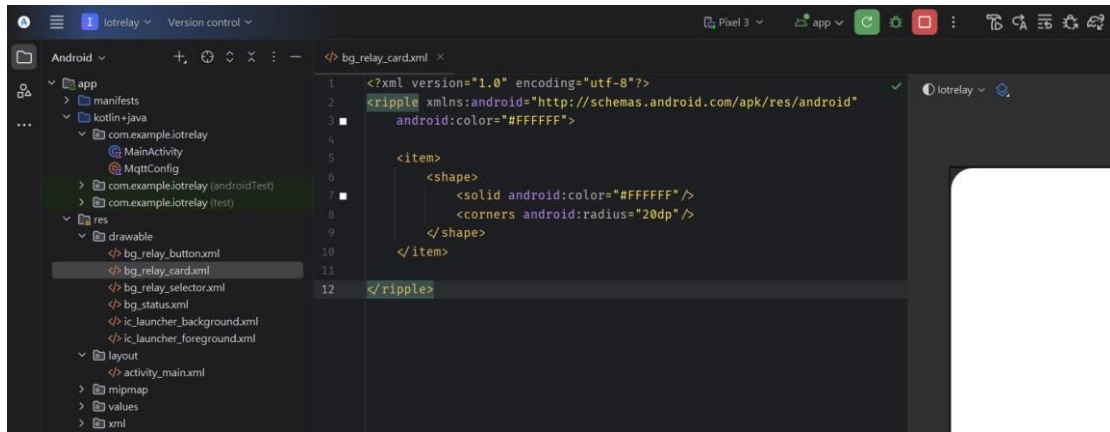
請將 activity_main.xml 畫面美化





提問：
將按鈕再美化





結果如下：



範例: lotrelay

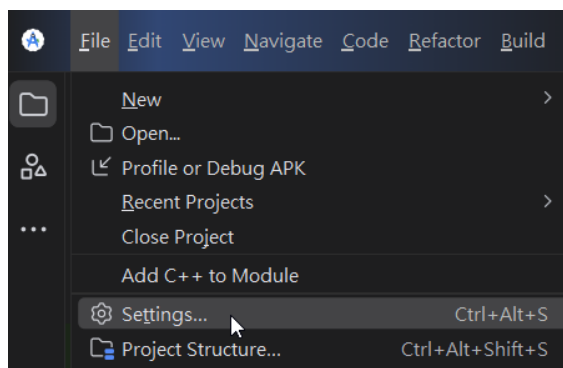
6-11 建立 Android 控制 4 pin relay

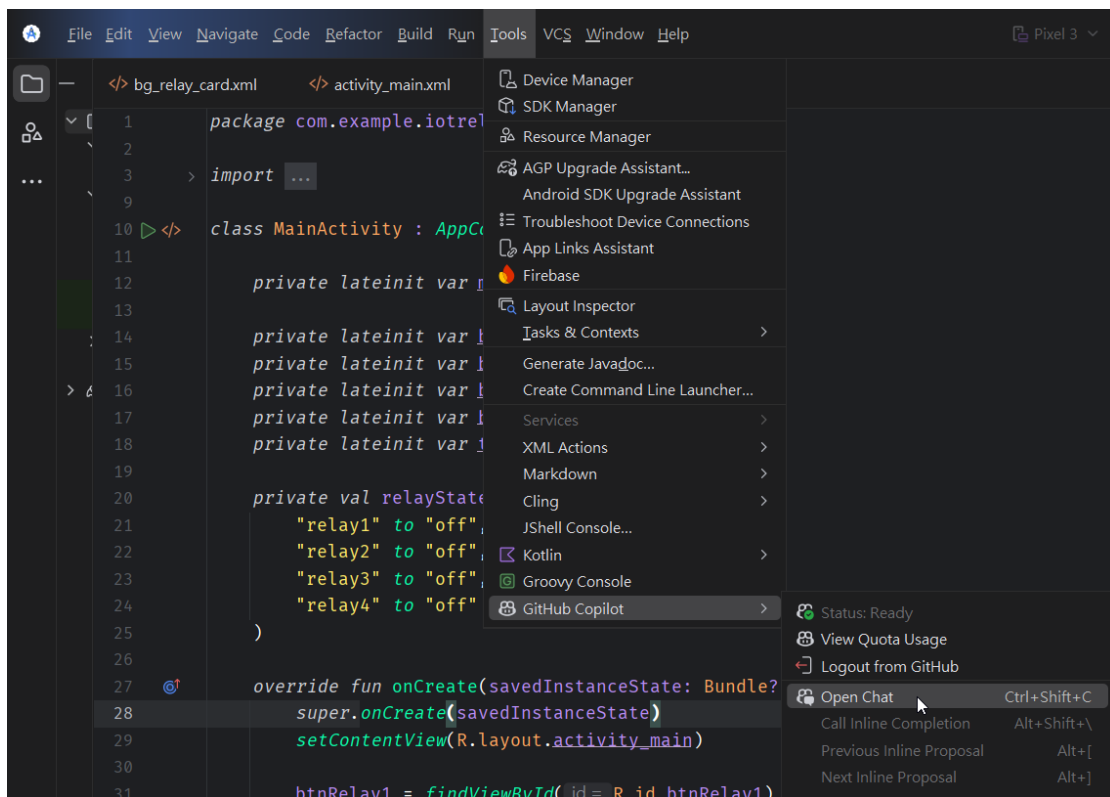
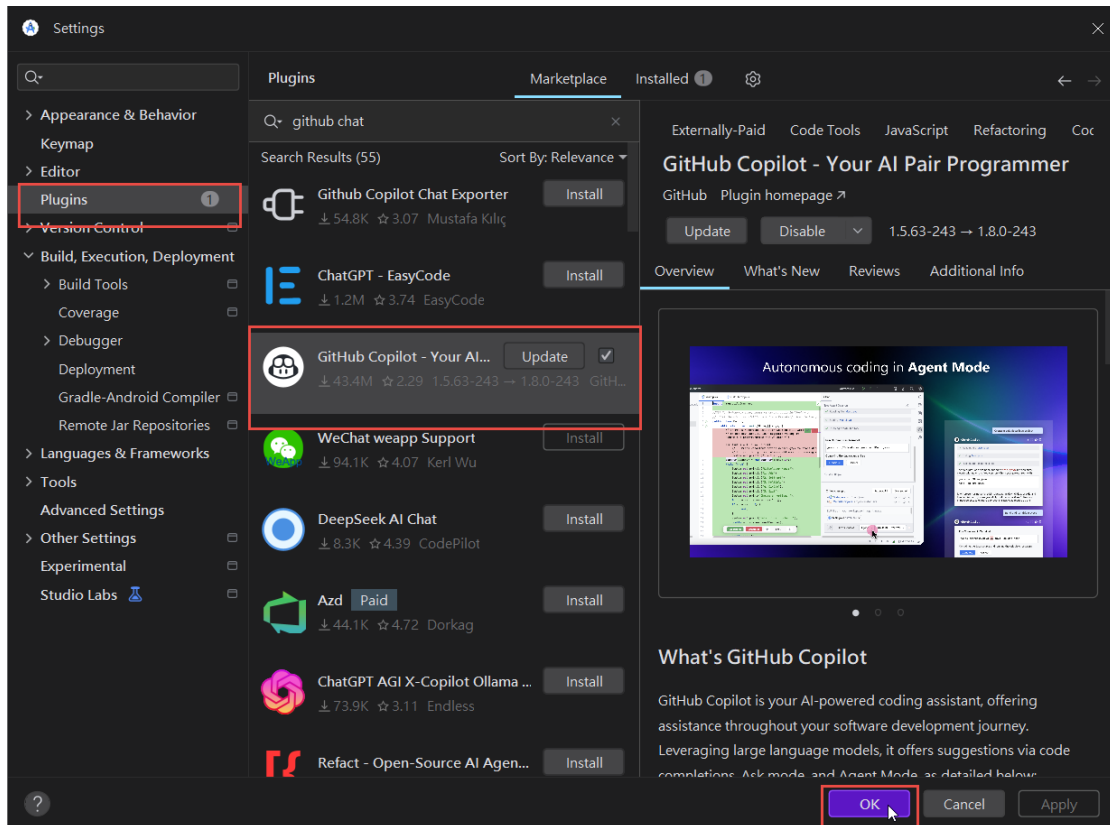
目標：

將 Android 利用 github copilot chat，加入兩個功能，一為控制 relay，二為顯示歷史資料。

安裝 GitHub Copilot 插件

- 1、開啟 Android Studio
- 2、點擊上方選單：
 - Windows / Linux：File > Settings
 - macOS：Android Studio > Preferences
- 3、在左側選單中選擇：Plugins（外掛）
- 4、在右上角搜尋欄輸入：GitHub Copilot
- 5、找到插件後，點擊：Install（安裝）
- 6、安裝完成後，重新啟動 Android Studio





尋問 by github copilot chat:

api web:http://192.168.57.246:8080/api/logs/

內容格式如下：

```
[{"relay1": "on","relay2": "off","relay3": "on","relay4": "on","source": "mqtt","topic": "relay_control/teacher","created_at": "2026-04-27 03:02:13"}, {"relay1": "off","relay2": "off","relay3": "on","relay4": "on","source": "web","topic": "relay_control/teacher","created_at": "2026-04-27 03:02:02"}]
```

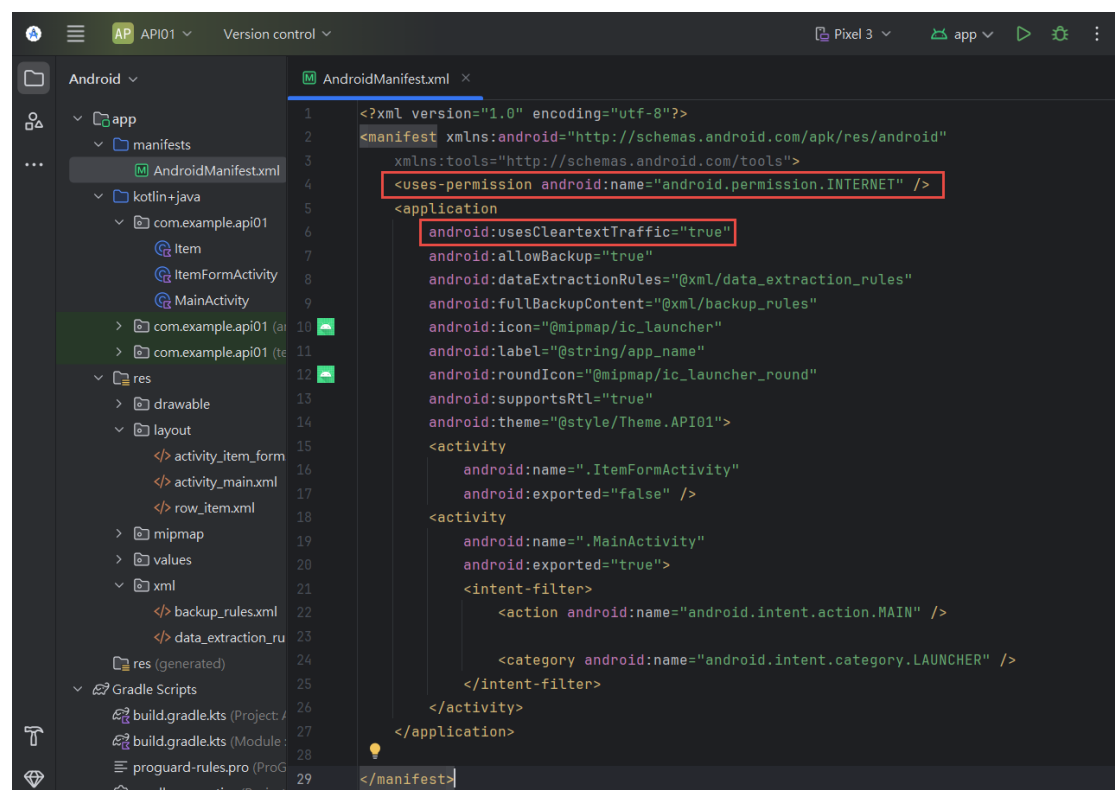
資料可能會有許多筆

我想要將此專案分成兩個分頁，一是原本的 relay 控制，二是查看 relay 歷史資料查看，幫我修正程式，若有修正或新增的程式，都再一次給我完整程式。最後印出，有要新增檔案的位置與名稱。

分頁 relay 控制，按鈕關底色為綠色、開底色為紅色，會自動更新 relay_status/teacher 的狀態

分頁 relay 歷史資料，請將其分頁美化，下拉時，會重新整理將最新資料載入，其中 off 字顏色為綠色、on 字顏色為紅色。

設定權限：



讓 App 能使用網路存取權限：

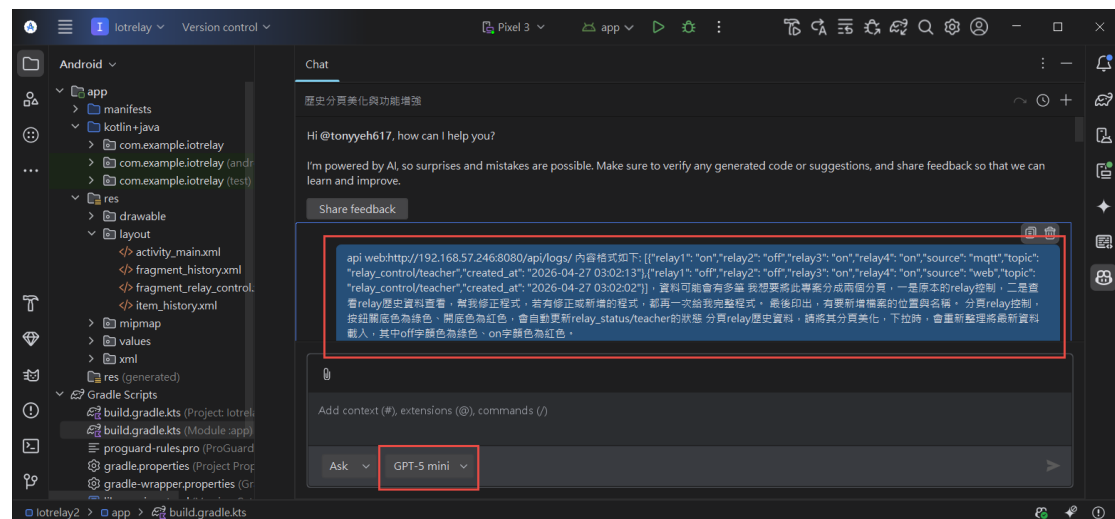
<manifest>

```
<uses-permission android:name="android.permission.INTERNET" />
</manifest>
```

允許 HTTP 明文請求（避免被系統封鎖）：

```
<application>
android:usesCleartextTraffic="true"
</application>
```

尋問 by github copilot chat:



新增或修改檔案清單：

- 修改：src/main/java/com/example/iotrelay/MainActivity.kt
- 新增：src/main/java/com/example/iotrelay/MainPagerAdapter.kt
- 新增：src/main/java/com/example/iotrelay/MqttConfig.kt
- 新增：src/main/java/com/example/iotrelay/RelayControlFragment.kt
- 新增：src/main/java/com/example/iotrelay/HistoryFragment.kt
- 新增：src/main/java/com/example/iotrelay/HistoryAdapter.kt
- 新增：src/main/java/com/example/iotrelay/LogEntry.kt
- 修改/新增 layout：src/main/res/layout/activity_main.xml
- 新增：src/main/res/layout/fragment_relay_control.xml
- 新增：src/main/res/layout/fragment_history.xml
- 新增：src/main/res/layout/item_history.xml

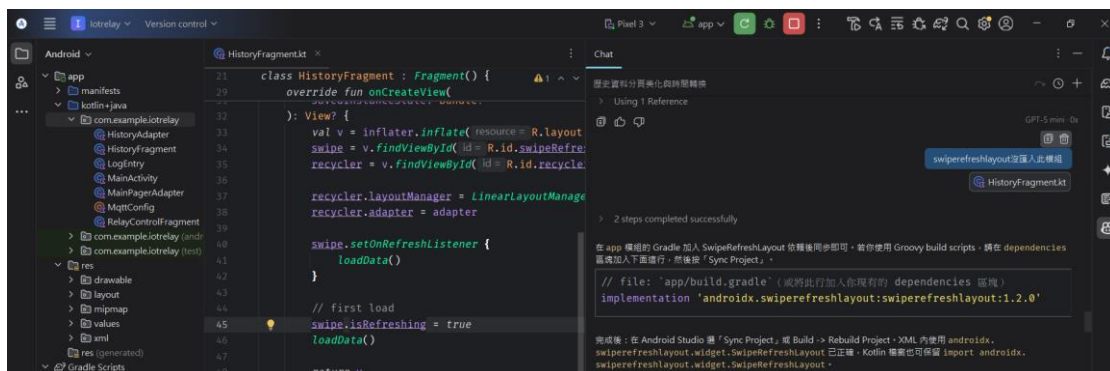
完成上述檔案後，請：

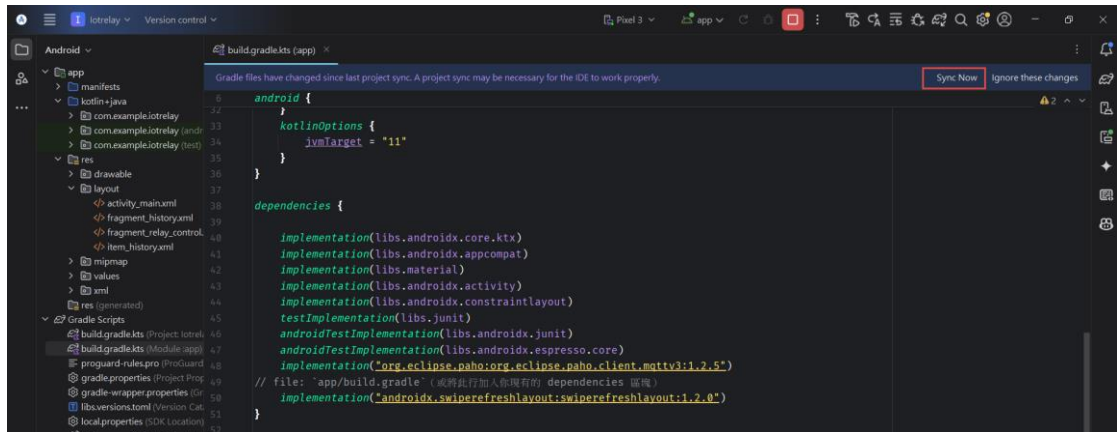
1. 在 `MqttConfig.SERVER_URI` 填入你的 MQTT broker 地址。
2. 確認 `AndroidManifest.xml` 有 `<uses-permission android:name="android.permission.INTERNET" />`。
3. 若未加入 `org.eclipse.paho` 的依賴，請在 `build.gradle` 加入 MQTT 的依賴。

結果如下：

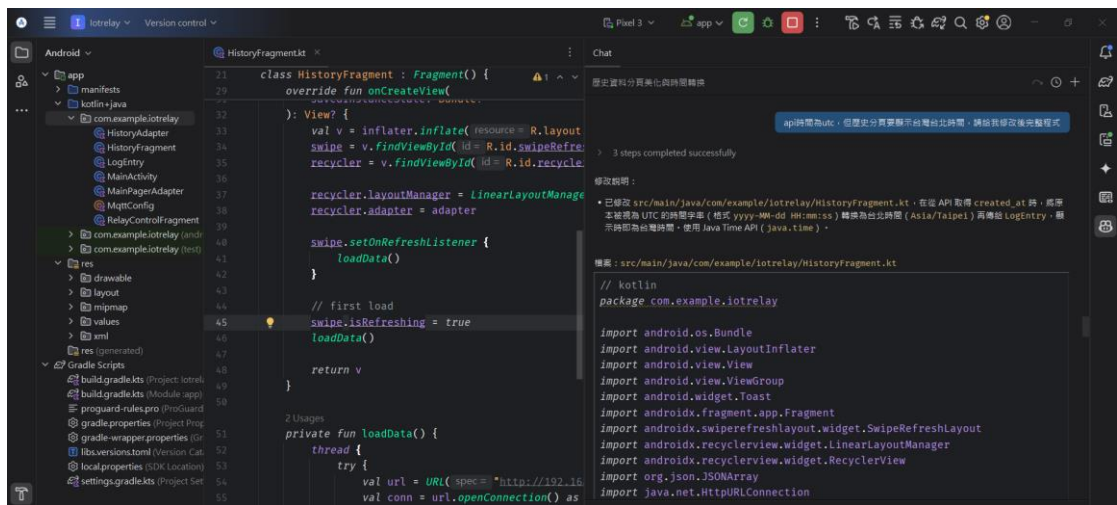


尋問 by github copilot chat:

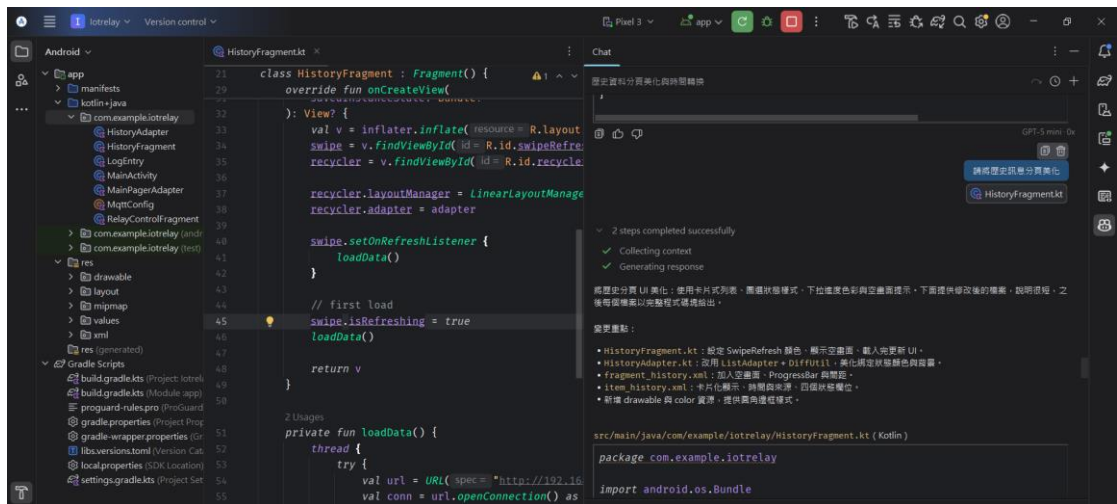




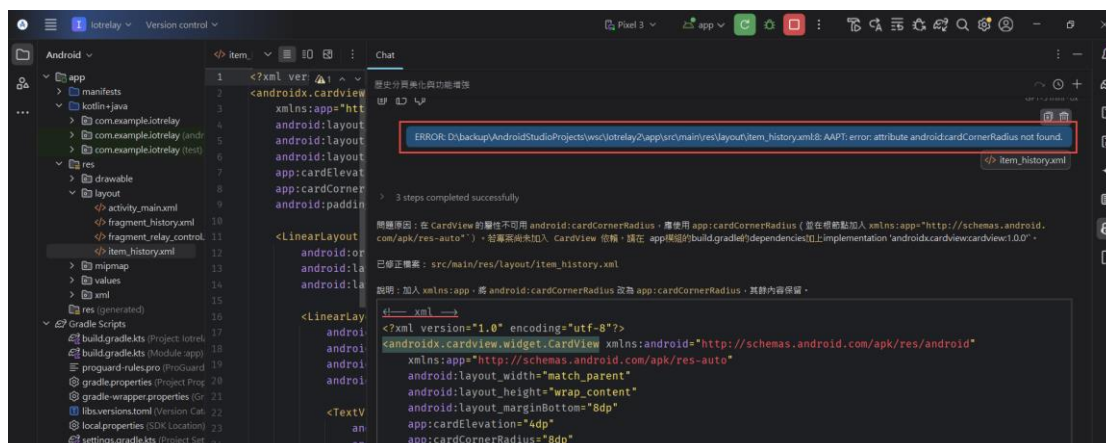
尋問 by github copilot chat:



尋問 by github copilot chat:



尋問 by github copilot chat:



結果如下：



範例: lotrelay2